



NEVER MISS BUS

COMPLETE SOURCE CODE

SRBS International School

Flutter (Dart) · Firebase · Cloud Functions (TypeScript)
Security Rules · Android & iOS Platform Config · Tests

14 sections · ~8,000 lines of code
flutter analyze: 0 issues · 15/15 tests passing

v1.0 · August 2026

Table of Contents

Section	Contents	Size
1. Project Configuration	Dependencies, lint rules and app entry point.	5 files · 206 lines
2. Design System & Core	Colors, typography, theme, constants, validators, formatters and the Result type.	8 files · 481 lines
3. Reusable Widgets	Shared UI components used by all three roles.	5 files · 441 lines
4. Data Models	Pure-Dart data classes (no Firebase imports).	9 files · 411 lines
5. Services	Firebase & platform gateways: auth, Firestore, live location, driver GPS trips, ETA, notifications, connectivity, admin Cloud Function calls.	8 files · 887 lines
6. State & Navigation	Riverpod provider graph and the role-fenced go_router.	3 files · 440 lines
7. Auth Screens	Splash and the single role-aware login.	2 files · 270 lines
8. Student App	Shell, home, live map, bus details, stops, alerts, profile, settings + shared widgets.	10 files · 1809 lines
9. Driver App	Shell, one-tap trip dashboard, route list, profile.	4 files · 720 lines
10. Admin App	Shell + all management screens.	11 files · 2275 lines
11. Firebase Backend — Security Rules	The real access control: Firestore rules + Realtime Database rules + indexes.	4 files · 230 lines
12. Cloud Functions (TypeScript)	Privileged backend: provisioning, claims, assignment, announcements, geofence alerts, stale-trip sweeper, audit logging.	3 files · 575 lines
13. Platform Configuration	Android manifest/gradle and iOS AppDelegate with key-injection (no secrets in source).	4 files · 168 lines
14. Tests	Unit tests: validators, freshness honesty, ETA guardrails, role parsing.	2 files · 163 lines

Note: lib/firebase_options.dart is intentionally a placeholder — real Firebase identifiers are generated per project by `flutterfire configure`. No API keys, passwords or private credentials appear anywhere in this document.

1. Project Configuration

Dependencies, lint rules and app entry point.

pubspec.yaml · analysis_options.yaml · main.dart · app.dart · firebase_options.dart

pubspec.yaml

47 lines

```

1 ■ name: never_miss_bus
2 ■ description: "Never Miss Bus – secure school bus live tracking for SRBS International School."
3 ■ publish_to: "none"
4 ■ version: 1.0.0+1
5 ■
6 ■ environment:
7 ■   sdk: ">=3.4.0 <4.0.0"
8 ■
9 ■ dependencies:
10 ■   flutter:
11 ■     sdk: flutter
12 ■
13 ■   # State management
14 ■   flutter_riverpod: ^2.5.1
15 ■
16 ■   # Navigation
17 ■   go_router: ^14.2.7
18 ■
19 ■   # Firebase (current stable, official FlutterFire)
20 ■   firebase_core: ^3.6.0
21 ■   firebase_auth: ^5.3.1
22 ■   cloud_firestore: ^5.4.4
23 ■   firebase_database: ^11.1.4
24 ■   firebase_messaging: ^15.1.3
25 ■   cloud_functions: ^5.1.3
26 ■
27 ■   # Maps & location
28 ■   google_maps_flutter: ^2.9.0
29 ■   geolocator: ^13.0.1
30 ■   flutter_polyline_points: ^2.1.0
31 ■
32 ■   # Platform / UX
33 ■   permission_handler: ^11.3.1
34 ■   connectivity_plus: ^6.0.5
35 ■   flutter_local_notifications: ^17.2.3
36 ■   intl: ^0.19.0
37 ■   cupertino_icons: ^1.0.8
38 ■
39 ■ dev_dependencies:
40 ■   flutter_test:
41 ■     sdk: flutter
42 ■   flutter_lints: ^4.0.0
43 ■
44 ■ flutter:
45 ■   uses-material-design: true
46 ■   assets:
47 ■     - assets/images/

```

analysis_options.yaml

9 lines

```

1 ■ include: package:flutter_lints/flutter.yaml
2 ■
3 ■ linter:
4 ■   rules:
5 ■     prefer_const_constructors: true
6 ■     prefer_final_locals: true
7 ■     avoid_print: true
8 ■     require_trailing_commas: true
9 ■     always_declare_return_types: true

```

lib/main.dart

48 lines

```

1 ■ import 'package:firebase_core/firebase_core.dart';
2 ■ import 'package:flutter/material.dart';
3 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
4 ■
5 ■ import 'app.dart';
6 ■ import 'firebase_options.dart';
7 ■
8 ■ Future<void> main() async {
9 ■   WidgetsFlutterBinding.ensureInitialized();
10 ■
11 ■   // Firebase configuration is generated per project by `flutterfire
12 ■   // configure` (see docs/SETUP.md). Fails fast with a clear message if the
13 ■   // project hasn't been configured yet.
14 ■   try {
15 ■     await Firebase.initializeApp(
16 ■       options: DefaultFirebaseOptions.currentPlatform,
17 ■     );
18 ■   } on UnsupportedError catch (e) {
19 ■     debugPrint('FIREBASE NOT CONFIGURED: $e');
20 ■     runApp(const _ConfigErrorApp());
21 ■     return;
22 ■   }
23 ■
24 ■   runApp(const ProviderScope(child: NeverMissBusApp()));
25 ■ }
26 ■
27 ■ class _ConfigErrorApp extends StatelessWidget {
28 ■   const _ConfigErrorApp();
29 ■
30 ■   @override
31 ■   Widget build(BuildContext context) {
32 ■     return const MaterialApp(
33 ■       home: Scaffold(
34 ■         body: Center(
35 ■           child: Padding(
36 ■             padding: EdgeInsets.all(32),
37 ■             child: Text(
38 ■               'Firebase is not configured for this build.\n\n'
39 ■               'Run: flutterfire configure\n'
40 ■               'See docs/SETUP.md for the full checklist.',
41 ■               textAlign: TextAlign.center,
42 ■             ),
43 ■           ),
44 ■         ),
45 ■       ),
46 ■     );
47 ■   }
48 ■ }

```

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■ import 'package:go_router/go_router.dart';
4 ■
5 ■ import 'core/constants/nmb_constants.dart';
6 ■ import 'core/theme/nmb_theme.dart';
7 ■ import 'providers/app_providers.dart';
8 ■ import 'router/app_router.dart';
9 ■ import 'services/auth_service.dart';
10 ■
11 ■ class NeverMissBusApp extends ConsumerStatefulWidget {
12 ■   const NeverMissBusApp({super.key});
13 ■
14 ■   @override
15 ■   ConsumerState<NeverMissBusApp> createState() => _NeverMissBusAppState();
16 ■ }
17 ■
18 ■ class _NeverMissBusAppState extends ConsumerState<NeverMissBusApp>
19 ■   with WidgetsBindingObserver {
20 ■   @override
21 ■   void initState() {
22 ■     super.initState();
23 ■     WidgetsBinding.instance.addObserver(this);
24 ■     // Initialize notifications once (permission prompt + channels).
25 ■     Future<void>.microtask(
26 ■       () => ref.read(notificationServiceProvider).initialize(),
27 ■     );
28 ■   }
29 ■
30 ■   @override
31 ■   void dispose() {
32 ■     WidgetsBinding.instance.removeObserver(this);
33 ■     super.dispose();
34 ■   }
35 ■
36 ■   @override
37 ■   void didChangeAppLifecycleState(AppLifecycleState state) {
38 ■     // On resume, force-refresh the ID token so revoked roles / disabled
39 ■     // accounts lose access promptly (session hardening).
40 ■     if (state == AppLifecycleState.resumed) {
41 ■       ref.read(authServiceProvider).readSession(forceRefresh: true);
42 ■     }
43 ■   }
44 ■
45 ■   @override
46 ■   Widget build(BuildContext context) {
47 ■     final GoRouter router = ref.watch(appRouterProvider);
48 ■
49 ■     // Register FCM device token whenever a session becomes available.
50 ■     ref.listen(sessionProvider, (Object? prev, Object? next) {
51 ■       final AuthSession? session =
52 ■         (next as AsyncValue<AuthSession?>?).valueOrNull;
53 ■       if (session != null) {
54 ■         ref.read(notificationServiceProvider).registerDevice(session.uid);
55 ■       }
56 ■     });
57 ■
58 ■     return MaterialApp.router(
59 ■       title: NmbConstants.appName,
60 ■       debugShowCheckedModeBanner: false,
61 ■       theme: NmbTheme.light(),
62 ■       routerConfig: router,
63 ■     );
64 ■   }
65 ■ }

```

```

1 ■ //
2 ■ // PLACEHOLDER - MUST BE REPLACED BEFORE RUNNING.
3 ■ //
4 ■ // Generate the real file with the FlutterFire CLI:
5 ■ //
6 ■ //   dart pub global activate flutterfire_cli
7 ■ //   flutterfire configure --project=<your-firebase-project-id>
8 ■ //
9 ■ // That command overwrites this file with your project's real (public)
10 ■ // Firebase app identifiers and also writes:
11 ■ //   android/app/google-services.json
12 ■ //   ios/Runner/GoogleService-Info.plist
13 ■ //
14 ■ // NOTE: FlutterFire "apiKey" values are app identifiers, not secrets - but
15 ■ // this repo intentionally ships none. Real secrets (service accounts, FCM
16 ■ // server credentials) live ONLY in Firebase/Cloud Functions, never here.
17 ■ //
18 ■ import 'package:firebase_core/firebase_core.dart' show FirebaseOptions;
19 ■ import 'package:flutter/foundation.dart'
20 ■     show defaultTargetPlatform, TargetPlatform;
21 ■
22 ■ class DefaultFirebaseOptions {
23 ■   static FirebaseOptions get currentPlatform {
24 ■     switch (defaultTargetPlatform) {
25 ■       case TargetPlatform.android:
26 ■       case TargetPlatform.iOS:
27 ■         throw UnsupportedError(
28 ■           'firebase_options.dart is a placeholder. '
29 ■           'Run `flutterfire configure` to generate real options.',
30 ■         );
31 ■       default:
32 ■         throw UnsupportedError(
33 ■           'Never Miss Bus targets Android and iOS only.',
34 ■         );
35 ■     }
36 ■   }
37 ■ }

```

2. Design System & Core

Colors, typography, theme, constants, validators, formatters and the Result type.

nmb_colors.dart · nmb_typography.dart · nmb_theme.dart · enums.dart · nmb_constants.dart · validators.dart · formatters.dart · result.dart

lib/core/theme/nmb_colors.dart

39 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■
3 ■ /// Never Miss Bus design-system palette.
4 ■ ///
5 ■ /// Friendly, school-appropriate, high-contrast. All pairs meet WCAG AA on
6 ■ /// their intended backgrounds.
7 ■ abstract final class NmbColors {
8 ■   // Brand
9 ■   static const Color primary = Color(0xFF2557D6); // trustworthy school blue
10 ■   static const Color primaryDark = Color(0xFF1A3FA0);
11 ■   static const Color primarySoft = Color(0xFFE8EEFC); // tinted card backgrounds
12 ■   static const Color accent = Color(0xFFFFB300); // school-bus amber
13 ■   static const Color accentDark = Color(0xFF8A5B00); // AA-safe on soft amber
14 ■   static const Color accentSoft = Color(0xFFFFF3D6);
15 ■
16 ■   // Semantic
17 ■   static const Color success = Color(0xFF1E8E3E);
18 ■   static const Color successSoft = Color(0xFFE2F3E7);
19 ■   static const Color warning = Color(0xFFB26A00);
20 ■   static const Color warningSoft = Color(0xFFFFF1DC);
21 ■   static const Color danger = Color(0xFFC5221F);
22 ■   static const Color dangerSoft = Color(0xFFFFCE8E7);
23 ■   static const Color info = Color(0xFF0B6CBD);
24 ■   static const Color infoSoft = Color(0xFFE3F0FA);
25 ■
26 ■   // Neutrals
27 ■   static const Color background = Color(0xFFF6F8FC);
28 ■   static const Color surface = Colors.white;
29 ■   static const Color textPrimary = Color(0xFF17233B);
30 ■   static const Color textSecondary = Color(0xFF5A6779);
31 ■   static const Color textTertiary = Color(0xFF8A94A6);
32 ■   static const Color divider = Color(0xFFE4E9F2);
33 ■   static const Color disabled = Color(0xFFB9C2D0);
34 ■
35 ■   // Live-tracking states
36 ■   static const Color live = success;
37 ■   static const Color stale = warning;
38 ■   static const Color offline = textTertiary;
39 ■ }
```

lib/core/theme/nmb_typography.dart

65 lines

```
1 ■ import 'package:flutter/material.dart';
2 ■ import 'nmb_colors.dart';
3 ■
4 ■ /// Typography scale. Large, readable, friendly.
5 ■ abstract final class NmbTypography {
6 ■   static const String fontFamily = 'Roboto'; // system default; swap freely
7 ■
8 ■   static const TextStyle displayTitle = TextStyle(
9 ■     fontSize: 28,
10 ■    fontWeight: FontWeight.w800,
11 ■    color: NmbColors.textPrimary,
12 ■    height: 1.2,
13 ■  );
14 ■
15 ■   static const TextStyle screenTitle = TextStyle(
16 ■     fontSize: 22,
17 ■     fontWeight: FontWeight.w700,
18 ■     color: NmbColors.textPrimary,
19 ■     height: 1.25,
20 ■   );
21 ■
22 ■   static const TextStyle sectionTitle = TextStyle(
23 ■     fontSize: 17,
24 ■     fontWeight: FontWeight.w700,
25 ■     color: NmbColors.textPrimary,
26 ■   );
27 ■
28 ■   static const TextStyle cardTitle = TextStyle(
29 ■     fontSize: 16,
30 ■     fontWeight: FontWeight.w600,
31 ■     color: NmbColors.textPrimary,
32 ■   );
33 ■
34 ■   static const TextStyle body = TextStyle(
35 ■     fontSize: 15,
36 ■     fontWeight: FontWeight.w400,
37 ■     color: NmbColors.textPrimary,
38 ■     height: 1.45,
39 ■   );
40 ■
41 ■   static const TextStyle bodySecondary = TextStyle(
42 ■     fontSize: 14,
43 ■     fontWeight: FontWeight.w400,
44 ■     color: NmbColors.textSecondary,
45 ■     height: 1.4,
46 ■   );
47 ■
48 ■   static const TextStyle caption = TextStyle(
49 ■     fontSize: 12.5,
50 ■     fontWeight: FontWeight.w500,
51 ■     color: NmbColors.textTertiary,
52 ■   );
53 ■
54 ■   static const TextStyle button = TextStyle(
55 ■     fontSize: 16,
56 ■     fontWeight: FontWeight.w700,
57 ■     letterSpacing: 0.2,
58 ■   );
59 ■
60 ■   static const TextStyle statLarge = TextStyle(
61 ■     fontSize: 30,
62 ■     fontWeight: FontWeight.w800,
63 ■     color: NmbColors.textPrimary,
64 ■   );
65 ■ }
```

lib/core/theme/nmb_theme.dart

143 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter/services.dart';
3 ■ import 'nmb_colors.dart';
4 ■ import 'nmb_typography.dart';
5 ■
6 ■ /// Single ThemeData used across Student, Driver and Admin experiences so the
7 ■ /// whole product feels like one design system.
8 ■ abstract final class NmbTheme {
9 ■   static const double radiusCard = 20;
10 ■   static const double radiusButton = 16;
11 ■   static const double radiusField = 14;
12 ■
13 ■   static ThemeData light() {
14 ■     final ColorScheme scheme = ColorScheme.fromSeed(
15 ■       seedColor: NmbColors.primary,
16 ■       primary: NmbColors.primary,
17 ■       secondary: NmbColors.accent,
18 ■       surface: NmbColors.surface,
19 ■       error: NmbColors.danger,
20 ■     );
21 ■
22 ■     return ThemeData(
23 ■       useMaterial3: true,
24 ■       colorScheme: scheme,
25 ■       scaffoldBackgroundColor: NmbColors.background,
26 ■       fontFamily: NmbTypography.fontFamily,
27 ■       splashFactory: InkSparkle.splashFactory,
28 ■       appBarTheme: const AppBarTheme(
29 ■         backgroundColor: NmbColors.background,
30 ■         foregroundColor: NmbColors.textPrimary,
31 ■         elevation: 0,
32 ■         scrolledUnderElevation: 0.5,
33 ■         centerTitle: false,
34 ■         titleTextStyle: NmbTypography.screenTitle,
35 ■         systemOverlayStyle: SystemUiOverlayStyle.dark,
36 ■       ),
37 ■       cardTheme: CardTheme(
38 ■         color: NmbColors.surface,
39 ■         elevation: 0,
40 ■         shape: RoundedRectangleBorder(
41 ■           borderRadius: BorderRadius.circular(radiusCard),
42 ■         ),
43 ■         margin: EdgeInsets.zero,
44 ■       ),
45 ■       filledButtonTheme: FilledButtonThemeData(
46 ■         style: FilledButton.styleFrom(
47 ■           minimumSize: const Size.fromHeight(54), // large touch target
48 ■           textStyle: NmbTypography.button,
49 ■           shape: RoundedRectangleBorder(
50 ■             borderRadius: BorderRadius.circular(radiusButton),
51 ■           ),
52 ■         ),
53 ■       ),
54 ■       outlinedButtonTheme: OutlinedButtonThemeData(
55 ■         style: OutlinedButton.styleFrom(
56 ■           minimumSize: const Size.fromHeight(52),
57 ■           textStyle: NmbTypography.button,
58 ■           side: const BorderSide(color: NmbColors.primary, width: 1.4),

```

```

59 ■         shape: RoundedRectangleBorder(
60 ■             borderRadius: BorderRadius.circular(radiusButton),
61 ■         ),
62 ■     ),
63 ■ ),
64 ■ textButtonTheme: TextButtonThemeData(
65 ■     style: TextButton.styleFrom(
66 ■         minimumSize: const Size(48, 44),
67 ■         textStyle: NmbTypography.button.copyWith(fontSize: 15),
68 ■     ),
69 ■ ),
70 ■ inputDecorationTheme: InputDecorationTheme(
71 ■     filled: true,
72 ■     fillColor: NmbColors.surface,
73 ■     contentPadding:
74 ■         const EdgeInsets.symmetric(horizontal: 16, vertical: 16),
75 ■     border: OutlineInputBorder(
76 ■         borderRadius: BorderRadius.circular(radiusField),
77 ■         borderSide: const BorderSide(color: NmbColors.divider),
78 ■     ),
79 ■     enabledBorder: OutlineInputBorder(
80 ■         borderRadius: BorderRadius.circular(radiusField),
81 ■         borderSide: const BorderSide(color: NmbColors.divider),
82 ■     ),
83 ■     focusedBorder: OutlineInputBorder(
84 ■         borderRadius: BorderRadius.circular(radiusField),
85 ■         borderSide: const BorderSide(color: NmbColors.primary, width: 1.6),
86 ■     ),
87 ■     errorBorder: OutlineInputBorder(
88 ■         borderRadius: BorderRadius.circular(radiusField),
89 ■         borderSide: const BorderSide(color: NmbColors.danger),
90 ■     ),
91 ■     hintStyle: NmbTypography.bodySecondary,
92 ■ ),
93 ■ navigationBarTheme: NavigationBarThemeData(
94 ■     backgroundColor: NmbColors.surface,
95 ■     indicatorColor: NmbColors.primarySoft,
96 ■     height: 68,
97 ■     elevation: 3,
98 ■     labelTextStyle: WidgetStatePropertyAll(
99 ■         NmbTypography.caption.copyWith(
100 ■             color: NmbColors.textSecondary,
101 ■             fontWeight: FontWeight.w600,
102 ■         ),
103 ■     ),
104 ■ ),
105 ■ snackBarTheme: SnackBarThemeData(
106 ■     behavior: SnackBarBehavior.floating,
107 ■     backgroundColor: NmbColors.textPrimary,
108 ■     contentTextStyle:
109 ■         NmbTypography.body.copyWith(color: Colors.white),
110 ■     shape: RoundedRectangleBorder(
111 ■         borderRadius: BorderRadius.circular(14),
112 ■     ),
113 ■ ),
114 ■ dialogTheme: DialogTheme(
115 ■     backgroundColor: NmbColors.surface,
116 ■     shape: RoundedRectangleBorder(

```

```
117 ■      borderRadius: BorderRadius.circular(24),
118 ■    ),
119 ■    titleTextStyle: NmbTypography.sectionTitle,
120 ■    contentTextStyle: NmbTypography.bodySecondary,
121 ■  ),
122 ■  dividerTheme: const DividerThemeData(
123 ■    color: NmbColors.divider,
124 ■    thickness: 1,
125 ■    space: 1,
126 ■  ),
127 ■  listTileTheme: const ListTileThemeData(
128 ■    iconColor: NmbColors.textSecondary,
129 ■    titleTextStyle: NmbTypography.cardTitle,
130 ■    subtitleTextStyle: NmbTypography.bodySecondary,
131 ■  ),
132 ■  chipTheme: ChipThemeData(
133 ■    backgroundColor: NmbColors.primarySoft,
134 ■    labelStyle: NmbTypography.caption
135 ■      .copyWith(color: NmbColors.primaryDark),
136 ■    shape: RoundedRectangleBorder(
137 ■      borderRadius: BorderRadius.circular(10),
138 ■    ),
139 ■    side: BorderSide.none,
140 ■  ),
141 ■ );
142 ■ }
143 ■ }
```

lib/core/constants/enums.dart

66 lines

```

1 ■ /// Core enums shared across the app. Names match backend string values.
2 ■ library;
3 ■
4 ■ enum UserRole {
5 ■   student,
6 ■   driver,
7 ■   admin;
8 ■
9 ■   static UserRole? tryParse(String? raw) {
10 ■     for (final UserRole r in UserRole.values) {
11 ■       if (r.name == raw) return r;
12 ■     }
13 ■     return null;
14 ■   }
15 ■ }
16 ■
17 ■ enum TripStatus {
18 ■   active,
19 ■   completed,
20 ■   autoClosed;
21 ■
22 ■   static TripStatus parse(String raw) => TripStatus.values.firstWhere(
23 ■     (TripStatus s) => s.name == raw,
24 ■     orElse: () => TripStatus.completed,
25 ■   );
26 ■ }
27 ■
28 ■ enum TripDirection {
29 ■   pickup,
30 ■   drop;
31 ■
32 ■   static TripDirection parse(String raw) => TripDirection.values.firstWhere(
33 ■     (TripDirection d) => d.name == raw,
34 ■     orElse: () => TripDirection.pickup,
35 ■   );
36 ■
37 ■   String get label => this == TripDirection.pickup ? 'Pickup' : 'Drop-off';
38 ■ }
39 ■
40 ■ /// Freshness of the live bus location – drives honest status UI.
41 ■ enum LocationFreshness {
42 ■   /// Updated within the last 30 seconds. Safe to call "LIVE".
43 ■   live,
44 ■
45 ■   /// Updated 30s-3min ago. Show "Last updated X ago", never "live".
46 ■   stale,
47 ■
48 ■   /// Older than 3 minutes, or no data / no active trip.
49 ■   unavailable,
50 ■ }
51 ■
52 ■ enum NotificationType {
53 ■   tripStarted,
54 ■   busApproaching,
55 ■   busReachedStop,
56 ■   busReachedSchool,
57 ■   busDelayed,
58 ■   trackingUnavailable,
59 ■   announcement;
60 ■
61 ■   static NotificationType parse(String raw) =>
62 ■     NotificationType.values.firstWhere(
63 ■       (NotificationType t) => t.name == raw,
64 ■       orElse: () => NotificationType.announcement,
65 ■     );
66 ■ }

```

```

1 ■ /// App-wide constants and tuning values.
2 ■ abstract final class NmbConstants {
3 ■     static const String appName = 'Never Miss Bus';
4 ■     static const String schoolName = 'SRBS International School';
5 ■
6 ■     // Live location freshness thresholds
7 ■     static const Duration liveThreshold = Duration(seconds: 30);
8 ■     static const Duration staleThreshold = Duration(minutes: 3);
9 ■
10 ■    // Driver GPS stream tuning
11 ■    static const int locationDistanceFilterMeters = 20;
12 ■    static const Duration locationMinInterval = Duration(seconds: 5);
13 ■
14 ■    // ETA guardrails
15 ■    static const double etaMinSpeedKmh = 12; // floor so ETA never explodes
16 ■    static const double etaMaxReliableDistanceKm = 30;
17 ■
18 ■    // Geofence for "Bus Approaching" (also enforced server-side)
19 ■    static const double approachingRadiusMeters = 800;
20 ■
21 ■    // Layout
22 ■    static const double screenPadding = 20;
23 ■    static const double cardGap = 14;
24 ■    static const double maxContentWidth = 560; // keeps tablets/large phones tidy
25 ■ }

```

lib/core/utls/validators.dart

44 lines

```

1  /// Client-side input validation (server re-validates everything).
2  abstract final class Validators {
3      static final RegExp _email =
4          RegExp(r'^[\w.+-]+@[a-zA-Z\d-]+\.[a-zA-Z\d-]+\.$');
5
6      static String? email(String? value) {
7          final String v = value?.trim() ?? '';
8          if (v.isEmpty) return 'Please enter your email address';
9          if (!_email.hasMatch(v)) return 'That email address doesn\'t look right';
10         return null;
11     }
12
13     static String? password(String? value) {
14         final String v = value ?? '';
15         if (v.isEmpty) return 'Please enter your password';
16         if (v.length < 8) return 'Password must be at least 8 characters';
17         return null;
18     }
19
20     static String? strongPassword(String? value) {
21         final String v = value ?? '';
22         if (v.length < 10) return 'Use at least 10 characters';
23         if (!v.contains(RegExp(r'[A-Z]'))) return 'Add an uppercase letter';
24         if (!v.contains(RegExp(r'[0-9]'))) return 'Add a number';
25         return null;
26     }
27
28     static String? requiredField(String? value, {String label = 'This field'}) {
29         if ((value ?? '').trim().isEmpty) return '$label is required';
30         return null;
31     }
32
33     static String? latitude(String? value) {
34         final double? v = double.tryParse((value ?? '').trim());
35         if (v == null || v < -90 || v > 90) return 'Enter a valid latitude';
36         return null;
37     }
38
39     static String? longitude(String? value) {
40         final double? v = double.tryParse((value ?? '').trim());
41         if (v == null || v < -180 || v > 180) return 'Enter a valid longitude';
42         return null;
43     }
44 }

```

lib/core/utils/formatters.dart

43 lines

```

1 ■ import 'package:intl/intl.dart';
2 ■
3 ■ /// Shared human-friendly formatting. Keeping ETA wording here guarantees the
4 ■ /// app never presents an estimate as a promise.
5 ■ abstract final class Formatters {
6 ■   static String relativeTime(DateTime time) {
7 ■     final Duration diff = DateTime.now().difference(time);
8 ■     if (diff.inSeconds < 45) return 'just now';
9 ■     if (diff.inMinutes < 1) return 'less than a minute ago';
10 ■    if (diff.inMinutes == 1) return '1 minute ago';
11 ■    if (diff.inMinutes < 60) return '${diff.inMinutes} minutes ago';
12 ■    if (diff.inHours == 1) return '1 hour ago';
13 ■    if (diff.inHours < 24) return '${diff.inHours} hours ago';
14 ■    return DateFormat('d MMM, h:mm a').format(time);
15 ■  }
16 ■
17 ■  /// Always approximate, never exact: "arriving in approximately 8 minutes".
18 ■  static String eta(Duration eta) {
19 ■    final int mins = eta.inMinutes;
20 ■    if (mins <= 1) return 'Bus arriving in approximately 1 minute';
21 ■    return 'Bus arriving in approximately $mins minutes';
22 ■  }
23 ■
24 ■  static String etaShort(Duration eta) {
25 ■    final int mins = eta.inMinutes.clamp(1, 999);
26 ■    return '~$mins min';
27 ■  }
28 ■
29 ■  static String timeOfDay(DateTime t) => DateFormat('h:mm a').format(t);
30 ■
31 ■  static String dateTime(DateTime t) =>
32 ■    DateFormat('EEE, d MMM • h:mm a').format(t);
33 ■
34 ■  static String greetingForNow({required String firstName}) {
35 ■    final int h = DateTime.now().hour;
36 ■    final String part = h < 12
37 ■      ? 'Good morning'
38 ■      : h < 17
39 ■        ? 'Good afternoon'
40 ■        : 'Good evening';
41 ■    return '$part, $firstName!';
42 ■  }
43 ■ }

```

lib/core/utils/result.dart

56 lines

```

1 ■ /// Lightweight Result type so services return typed failures instead of
2 ■ /// throwing raw exceptions into the UI layer.
3 ■ sealed class Result<T> {
4 ■     const Result();
5 ■
6 ■     R when<R>({
7 ■         required R Function(T value) ok,
8 ■         required R Function(AppFailure failure) err,
9 ■     }) {
10 ■         final Result<T> self = this;
11 ■         return switch (self) {
12 ■             Ok<T>(:final T value) => ok(value),
13 ■             Err<T>(:final AppFailure failure) => err(failure),
14 ■         };
15 ■     }
16 ■
17 ■     bool get isOk => this is Ok<T>;
18 ■ }
19 ■
20 ■ final class Ok<T> extends Result<T> {
21 ■     const Ok(this.value);
22 ■     final T value;
23 ■ }
24 ■
25 ■ final class Err<T> extends Result<T> {
26 ■     const Err(this.failure);
27 ■     final AppFailure failure;
28 ■ }
29 ■
30 ■ /// Normalized, user-presentable failure.
31 ■ class AppFailure {
32 ■     const AppFailure(this.code, this.message);
33 ■
34 ■     final String code;
35 ■     final String message;
36 ■
37 ■     static const AppFailure network = AppFailure(
38 ■         'network',
39 ■         'No internet connection. Please check your network and try again.',
40 ■     );
41 ■     static const AppFailure permissionDenied = AppFailure(
42 ■         'permission-denied',
43 ■         'You don\'t have permission to do that.',
44 ■     );
45 ■     static const AppFailure sessionExpired = AppFailure(
46 ■         'session-expired',
47 ■         'Your session has expired. Please sign in again.',
48 ■     );
49 ■     static const AppFailure unknown = AppFailure(
50 ■         'unknown',
51 ■         'Something went wrong. Please try again.',
52 ■     );
53 ■
54 ■     @override
55 ■     String toString() => 'AppFailure($code: $message)';
56 ■ }

```

3. Reusable Widgets

Shared UI components used by all three roles.

nmb_card.dart · status_pill.dart · state_views.dart · nmb_dialogs.dart ·
responsive_scaffold_body.dart

lib/core/widgets/nmb_card.dart

52 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import '../theme/nmb_colors.dart';
3 ■ import '../theme/nmb_theme.dart';
4 ■
5 ■ /// Rounded card with a soft shadow – the base surface of the design system.
6 ■ class NmbCard extends StatelessWidget {
7 ■   const NmbCard({
8 ■     super.key,
9 ■     required this.child,
10 ■     this.padding = const EdgeInsets.all(18),
11 ■     this.color = NmbColors.surface,
12 ■     this.onTap,
13 ■     this.borderColor,
14 ■   });
15 ■
16 ■   final Widget child;
17 ■   final EdgeInsetsGeometry padding;
18 ■   final Color color;
19 ■   final VoidCallback? onTap;
20 ■   final Color? borderColor;
21 ■
22 ■   @override
23 ■   Widget build(BuildContext context) {
24 ■     final Widget card = Container(
25 ■       decoration: BoxDecoration(
26 ■         color: color,
27 ■         borderRadius: BorderRadius.circular(NmbTheme.radiusCard),
28 ■         border: borderColor != null
29 ■           ? Border.all(color: borderColor!, width: 1.2)
30 ■           : null,
31 ■         boxShadow: const <BoxShadow>[
32 ■           BoxShadow(
33 ■             color: Color(0x0D17233B),
34 ■             blurRadius: 18,
35 ■             offset: Offset(0, 6),
36 ■           ),
37 ■         ],
38 ■       ),
39 ■       child: Padding(padding: padding, child: child),
40 ■     );
41 ■
42 ■     if (onTap == null) return card;
43 ■     return Material(
44 ■       color: Colors.transparent,
45 ■       child: InkWell(
46 ■         onTap: onTap,
47 ■         borderRadius: BorderRadius.circular(NmbTheme.radiusCard),
48 ■         child: card,
49 ■       ),
50 ■     );
51 ■   }
52 ■ }

```

lib/core/widgets/status_pill.dart

119 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import '../constants/enums.dart';
3 ■ import '../theme/nmb_colors.dart';
4 ■ import '../theme/nmb_typography.dart';
5 ■
6 ■ /// Small colored capsule used for every status in the app (bus status,
7 ■ /// tracking status, account status ...) so states always look consistent.
8 ■ class StatusPill extends StatelessWidget {
9 ■   const StatusPill({
10 ■     super.key,
11 ■     required this.label,
12 ■     required this.color,
13 ■     required this.background,
14 ■     this.pulse = false,
15 ■   });
16 ■
17 ■   final String label;
18 ■   final Color color;
19 ■   final Color background;
20 ■
21 ■   /// Shows an animated dot for genuinely-live states only.
22 ■   final bool pulse;
23 ■
24 ■   factory StatusPill.freshness(LocationFreshness f) {
25 ■     switch (f) {
26 ■       case LocationFreshness.live:
27 ■         return const StatusPill(
28 ■           label: 'LIVE',
29 ■           color: NmbColors.live,
30 ■           background: NmbColors.successSoft,
31 ■           pulse: true,
32 ■         );
33 ■       case LocationFreshness.stale:
34 ■         return const StatusPill(
35 ■           label: 'DELAYED SIGNAL',
36 ■           color: NmbColors.stale,
37 ■           background: NmbColors.warningSoft,
38 ■         );
39 ■       case LocationFreshness.unavailable:
40 ■         return const StatusPill(
41 ■           label: 'NOT TRACKING',
42 ■           color: NmbColors.offline,
43 ■           background: NmbColors.divider,
44 ■         );
45 ■     }
46 ■   }
47 ■
48 ■   @override
49 ■   Widget build(BuildContext context) {
50 ■     return Container(
51 ■       padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 5),
52 ■       decoration: BoxDecoration(
53 ■         color: background,
54 ■         borderRadius: BorderRadius.circular(999),
55 ■       ),
56 ■       child: Row(
57 ■         mainAxisAlignment: MainAxisAlignment.min,
58 ■         children: <Widget>[

```

```

59 ■         if (pulse) ...<Widget>[
60 ■             _PulsingDot(color: color),
61 ■             const SizedBox(width: 6),
62 ■         ] else ...<Widget>[
63 ■             Container(
64 ■                 width: 7,
65 ■                 height: 7,
66 ■                 decoration: BoxDecoration(color: color, shape: BoxShape.circle),
67 ■             ),
68 ■             const SizedBox(width: 6),
69 ■         ],
70 ■         Text(
71 ■             label,
72 ■             style: NmbTypography.caption.copyWith(
73 ■                 color: color,
74 ■                 fontWeight: FontWeight.w800,
75 ■                 letterSpacing: 0.6,
76 ■             ),
77 ■         ),
78 ■     ],
79 ■ ),
80 ■ );
81 ■ }
82 ■ }
83 ■
84 ■ class _PulsingDot extends StatefulWidget {
85 ■     const _PulsingDot({required this.color});
86 ■     final Color color;
87 ■
88 ■     @override
89 ■     State<_PulsingDot> createState() => _PulsingDotState();
90 ■ }
91 ■
92 ■ class _PulsingDotState extends State<_PulsingDot>
93 ■     with SingleTickerProviderStateMixin {
94 ■     late final AnimationController _controller = AnimationController(
95 ■         vsync: this,
96 ■         duration: const Duration(milliseconds: 1100),
97 ■     )..repeat(reverse: true);
98 ■
99 ■     @override
100 ■     void dispose() {
101 ■         _controller.dispose();
102 ■         super.dispose();
103 ■     }
104 ■
105 ■     @override
106 ■     Widget build(BuildContext context) {
107 ■         return FadeTransition(
108 ■             opacity: Tween<double>(begin: 0.35, end: 1).animate(
109 ■                 CurvedAnimation(parent: _controller, curve: Curves.easeInOut),
110 ■             ),
111 ■             child: Container(
112 ■                 width: 8,
113 ■                 height: 8,
114 ■                 decoration:
115 ■                     BoxDecoration(color: widget.color, shape: BoxShape.circle),
116 ■             ),
117 ■         );
118 ■     }
119 ■ }

```

```

1 ■ import 'package:flutter/material.dart';
2 ■ import '../theme/nmb_colors.dart';
3 ■ import '../theme/nmb_typography.dart';
4 ■
5 ■ /// Full-area loading indicator.
6 ■ class LoadingView extends StatelessWidget {
7 ■   const LoadingView({super.key, this.message});
8 ■   final String? message;
9 ■
10 ■   @override
11 ■   Widget build(BuildContext context) {
12 ■     return Center(
13 ■       child: Column(
14 ■         mainAxisAlignment: MainAxisAlignment.min,
15 ■         children: <Widget>[
16 ■           const SizedBox(
17 ■             width: 34,
18 ■             height: 34,
19 ■             child: CircularProgressIndicator(strokeWidth: 3),
20 ■           ),
21 ■           if (message != null) ...<Widget>[
22 ■             const SizedBox(height: 14),
23 ■             Text(message!, style: NmbTypography.bodySecondary),
24 ■           ],
25 ■         ],
26 ■       ),
27 ■     );
28 ■   }
29 ■ }
30 ■
31 ■ /// Friendly empty state with icon, title, message and optional action.
32 ■ class EmptyState extends StatelessWidget {
33 ■   const EmptyState({
34 ■     super.key,
35 ■     required this.icon,
36 ■     required this.title,
37 ■     required this.message,
38 ■     this.actionLabel,
39 ■     this.onAction,
40 ■   });
41 ■
42 ■   final IconData icon;
43 ■   final String title;
44 ■   final String message;
45 ■   final String? actionLabel;
46 ■   final VoidCallback? onAction;
47 ■
48 ■   @override
49 ■   Widget build(BuildContext context) {
50 ■     return Center(
51 ■       child: Padding(
52 ■         padding: const EdgeInsets.all(32),
53 ■         child: Column(
54 ■           mainAxisAlignment: MainAxisAlignment.min,
55 ■           children: <Widget>[
56 ■             Container(
57 ■               width: 88,
58 ■               height: 88,

```

```

59 ■         decoration: const BoxDecoration(
60 ■             color: NmbColors.primarySoft,
61 ■             shape: BoxShape.circle,
62 ■         ),
63 ■         child: Icon(icon, size: 42, color: NmbColors.primary),
64 ■     ),
65 ■     const SizedBox(height: 18),
66 ■     Text(title,
67 ■         style: NmbTypography.sectionTitle,
68 ■         textAlign: TextAlign.center,),
69 ■     const SizedBox(height: 8),
70 ■     Text(message,
71 ■         style: NmbTypography.bodySecondary,
72 ■         textAlign: TextAlign.center,),
73 ■     if (actionLabel != null && onAction != null) ...<Widget>[
74 ■         const SizedBox(height: 20),
75 ■         FilledButton(onPressed: onAction, child: Text(actionLabel!)),
76 ■     ],
77 ■ ],
78 ■ ),
79 ■ ),
80 ■ );
81 ■ }
82 ■ }
83 ■
84 ■ /// Error state with retry – used for network/database failures.
85 ■ class ErrorView extends StatelessWidget {
86 ■     const ErrorView({
87 ■         super.key,
88 ■         this.title = 'Something went wrong',
89 ■         required this.message,
90 ■         this.onRetry,
91 ■     });
92 ■
93 ■     final String title;
94 ■     final String message;
95 ■     final VoidCallback? onRetry;
96 ■
97 ■     @override
98 ■     Widget build(BuildContext context) {
99 ■         return EmptyState(
100 ■             icon: Icons.cloud_off_rounded,
101 ■             title: title,
102 ■             message: message,
103 ■             actionLabel: onRetry != null ? 'Try again' : null,
104 ■             onAction: onRetry,
105 ■         );
106 ■     }
107 ■ }
108 ■
109 ■ /// Banner shown when the device is offline.
110 ■ class OfflineBanner extends StatelessWidget {
111 ■     const OfflineBanner({super.key});
112 ■
113 ■     @override
114 ■     Widget build(BuildContext context) {
115 ■         return Material(
116 ■             color: NmbColors.warningSoft,
117 ■
118 ■             child: SafeArea(
119 ■                 bottom: false,
120 ■                 child: Padding(
121 ■                     padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
122 ■                     child: Row(
123 ■                         children: <Widget>[
124 ■                             const Icon(Icons.wifi_off_rounded,
125 ■                                 size: 18, color: NmbColors.warning,),
126 ■                             const SizedBox(width: 8),
127 ■                             Expanded(
128 ■                                 child: Text(
129 ■                                     'You\'re offline. Information may not be up to date.',
130 ■                                     style: NmbTypography.caption
131 ■                                     .copyWith(color: NmbColors.warning),
132 ■                                 ),
133 ■                             ),
134 ■                         ],
135 ■                     ),
136 ■                 ),
137 ■             );
138 ■         }
139 ■     }

```

lib/core/widgets/nmb_dialogs.dart

88 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import '../theme/nmb_colors.dart';
3 ■ import '../theme/nmb_typography.dart';
4 ■
5 ■ /// Consistent confirmation dialog. Returns true if confirmed.
6 ■ Future<bool> showNmbConfirmDialog(
7 ■   BuildContext context, {
8 ■     required String title,
9 ■     required String message,
10 ■     String confirmLabel = 'Confirm',
11 ■     String cancelLabel = 'Cancel',
12 ■     bool destructive = false,
13 ■     IconData? icon,
14 ■ }) async {
15 ■   final bool? result = await showDialog<bool>(
16 ■     context: context,
17 ■     builder: (BuildContext ctx) => AlertDialog(
18 ■       title: Row(
19 ■         children: <Widget>[
20 ■           if (icon != null) ...<Widget>[
21 ■             Icon(icon,
22 ■               color: destructive ? NmbColors.danger : NmbColors.primary,),
23 ■             const SizedBox(width: 10),
24 ■           ],
25 ■           Expanded(child: Text(title)),
26 ■         ],
27 ■       ),
28 ■       content: Text(message),
29 ■       actions: <Widget>[
30 ■         TextButton(
31 ■           onPressed: () => Navigator.of(ctx).pop(false),
32 ■           child: Text(cancelLabel),
33 ■         ),
34 ■         FilledButton(
35 ■           style: destructive
36 ■             ? FilledButton.styleFrom(
37 ■               backgroundColor: NmbColors.danger,
38 ■               minimumSize: const Size(110, 46),
39 ■             )
40 ■             : FilledButton.styleFrom(minimumSize: const Size(110, 46)),
41 ■           onPressed: () => Navigator.of(ctx).pop(true),
42 ■           child: Text(confirmLabel),
43 ■         ),
44 ■       ],
45 ■     ),
46 ■   );
47 ■   return result ?? false;
48 ■ }
49 ■
50 ■ /// Consistent snackbars.
51 ■ void showNmbSnack(
52 ■   BuildContext context,
53 ■   String message, {
54 ■     bool isError = false,
55 ■     bool isSuccess = false,
56 ■ }) {
57 ■   ScaffoldMessenger.of(context)
58 ■     ..hideCurrentSnackBar()

```

```

59 ■ ..showSnackBar(
60 ■   SnackBar(
61 ■     backgroundColor: isError
62 ■       ? NmbColors.danger
63 ■       : isSuccess
64 ■       ? NmbColors.success
65 ■       : NmbColors.textPrimary,
66 ■     content: Row(
67 ■       children: <Widget>[
68 ■         Icon(
69 ■           isError
70 ■             ? Icons.error_outline_rounded
71 ■             : isSuccess
72 ■             ? Icons.check_circle_outline_rounded
73 ■             : Icons.info_outline_rounded,
74 ■           color: Colors.white,
75 ■           size: 20,
76 ■         ),
77 ■         const SizedBox(width: 10),
78 ■         Expanded(
79 ■           child: Text(
80 ■             message,
81 ■             style: NmbTypography.body.copyWith(color: Colors.white),
82 ■           ),
83 ■         ),
84 ■       ],
85 ■     ),
86 ■   ),
87 ■ );
88 ■ }

```

lib/core/widgets/responsive_scaffold_body.dart

43 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import '../constants/nmb_constants.dart';
3 ■
4 ■ /// Constrains content width on large screens and applies standard padding –
5 ■ /// keeps layouts responsive without hard-coded dimensions in screens.
6 ■ class ResponsiveBody extends StatelessWidget {
7 ■   const ResponsiveBody({
8 ■     super.key,
9 ■     required this.child,
10 ■     this.scrollable = true,
11 ■     this.padding,
12 ■   });
13 ■
14 ■   final Widget child;
15 ■   final bool scrollable;
16 ■   final EdgeInsetsGeometry? padding;
17 ■
18 ■   @override
19 ■   Widget build(BuildContext context) {
20 ■     final Widget constrained = Center(
21 ■       child: ConstrainedBox(
22 ■         constraints:
23 ■           const BoxConstraints(maxWidth: NmbConstants.maxContentWidth),
24 ■         child: Padding(
25 ■           padding: padding ??
26 ■             const EdgeInsets.symmetric(
27 ■               horizontal: NmbConstants.screenPadding,
28 ■               vertical: 12,
29 ■             ),
30 ■         child: child,
31 ■       ),
32 ■     ),
33 ■   );
34 ■
35 ■   if (!scrollable) return SafeArea(child: constrained);
36 ■   return SafeArea(
37 ■     child: SingleChildScrollView(
38 ■       physics: const BouncingScrollPhysics(),
39 ■       child: constrained,
40 ■     ),
41 ■   );
42 ■ }
43 ■ }

```

4. Data Models

Pure-Dart data classes (no Firebase imports).

geo_point_data.dart · app_user.dart · bus.dart · bus_route.dart · bus_stop.dart · trip.dart · live_location.dart · app_notification.dart · audit_log.dart

lib/models/geo_point_data.dart

38 lines

```

1 ■ import 'dart:math' as math;
2 ■
3 ■ /// Plain lat/lng value object (keeps models free of Firebase imports).
4 ■ class GeoPointData {
5 ■   const GeoPointData({required this.lat, required this.lng});
6 ■
7 ■   final double lat;
8 ■   final double lng;
9 ■
10 ■   factory GeoPointData.fromMap(Map<String, dynamic> map) => GeoPointData(
11 ■     lat: (map['lat'] as num?)?.toDouble() ?? 0,
12 ■     lng: (map['lng'] as num?)?.toDouble() ?? 0,
13 ■   );
14 ■
15 ■   Map<String, dynamic> toMap() => <String, dynamic>{'lat': lat, 'lng': lng};
16 ■
17 ■   /// Haversine distance in meters.
18 ■   double distanceTo(GeoPointData other) {
19 ■     const double r = 6371000;
20 ■     final double dLat = _rad(other.lat - lat);
21 ■     final double dLng = _rad(other.lng - lng);
22 ■     final double a = math.sin(dLat / 2) * math.sin(dLat / 2) +
23 ■       math.cos(_rad(lat)) *
24 ■       math.cos(_rad(other.lat)) *
25 ■       math.sin(dLng / 2) *
26 ■       math.sin(dLng / 2);
27 ■     return 2 * r * math.atan2(math.sqrt(a), math.sqrt(1 - a));
28 ■   }
29 ■
30 ■   static double _rad(double deg) => deg * math.pi / 180;
31 ■
32 ■   @override
33 ■   bool operator ==(Object other) =>
34 ■     other is GeoPointData && other.lat == lat && other.lng == lng;
35 ■
36 ■   @override
37 ■   int get hashCode => Object.hash(lat, lng);
38 ■ }

```

lib/models/app_user.dart

69 lines

```

1 ■ import '../core/constants/enums.dart';
2 ■
3 ■ /// A user profile document (`users/{uid}`).
4 ■ ///
5 ■ /// NOTE: [role] and [busId] here are display mirrors. The *authoritative*
6 ■ /// values live in Firebase Auth custom claims and are enforced by security
7 ■ /// rules – the app never grants access based on this document alone.
8 ■ class AppUser {
9 ■   const AppUser({
10 ■     required this.uid,
11 ■     required this.role,
12 ■     required this.fullName,
13 ■     required this.email,
14 ■     this.phone,
15 ■     this.busId,
16 ■     this.stopId,
17 ■     this.classSection,
18 ■     this.isActive = true,
19 ■     this.createdAt,
20 ■   });
21 ■
22 ■   final String uid;
23 ■   final UserRole role;
24 ■   final String fullName;
25 ■   final String email;
26 ■   final String? phone;
27 ■   final String? busId;
28 ■   final String? stopId;
29 ■   final String? classSection;
30 ■   final bool isActive;
31 ■   final DateTime? createdAt;
32 ■
33 ■   String get firstName =>
34 ■     fullName.trim().isEmpty ? 'there' : fullName.trim().split(' ').first;
35 ■
36 ■   factory AppUser.fromMap(String uid, Map<String, dynamic> map) => AppUser(
37 ■     uid: uid,
38 ■     role: UserRole.tryParse(map['role'] as String?) ?? UserRole.student,
39 ■     fullName: (map['fullName'] as String?) ?? '',
40 ■     email: (map['email'] as String?) ?? '',
41 ■     phone: map['phone'] as String?,
42 ■     busId: map['busId'] as String?,
43 ■     stopId: map['stopId'] as String?,
44 ■     classSection: map['classSection'] as String?,
45 ■     isActive: (map['isActive'] as bool?) ?? true,
46 ■     createdAt: _toDate(map['createdAt']),
47 ■   );
48 ■
49 ■   Map<String, dynamic> toMap() => <String, dynamic>{
50 ■     'role': role.name,
51 ■     'fullName': fullName,
52 ■     'email': email,
53 ■     if (phone != null) 'phone': phone,
54 ■     if (busId != null) 'busId': busId,
55 ■     if (stopId != null) 'stopId': stopId,
56 ■     if (classSection != null) 'classSection': classSection,
57 ■     'isActive': isActive,
58 ■   };
59 ■
60 ■   static DateTime? _toDate(Object? v) {
61 ■     if (v == null) return null;
62 ■     // cloud_firestore Timestamp exposes toDate(); avoid importing it here.
63 ■     try {
64 ■       return (v as dynamic).toDate() as DateTime?;
65 ■     } catch (_) {
66 ■       return null;
67 ■     }
68 ■   }
69 ■ }

```

```
1 ■ class Bus {
2 ■   const Bus({
3 ■     required this.id,
4 ■     required this.busNumber,
5 ■     required this.plateNumber,
6 ■     this.capacity = 0,
7 ■     this.driverId,
8 ■     this.routeId,
9 ■     this.isActive = true,
10 ■   });
11 ■
12 ■   final String id;
13 ■   final String busNumber; // e.g. "Bus 3"
14 ■   final String plateNumber; // e.g. "UK07 PA 1234"
15 ■   final int capacity;
16 ■   final String? driverId;
17 ■   final String? routeId;
18 ■   final bool isActive;
19 ■
20 ■   factory Bus.fromMap(String id, Map<String, dynamic> map) => Bus(
21 ■     id: id,
22 ■     busNumber: (map['busNumber'] as String?) ?? '',
23 ■     plateNumber: (map['plateNumber'] as String?) ?? '',
24 ■     capacity: (map['capacity'] as num?)?.toInt() ?? 0,
25 ■     driverId: map['driverId'] as String?,
26 ■     routeId: map['routeId'] as String?,
27 ■     isActive: (map['isActive'] as bool?) ?? true,
28 ■   );
29 ■
30 ■   Map<String, dynamic> toMap() => <String, dynamic>{
31 ■     'busNumber': busNumber,
32 ■     'plateNumber': plateNumber,
33 ■     'capacity': capacity,
34 ■     'driverId': driverId,
35 ■     'routeId': routeId,
36 ■     'isActive': isActive,
37 ■   };
38 ■ }
```

lib/models/bus_route.dart

48 lines

```

1 ■ import '../core/constants/enums.dart';
2 ■ import 'geo_point_data.dart';
3 ■
4 ■ class BusRoute {
5 ■   const BusRoute({
6 ■     required this.id,
7 ■     required this.name,
8 ■     required this.busId,
9 ■     required this.stopOrder,
10 ■    required this.schoolLocation,
11 ■    this.polyline = '',
12 ■    this.direction = TripDirection.pickup,
13 ■    this.isActive = true,
14 ■  });
15 ■
16 ■   final String id;
17 ■   final String name;
18 ■   final String busId;
19 ■   final List<String> stopOrder; // ordered stop ids
20 ■   final GeoPointData schoolLocation;
21 ■   final String polyline; // encoded polyline; may be empty (fallback: stops)
22 ■   final TripDirection direction;
23 ■   final bool isActive;
24 ■
25 ■   factory BusRoute.fromMap(String id, Map<String, dynamic> map) => BusRoute(
26 ■     id: id,
27 ■     name: (map['name'] as String?) ?? '',
28 ■     busId: (map['busId'] as String?) ?? '',
29 ■     stopOrder: List<String>.from(map['stopOrder'] as List? ?? <String>[]),
30 ■     schoolLocation: GeoPointData.fromMap(
31 ■       Map<String, dynamic>.from(map['schoolLocation'] as Map? ?? {}),
32 ■     ),
33 ■     polyline: (map['polyline'] as String?) ?? '',
34 ■     direction:
35 ■       TripDirection.parse((map['direction'] as String?) ?? 'pickup'),
36 ■     isActive: (map['isActive'] as bool?) ?? true,
37 ■   );
38 ■
39 ■   Map<String, dynamic> toMap() => <String, dynamic>{
40 ■     'name': name,
41 ■     'busId': busId,
42 ■     'stopOrder': stopOrder,
43 ■     'schoolLocation': schoolLocation.toMap(),
44 ■     'polyline': polyline,
45 ■     'direction': direction.name,
46 ■     'isActive': isActive,
47 ■   };
48 ■ }

```

lib/models/bus_stop.dart

46 lines

```

1 ■ import 'geo_point_data.dart';
2 ■
3 ■ class BusStop {
4 ■   const BusStop({
5 ■     required this.id,
6 ■     required this.name,
7 ■     required this.routeId,
8 ■     required this.busId,
9 ■     required this.location,
10 ■    required this.order,
11 ■    this.scheduledTime,
12 ■    this.isActive = true,
13 ■  });
14 ■
15 ■   final String id;
16 ■   final String name;
17 ■   final String routeId;
18 ■   final String busId;
19 ■   final GeoPointData location;
20 ■   final int order;
21 ■   final String? scheduledTime; // "07:15"
22 ■   final bool isActive;
23 ■
24 ■   factory BusStop.fromMap(String id, Map<String, dynamic> map) => BusStop(
25 ■     id: id,
26 ■     name: (map['name'] as String?) ?? '',
27 ■     routeId: (map['routeId'] as String?) ?? '',
28 ■     busId: (map['busId'] as String?) ?? '',
29 ■     location: GeoPointData.fromMap(
30 ■       Map<String, dynamic>.from(map['location'] as Map? ?? {}),
31 ■     ),
32 ■     order: (map['order'] as num?)?.toInt() ?? 0,
33 ■     scheduledTime: map['scheduledTime'] as String?,
34 ■     isActive: (map['isActive'] as bool?) ?? true,
35 ■   );
36 ■
37 ■   Map<String, dynamic> toMap() => <String, dynamic>{
38 ■     'name': name,
39 ■     'routeId': routeId,
40 ■     'busId': busId,
41 ■     'location': location.toMap(),
42 ■     'order': order,
43 ■     if (scheduledTime != null) 'scheduledTime': scheduledTime,
44 ■     'isActive': isActive,
45 ■   };
46 ■ }

```

lib/models/trip.dart

46 lines

```

1 ■ import '../core/constants/enums.dart';
2 ■
3 ■ class Trip {
4 ■   const Trip({
5 ■     required this.id,
6 ■     required this.busId,
7 ■     required this.driverId,
8 ■     required this.routeId,
9 ■     required this.direction,
10 ■    required this.status,
11 ■    this.startedAt,
12 ■    this.endedAt,
13 ■   });
14 ■
15 ■   final String id;
16 ■   final String busId;
17 ■   final String driverId;
18 ■   final String routeId;
19 ■   final TripDirection direction;
20 ■   final TripStatus status;
21 ■   final DateTime? startedAt;
22 ■   final DateTime? endedAt;
23 ■
24 ■   bool get isActive => status == TripStatus.active;
25 ■
26 ■   factory Trip.fromMap(String id, Map<String, dynamic> map) => Trip(
27 ■     id: id,
28 ■     busId: (map['busId'] as String?) ?? '',
29 ■     driverId: (map['driverId'] as String?) ?? '',
30 ■     routeId: (map['routeId'] as String?) ?? '',
31 ■     direction:
32 ■       TripDirection.parse((map['direction'] as String?) ?? 'pickup'),
33 ■     status: TripStatus.parse((map['status'] as String?) ?? 'completed'),
34 ■     startedAt: _toDate(map['startedAt']),
35 ■     endedAt: _toDate(map['endedAt']),
36 ■   );
37 ■
38 ■   static DateTime? _toDate(Object? v) {
39 ■     if (v == null) return null;
40 ■     try {
41 ■       return (v as dynamic).toDate() as DateTime?;
42 ■     } catch (_) {
43 ■       return null;
44 ■     }
45 ■   }
46 ■ }

```

lib/models/live_location.dart

45 lines

```

1 ■ import '../core/constants/enums.dart';
2 ■ import '../core/constants/nmb_constants.dart';
3 ■ import 'geo_point_data.dart';
4 ■
5 ■ /// A single live GPS fix from `/liveLocations/{busId}` in Realtime Database.
6 ■ class LiveLocation {
7 ■   const LiveLocation({
8 ■     required this.position,
9 ■     required this.heading,
10 ■    required this.speedKmh,
11 ■    required this.accuracy,
12 ■    required this.tripId,
13 ■    required this.updatedAt,
14 ■   });
15 ■
16 ■   final GeoPointData position;
17 ■   final double heading;
18 ■   final double speedKmh;
19 ■   final double accuracy;
20 ■   final String tripId;
21 ■   final DateTime updatedAt;
22 ■
23 ■   factory LiveLocation.fromMap(Map<Object?, Object?> map) => LiveLocation(
24 ■     position: GeoPointData(
25 ■       lat: (map['lat'] as num?)?.toDouble() ?? 0,
26 ■       lng: (map['lng'] as num?)?.toDouble() ?? 0,
27 ■     ),
28 ■     heading: (map['heading'] as num?)?.toDouble() ?? 0,
29 ■     speedKmh: (map['speedKmh'] as num?)?.toDouble() ?? 0,
30 ■     accuracy: (map['accuracy'] as num?)?.toDouble() ?? 0,
31 ■     tripId: (map['tripId'] as String?) ?? '',
32 ■     updatedAt: DateTime.fromMillisecondsSinceEpoch(
33 ■       (map['updatedAt'] as num?)?.toInt() ?? 0,
34 ■     ),
35 ■   );
36 ■
37 ■   /// Honest freshness classification – the UI must never show a stale fix
38 ■   /// as "live".
39 ■   LocationFreshness freshness({DateTime? now}) {
40 ■     final Duration age = (now ?? DateTime.now()).difference(updatedAt);
41 ■     if (age <= NmbConstants.liveThreshold) return LocationFreshness.live;
42 ■     if (age <= NmbConstants.staleThreshold) return LocationFreshness.stale;
43 ■     return LocationFreshness.unavailable;
44 ■   }
45 ■ }

```

```
1 ■ import '../core/constants/enums.dart';
2 ■
3 ■ /// An item in the user's private inbox (`users/{uid}/inbox/{id}`).
4 ■ class AppNotification {
5 ■   const AppNotification({
6 ■     required this.id,
7 ■     required this.type,
8 ■     required this.title,
9 ■     required this.body,
10 ■    required this.receivedAt,
11 ■    this.read = false,
12 ■  });
13 ■
14 ■   final String id;
15 ■   final NotificationType type;
16 ■   final String title;
17 ■   final String body;
18 ■   final DateTime receivedAt;
19 ■   final bool read;
20 ■
21 ■   factory AppNotification.fromMap(String id, Map<String, dynamic> map) =>
22 ■     AppNotification(
23 ■       id: id,
24 ■       type: NotificationType.parse((map['type'] as String?) ?? ''),
25 ■       title: (map['title'] as String?) ?? '',
26 ■       body: (map['body'] as String?) ?? '',
27 ■       receivedAt: _toDate(map['receivedAt']) ?? DateTime.now(),
28 ■       read: (map['read'] as bool?) ?? false,
29 ■     );
30 ■
31 ■   static DateTime? _toDate(Object? v) {
32 ■     if (v == null) return null;
33 ■     try {
34 ■       return (v as dynamic).toDate() as DateTime?;
35 ■     } catch (_) {
36 ■       return null;
37 ■     }
38 ■   }
39 ■ }
```

lib/models/audit_log.dart

42 lines

```

1 ■ class AuditLog {
2 ■   const AuditLog({
3 ■     required this.id,
4 ■     required this.actorUid,
5 ■     required this.actorRole,
6 ■     required this.action,
7 ■     required this.targetType,
8 ■     required this.targetId,
9 ■     required this.at,
10 ■    this.details = const <String, dynamic>{},
11 ■  });
12 ■
13 ■   final String id;
14 ■   final String actorUid;
15 ■   final String actorRole;
16 ■   final String action;
17 ■   final String targetType;
18 ■   final String targetId;
19 ■   final DateTime at;
20 ■   final Map<String, dynamic> details;
21 ■
22 ■   factory AuditLog.fromMap(String id, Map<String, dynamic> map) => AuditLog(
23 ■     id: id,
24 ■     actorUid: (map['actorUid'] as String?) ?? '',
25 ■     actorRole: (map['actorRole'] as String?) ?? '',
26 ■     action: (map['action'] as String?) ?? '',
27 ■     targetType:
28 ■       ((map['target'] as Map?)?['type'] as String?) ?? '',
29 ■     targetId: ((map['target'] as Map?)?['id'] as String?) ?? '',
30 ■     at: _toDate(map['at']) ?? DateTime.now(),
31 ■     details: Map<String, dynamic>.from(map['details'] as Map? ?? {}),
32 ■   );
33 ■
34 ■   static DateTime? _toDate(Object? v) {
35 ■     if (v == null) return null;
36 ■     try {
37 ■       return (v as dynamic).toDate() as DateTime?;
38 ■     } catch (_) {
39 ■       return null;
40 ■     }
41 ■   }
42 ■ }

```

5. Services

Firebase & platform gateways: auth, Firestore, live location, driver GPS trips, ETA, notifications, connectivity, admin Cloud Function calls.

```
auth_service.dart · firestore_service.dart · admin_functions_service.dart ·
live_location_service.dart · driver_trip_service.dart · eta_service.dart ·
notification_service.dart · connectivity_service.dart
```

lib/services/auth_service.dart

117 lines

```
1 ■ import 'package:firebase_auth/firebase_auth.dart';
2 ■ import '../core/constants/enums.dart';
3 ■ import '../core/utils/result.dart';
4 ■
5 ■ /// Session snapshot derived from the signed ID token (custom claims).
6 ■ /// Claims are the ONLY source of truth for authorization in the client;
7 ■ /// the backend re-enforces them in security rules regardless.
8 ■ class AuthSession {
9 ■   const AuthSession({
10 ■     required this.uid,
11 ■     required this.email,
12 ■     required this.role,
13 ■     this.busId,
14 ■   });
15 ■
16 ■   final String uid;
17 ■   final String email;
18 ■   final UserRole role;
19 ■   final String? busId;
20 ■ }
21 ■
22 ■ class AuthService {
23 ■   AuthService(this._auth);
24 ■
25 ■   final FirebaseAuth _auth;
26 ■
27 ■   Stream<User?> get authStateChanges => _auth.authStateChanges();
28 ■
29 ■   User? get currentUser => _auth.currentUser;
30 ■
31 ■   Future<Result<AuthSession>> signIn({
32 ■     required String email,
33 ■     required String password,
34 ■   }) async {
35 ■     try {
36 ■       await _auth.signInWithEmailAndPassword(
37 ■         email: email.trim(),
38 ■         password: password,
39 ■       );
40 ■       final AuthSession? session = await readSession(forceRefresh: true);
41 ■       if (session == null) {
42 ■         await _auth.signOut();
43 ■         return const Err<AuthSession>(
44 ■           AppFailure(
45 ■             'no-role',
46 ■             'This account is not set up yet. Please contact the school office.',
47 ■           ),
48 ■         );
49 ■       }
50 ■       return Ok<AuthSession>(session);
51 ■     } on FirebaseAuthException catch (e) {
52 ■       return Err<AuthSession>(_mapAuthError(e));
53 ■     } catch (_) {
54 ■       return const Err<AuthSession>(AppFailure.unknown);
55 ■     }
56 ■   }
57 ■
58 ■   /// Reads role/bus from custom claims. `forceRefresh` pulls a fresh token –
```

```

59 ■ /// used after login and on app resume so revoked access dies quickly.
60 ■ Future<AuthSession?> readSession({bool forceRefresh = false}) async {
61 ■   final User? user = _auth.currentUser;
62 ■   if (user == null) return null;
63 ■   try {
64 ■     final IdTokenResult token = await user.getIdTokenResult(forceRefresh);
65 ■     final Map<String, dynamic> claims = token.claims ?? <String, dynamic>{};
66 ■     final UserRole? role = UserRole.tryParse(claims['role'] as String?);
67 ■     final bool active = (claims['active'] as bool?) ?? true;
68 ■     if (role == null || !active) return null;
69 ■     return AuthSession(
70 ■       uid: user.uid,
71 ■       email: user.email ?? '',
72 ■       role: role,
73 ■       busId: claims['busId'] as String?,
74 ■     );
75 ■   } on FirebaseAuthException {
76 ■     // Token refresh failed → session revoked/expired.
77 ■     return null;
78 ■   }
79 ■ }
80 ■
81 ■ Future<Result<void>> sendPasswordReset(String email) async {
82 ■   try {
83 ■     await _auth.sendPasswordResetEmail(email: email.trim());
84 ■     return const Ok<void>(null);
85 ■   } on FirebaseAuthException catch (e) {
86 ■     return Err<void>(_mapAuthError(e));
87 ■   }
88 ■ }
89 ■
90 ■ Future<void> signOut() => _auth.signOut();
91 ■
92 ■ AppFailure _mapAuthError(FirebaseAuthException e) {
93 ■   switch (e.code) {
94 ■     case 'invalid-credential':
95 ■     case 'wrong-password':
96 ■     case 'user-not-found':
97 ■       return const AppFailure(
98 ■         'invalid-credential',
99 ■         'Email or password is incorrect. Please try again.',
100 ■       );
101 ■     case 'user-disabled':
102 ■       return const AppFailure(
103 ■         'user-disabled',
104 ■         'This account has been disabled. Please contact the school office.',
105 ■       );
106 ■     case 'too-many-requests':
107 ■       return const AppFailure(
108 ■         'too-many-requests',
109 ■         'Too many attempts. Please wait a moment and try again.',
110 ■       );
111 ■     case 'network-request-failed':
112 ■       return AppFailure.network;
113 ■     default:
114 ■       return AppFailure(e.code, 'Sign-in failed. Please try again.');
```

```

115 ■   }
116 ■ }
117 ■ }
```

[illegible]

```

59 // Routes / stops
60 Stream<Bus?> watchBus(String busId) =>
61     _db.collection('buses').doc(busId).snapshots().map(
62         (DocumentSnapshot<Map<String, dynamic>> s) =>
63             s.exists ? Bus.fromMap(s.id, s.data(!)) : null,
64     );
65
66 Stream<List<Bus>> watchAllBuses() =>
67     _db.collection('buses').orderBy('busNumber').snapshots().map(
68         (QuerySnapshot<Map<String, dynamic>> q) => q.docs
69             .map((QueryDocumentSnapshot<Map<String, dynamic>> d) =>
70                 Bus.fromMap(d.id, d.data()),)
71             .toList(),
72     );
73
74 Stream<BusRoute?> watchRouteOfBus(String busId) => _db
75     .collection('routes')
76     .where('busId', isEqualTo: busId)
77     .where('isActive', isEqualTo: true)
78     .limit(1)
79     .snapshots()
80     .map(
81         (QuerySnapshot<Map<String, dynamic>> q) => q.docs.isEmpty
82             ? null
83             : BusRoute.fromMap(q.docs.first.id, q.docs.first.data()),
84     );
85
86 Stream<List<BusStop>> watchStopsOfBus(String busId) => _db
87     .collection('stops')
88     .where('busId', isEqualTo: busId)
89     .where('isActive', isEqualTo: true)
90     .orderBy('order')
91     .snapshots()
92     .map(
93         (QuerySnapshot<Map<String, dynamic>> q) => q.docs
94             .map((QueryDocumentSnapshot<Map<String, dynamic>> d) =>
95                 BusStop.fromMap(d.id, d.data()),)
96             .toList(),
97     );
98
99 // Admin CRUD (rules restrict to admin claim)
100 Future<String> createBus(Bus bus) async {
101     final DocumentReference<Map<String, dynamic>> ref =
102         await _db.collection('buses').add(<String, dynamic>{
103             ...bus.toMap(),
104             'createdAt': FieldValue.serverTimestamp(),
105             'updatedAt': FieldValue.serverTimestamp(),
106         });
107     return ref.id;
108 }
109
110 Future<void> updateBus(String busId, Map<String, dynamic> data) =>
111     _db.collection('buses').doc(busId).update(<String, dynamic>{
112         ...data,
113         'updatedAt': FieldValue.serverTimestamp(),
114     });
115
116 Future<String> createStop(BusStop stop) async {

```

[illegible]

[illegible]

lib/services/admin_functions_service.dart

99 lines

```

1 ■ import 'package:cloud_functions/cloud_functions.dart' hide Result;
2 ■ import '../core/utils/result.dart';
3 ■
4 ■ /// Privileged operations. These NEVER touch Firestore directly from the
5 ■ /// client – each is a callable Cloud Function that:
6 ■ /// 1. verifies the caller's `admin` custom claim server-side,
7 ■ /// 2. performs the change atomically (Auth + claims + Firestore),
8 ■ /// 3. revokes refresh tokens where access changed,
9 ■ /// 4. writes an audit log entry.
10 ■ class AdminFunctionsService {
11 ■   AdminFunctionsService(this._functions);
12 ■
13 ■   final FirebaseFunctions _functions;
14 ■
15 ■   Future<Result<String>> provisionUser({
16 ■     required String email,
17 ■     required String fullName,
18 ■     required String role, // 'student' | 'driver' | 'admin'
19 ■     required String temporaryPassword,
20 ■     String? busId,
21 ■     String? stopId,
22 ■     String? classSection,
23 ■     String? phone,
24 ■   }) =>
25 ■     _call<String>('provisionUser', <String, dynamic>{
26 ■       'email': email,
27 ■       'fullName': fullName,
28 ■       'role': role,
29 ■       'temporaryPassword': temporaryPassword,
30 ■       'busId': busId,
31 ■       'stopId': stopId,
32 ■       'classSection': classSection,
33 ■       'phone': phone,
34 ■     }, (Object? data) => (data as Map)['uid'] as String,);
35 ■
36 ■   Future<Result<void>> assignUserToBus({
37 ■     required String uid,
38 ■     String? busId, // null = unassign
39 ■     String? stopId, // students only
40 ■   }) =>
41 ■     _call<void>('assignUserToBus', <String, dynamic>{
42 ■       'uid': uid,
43 ■       'busId': busId,
44 ■       'stopId': stopId,
45 ■     }, (_) {},);
46 ■
47 ■   Future<Result<void>> assignDriverToBus({
48 ■     required String driverUid,
49 ■     String? busId,
50 ■   }) =>
51 ■     _call<void>('assignDriverToBus', <String, dynamic>{
52 ■       'uid': driverUid,
53 ■       'busId': busId,
54 ■     }, (_) {},);
55 ■
56 ■   Future<Result<void>> setAccountActive({
57 ■     required String uid,
58 ■     required bool active,

```

```

59 ■ }) =>
60 ■   _call<void>('setAccountActive', <String, dynamic>{
61 ■     'uid': uid,
62 ■     'active': active,
63 ■   }, (_) {});
64 ■
65 ■ Future<Result<void>> sendAnnouncement({
66 ■   required String title,
67 ■   required String body,
68 ■   required String scope, // 'all' | 'bus' | 'user'
69 ■   String? busId,
70 ■   String? uid,
71 ■ }) =>
72 ■   _call<void>('sendAnnouncement', <String, dynamic>{
73 ■     'title': title,
74 ■     'body': body,
75 ■     'scope': scope,
76 ■     'busId': busId,
77 ■     'uid': uid,
78 ■   }, (_) {});
79 ■
80 ■ Future<Result<T>> _call<T>(  
81 ■   String name,  
82 ■   Map<String, dynamic> payload,  
83 ■   T Function(Object? data) parse,  
84 ■ ) async {  
85 ■   try {  
86 ■     final HttpsCallableResult<Object?> res =  
87 ■       await _functions.httpsCallable(name).call(payload);  
88 ■     return Ok<T>(parse(res.data));  
89 ■   } on FirebaseFunctionsException catch (e) {  
90 ■     if (e.code == 'permission-denied' || e.code == 'unauthenticated') {  
91 ■       return const Err(AppFailure.permissionDenied);  
92 ■     }  
93 ■     if (e.code == 'unavailable') return const Err(AppFailure.network);  
94 ■     return Err<T>(AppFailure(e.code, e.message ?? 'Operation failed.'));  
95 ■   } catch (_) {  
96 ■     return const Err(AppFailure.unknown);  
97 ■   }  
98 ■ }  
99 ■ }

```

lib/services/live_location_service.dart

51 lines

```

1 ■ import 'package:firebase_database/firebase_database.dart';
2 ■ import '../models/live_location.dart';
3 ■
4 ■ /// Reads/writes the single live-location node per bus in Realtime Database.
5 ■ ///
6 ■ /// RTDB security rules guarantee:
7 ■ /// - students can subscribe ONLY to `/liveLocations/{their claim busId}`
8 ■ /// - drivers can write ONLY to their own bus node
9 ■ /// so no client can browse other buses regardless of what this code does.
10 ■ class LiveLocationService {
11 ■   LiveLocationService(this._rtdb);
12 ■
13 ■   final FirebaseDatabase _rtdb;
14 ■
15 ■   DatabaseReference _busRef(String busId) =>
16 ■     _rtdb.ref('liveLocations/$busId');
17 ■
18 ■   /// Live stream of the bus location; emits null when the node is absent
19 ■   /// (no active trip) – an explicit, honest "not tracking" state.
20 ■   Stream<LiveLocation?> watchBusLocation(String busId) =>
21 ■     _busRef(busId).onValue.map((DatabaseEvent event) {
22 ■       final Object? value = event.snapshot.value;
23 ■       if (value is! Map) return null;
24 ■       return LiveLocation.fromMap(Map<Object?, Object?>.from(value));
25 ■     });
26 ■
27 ■   /// Driver-side write of one GPS fix. `updatedAt` uses the SERVER timestamp
28 ■   /// so freshness checks can't be spoofed by a wrong device clock.
29 ■   Future<void> publishFix({
30 ■     required String busId,
31 ■     required String tripId,
32 ■     required double lat,
33 ■     required double lng,
34 ■     required double heading,
35 ■     required double speedKmh,
36 ■     required double accuracy,
37 ■   }) =>
38 ■     _busRef(busId).set(<String, Object?>{
39 ■       'lat': lat,
40 ■       'lng': lng,
41 ■       'heading': heading,
42 ■       'speedKmh': speedKmh,
43 ■       'accuracy': accuracy,
44 ■       'tripId': tripId,
45 ■       'updatedAt': ServerValue.timestamp,
46 ■     });
47 ■
48 ■   /// Removes the node when the trip ends so students see "not tracking"
49 ■   /// instead of a frozen old position.
50 ■   Future<void> clear(String busId) => _busRef(busId).remove();
51 ■ }

```

```

1 ■ import 'dart:async';
2 ■
3 ■ import 'package:geolocator/geolocator.dart';
4 ■
5 ■ import '../core/constants/nmb_constants.dart';
6 ■ import '../core/utils/result.dart';
7 ■ import 'firestore_service.dart';
8 ■ import 'live_location_service.dart';
9 ■
10 ■ /// Orchestrates a driver trip:
11 ■ ///   Start Trip → create trips doc → stream GPS → publish to RTDB
12 ■ ///   End Trip   → stop stream → complete trips doc → clear RTDB node
13 ■ ///
14 ■ /// Uses a foreground-service-backed stream on Android (see
15 ■ /// AndroidSettings.foregroundNotificationConfig) so tracking survives the
16 ■ /// screen turning off – the driver should not need to touch the phone while
17 ■ /// driving.
18 ■ class DriverTripService {
19 ■   DriverTripService(this._firestore, this._liveLocation);
20 ■
21 ■   final FirestoreService _firestore;
22 ■   final LiveLocationService _liveLocation;
23 ■
24 ■   StreamSubscription<Position>? _positionSub;
25 ■   String? _activeTripId;
26 ■   String? _activeBusId;
27 ■
28 ■   bool get hasLocalActiveTrip => _activeTripId != null;
29 ■
30 ■   /// Checks GPS service + permission. Returns a failure describing exactly
31 ■   /// what's wrong so the UI can guide the driver.
32 ■   Future<Result<void>> ensureLocationReady() async {
33 ■     if (!await Geolocator.isLocationServiceEnabled()) {
34 ■       return const Err<void>(
35 ■         AppFailure('gps-off', 'GPS is turned off. Please enable Location.'),
36 ■       );
37 ■     }
38 ■     LocationPermission permission = await Geolocator.checkPermission();
39 ■     if (permission == LocationPermission.denied) {
40 ■       permission = await Geolocator.requestPermission();
41 ■     }
42 ■     if (permission == LocationPermission.denied ||
43 ■       permission == LocationPermission.deniedForever) {
44 ■       return const Err<void>(
45 ■         AppFailure(
46 ■           'location-denied',
47 ■           'Location permission is required to share the bus position. '
48 ■           'Please allow it in Settings.',
49 ■         ),
50 ■       );
51 ■     }
52 ■     return const Ok<void>(null);
53 ■   }
54 ■
55 ■   Future<Result<String>> startTrip({
56 ■     required String driverId,
57 ■     required String busId,
58 ■     required String routeId,

```

```

59 ■     required String direction,
60 ■ } ) async {
61 ■     final Result<void> ready = await ensureLocationReady();
62 ■     if (ready case Err<void>(:final AppFailure failure)) {
63 ■         return Err<String>(failure);
64 ■     }
65 ■
66 ■     try {
67 ■         final String tripId = await _firestore.createTrip(
68 ■             busId: busId,
69 ■             driverId: driverId,
70 ■             routeId: routeId,
71 ■             direction: direction,
72 ■         );
73 ■         _activeTripId = tripId;
74 ■         _activeBusId = busId;
75 ■         await _startPublishing(busId: busId, tripId: tripId);
76 ■         return Ok<String>(tripId);
77 ■     } catch (_) {
78 ■         return const Err<String>(
79 ■             AppFailure('trip-start', 'Could not start the trip. Please retry.'),
80 ■         );
81 ■     }
82 ■ }
83 ■
84 ■ Future<void> _startPublishing({
85 ■     required String busId,
86 ■     required String tripId,
87 ■ }) async {
88 ■     await _positionSub?.cancel();
89 ■
90 ■     final LocationSettings settings = AndroidSettings(
91 ■         accuracy: LocationAccuracy.high,
92 ■         distanceFilter: NmbConstants.locationDistanceFilterMeters,
93 ■         intervalDuration: NmbConstants.locationMinInterval,
94 ■         foregroundNotificationConfig: const ForegroundNotificationConfig(
95 ■             notificationTitle: 'Never Miss Bus – trip active',
96 ■             notificationText: 'Sharing bus location with assigned students.',
97 ■             enableWakeLock: true,
98 ■         ),
99 ■     );
100 ■
101 ■     _positionSub =
102 ■         Geolocator.getPositionStream(locationSettings: settings).listen(
103 ■             (Position pos) {
104 ■                 // Fire-and-forget: a missed packet is fine, freshness logic on the
105 ■                 // student side handles gaps honestly.
106 ■                 _liveLocation.publishFix(
107 ■                     busId: busId,
108 ■                     tripId: tripId,
109 ■                     lat: pos.latitude,
110 ■                     lng: pos.longitude,
111 ■                     heading: pos.heading,
112 ■                     speedKmh: pos.speed * 3.6,
113 ■                     accuracy: pos.accuracy,
114 ■                 );
115 ■             },
116 ■             onError: (_) {

```

```

117 ■      // GPS dropped – keep the subscription; geolocator resumes when the
118 ■      // signal returns. Students see "last updated X ago" meanwhile.
119 ■    },
120 ■  );
121 ■ }
122 ■
123 ■ Future<Result<void>> endTrip() async {
124 ■   final String? tripId = _activeTripId;
125 ■   final String? busId = _activeBusId;
126 ■   await _positionSub?.cancel();
127 ■   _positionSub = null;
128 ■   _activeTripId = null;
129 ■   _activeBusId = null;
130 ■
131 ■   if (tripId == null || busId == null) return const Ok<void>(null);
132 ■   try {
133 ■     await _firestore.endTrip(tripId);
134 ■     await _liveLocation.clear(busId);
135 ■     return const Ok<void>(null);
136 ■   } catch (_) {
137 ■     return const Err<void>(
138 ■       AppFailure(
139 ■         'trip-end',
140 ■         'Trip end could not be confirmed with the server. '
141 ■         'It will be closed automatically if you stay offline.',
142 ■       ),
143 ■     );
144 ■   }
145 ■ }
146 ■
147 ■ /// Re-attach publishing after app restart while a trip is still active
148 ■ /// (e.g. driver phone rebooted mid-route).
149 ■ Future<void> resumePublishing({
150 ■   required String busId,
151 ■   required String tripId,
152 ■ }) async {
153 ■   _activeTripId = tripId;
154 ■   _activeBusId = busId;
155 ■   await _startPublishing(busId: busId, tripId: tripId);
156 ■ }
157 ■
158 ■ Future<void> dispose() async => _positionSub?.cancel();
159 ■ }

```

lib/services/eta_service.dart

112 lines

```

1 ■ import '../core/constants/nmb_constants.dart';
2 ■ import '../models/bus_stop.dart';
3 ■ import '../models/geo_point_data.dart';
4 ■ import '../models/live_location.dart';
5 ■
6 ■ /// Result of an ETA computation. Either a usable approximation, or an
7 ■ explicit "unavailable" – the UI never invents a number.
8 ■ sealed class EtaResult {
9 ■   const EtaResult();
10 ■ }
11 ■
12 ■ class EtaAvailable extends EtaResult {
13 ■   const EtaAvailable(this.duration, this.distanceMeters);
14 ■   final Duration duration;
15 ■   final double distanceMeters;
16 ■ }
17 ■
18 ■ class EtaUnavailable extends EtaResult {
19 ■   const EtaUnavailable(this.reason);
20 ■   final String reason;
21 ■ }
22 ■
23 ■ /// Conservative ETA: distance along the remaining stop sequence divided by a
24 ■ rolling average speed with a sane floor. Deliberately simple and honest –
25 ■ no traffic model, so wording elsewhere is always "approximately".
26 ■ class EtaService {
27 ■   final List<double> _recentSpeeds = <double>[];
28 ■
29 ■   void recordSpeed(double speedKmh) {
30 ■     if (speedKmh.isNaN || speedKmh < 0 || speedKmh > 120) return;
31 ■     _recentSpeeds.add(speedKmh);
32 ■     if (_recentSpeeds.length > 12) _recentSpeeds.removeAt(0);
33 ■   }
34 ■
35 ■   EtaResult etaToStop({
36 ■     required LiveLocation? live,
37 ■     required BusStop targetStop,
38 ■     required List<BusStop> orderedStops,
39 ■   }) {
40 ■     if (live == null) {
41 ■       return const EtaUnavailable('Bus location not available');
42 ■     }
43 ■
44 ■     // Never estimate from stale data.
45 ■     final Duration age = DateTime.now().difference(live.updatedAt);
46 ■     if (age > NmbConstants.staleThreshold) {
47 ■       return const EtaUnavailable('Location data too old for an estimate');
48 ■     }
49 ■
50 ■     final double distance = _remainingDistanceMeters(
51 ■       from: live.position,
52 ■       target: targetStop,
53 ■       orderedStops: orderedStops,
54 ■     );
55 ■
56 ■     if (distance > NmbConstants.etaMaxReliableDistanceKm * 1000) {
57 ■       return const EtaUnavailable('Bus too far for a reliable estimate');
58 ■     }

```

```

59 ■ // Bus already at/past the stop.
60 ■ if (distance < 60) {
61 ■   return EtaAvailable(const Duration(minutes: 1), distance);
62 ■ }
63 ■
64 ■ final double avgSpeed = _recentSpeeds.isEmpty
65 ■   ? NmbConstants.etaMinSpeedKmh
66 ■   : _recentSpeeds.reduce((double a, double b) => a + b) /
67 ■     _recentSpeeds.length;
68 ■ final double effectiveKmh =
69 ■   avgSpeed < NmbConstants.etaMinSpeedKmh
70 ■     ? NmbConstants.etaMinSpeedKmh
71 ■     : avgSpeed;
72 ■
73 ■ final double hours = (distance / 1000) / effectiveKmh;
74 ■ final Duration eta = Duration(seconds: (hours * 3600).round());
75 ■
76 ■ if (eta > const Duration(hours: 2)) {
77 ■   return const EtaUnavailable('Estimate not reliable right now');
78 ■ }
79 ■ return EtaAvailable(eta, distance);
80 ■ }
81 ■
82 ■ /// Distance: bus → next stop → ... → target stop (follows the route order,
83 ■ /// not a straight line to the target).
84 ■ double _remainingDistanceMeters({
85 ■   required GeoPointData from,
86 ■   required BusStop target,
87 ■   required List<BusStop> orderedStops,
88 ■ }) {
89 ■   final int targetIndex =
90 ■     orderedStops.indexWhere((BusStop s) => s.id == target.id);
91 ■   if (targetIndex < 0) return from.distanceTo(target.location);
92 ■
93 ■   // Find the upcoming stop = nearest stop at/before the target.
94 ■   int nearestIndex = 0;
95 ■   double nearest = double.infinity;
96 ■   for (int i = 0; i <= targetIndex; i++) {
97 ■     final double d = from.distanceTo(orderedStops[i].location);
98 ■     if (d < nearest) {
99 ■       nearest = d;
100 ■       nearestIndex = i;
101 ■     }
102 ■   }
103 ■
104 ■   double total = from.distanceTo(orderedStops[nearestIndex].location);
105 ■   for (int i = nearestIndex; i < targetIndex; i++) {
106 ■     total += orderedStops[i]
107 ■       .location
108 ■       .distanceTo(orderedStops[i + 1].location);
109 ■   }
110 ■   return total;
111 ■ }
112 ■ }

```

```

1 ■ import 'package:firebase_messaging/firebase_messaging.dart';
2 ■ import 'package:flutter_local_notifications/flutter_local_notifications.dart';
3 ■
4 ■ import 'firestore_service.dart';
5 ■
6 ■ /// FCM wiring:
7 ■ /// - requests notification permission (Android 13+ / iOS),
8 ■ /// - registers the device token under users/{uid}.fcmTokens (rules allow
9 ■ ///   only the owner to write this field),
10 ■ /// - shows foreground pushes via a local notification channel.
11 ■ ///
12 ■ /// Targeting is done entirely by Cloud Functions against explicit tokens –
13 ■ /// no FCM topics, because any client can subscribe to a topic and that would
14 ■ /// leak bus-scoped notifications.
15 ■ class NotificationService {
16 ■   NotificationService(this._messaging, this._firestore);
17 ■
18 ■   final FirebaseMessaging _messaging;
19 ■   final FirestoreService _firestore;
20 ■   final FlutterLocalNotificationsPlugin _local =
21 ■     FlutterLocalNotificationsPlugin();
22 ■
23 ■   static const AndroidNotificationChannel _channel =
24 ■     AndroidNotificationChannel(
25 ■       'nmb_alerts',
26 ■       'Bus Alerts',
27 ■       description: 'Trip updates, arrival alerts and school announcements.',
28 ■       importance: Importance.high,
29 ■     );
30 ■
31 ■   bool _permissionGranted = false;
32 ■   bool get permissionGranted => _permissionGranted;
33 ■
34 ■   Future<void> initialize() async {
35 ■     const InitializationSettings initSettings = InitializationSettings(
36 ■       android: AndroidInitializationSettings('@mipmap/ic_launcher'),
37 ■       iOS: DarwinInitializationSettings(),
38 ■     );
39 ■     await _local.initialize(initSettings);
40 ■     await _local
41 ■       .resolvePlatformSpecificImplementation<
42 ■         AndroidFlutterLocalNotificationsPlugin>()
43 ■       ?.createNotificationChannel(_channel);
44 ■
45 ■     final NotificationSettings settings = await _messaging.requestPermission(
46 ■       alert: true,
47 ■       badge: true,
48 ■       sound: true,
49 ■     );
50 ■     _permissionGranted =
51 ■       settings.authorizationStatus == AuthorizationStatus.authorized ||
52 ■       settings.authorizationStatus == AuthorizationStatus.provisional;
53 ■
54 ■     // Foreground messages → local notification (FCM shows nothing in
55 ■     // foreground by default).
56 ■     FirebaseMessaging.onMessage.listen((RemoteMessage message) {
57 ■       final RemoteNotification? n = message.notification;
58 ■       if (n == null) return;

```

```

59 ■      _local.show(
60 ■          n.hashCode,
61 ■          n.title,
62 ■          n.body,
63 ■          const NotificationDetails(
64 ■              android: AndroidNotificationDetails(
65 ■                  'nmb_alerts',
66 ■                  'Bus Alerts',
67 ■                  importance: Importance.high,
68 ■                  priority: Priority.high,
69 ■              ),
70 ■              iOS: DarwinNotificationDetails(),
71 ■          ),
72 ■      );
73 ■  });
74 ■ }
75 ■
76 ■  /// Call after login; keeps the token registered for this user.
77 ■  Future<void> registerDevice(String uid) async {
78 ■      try {
79 ■          final String? token = await _messaging.getToken();
80 ■          if (token != null) await _firestore.saveFcmToken(uid, token);
81 ■          _messaging.onTokenRefresh
82 ■              .listen((String t) => _firestore.saveFcmToken(uid, t));
83 ■      } catch (_) {
84 ■          // Non-fatal: push simply won't arrive on this device.
85 ■      }
86 ■  }
87 ■
88 ■  /// Call on logout so the signed-out device stops receiving user pushes.
89 ■  Future<void> unregisterDevice(String uid) async {
90 ■      try {
91 ■          final String? token = await _messaging.getToken();
92 ■          if (token != null) await _firestore.removeFcmToken(uid, token);
93 ■          await _messaging.deleteToken();
94 ■      } catch (_) {/* best effort */}
95 ■  }
96 ■ }

```

lib/services/connectivity_service.dart

20 lines

```

1 ■ import 'package:connectivity_plus/connectivity_plus.dart';
2 ■
3 ■ /// Simple online/offline stream used for offline banners and driver
4 ■ /// connection status.
5 ■ class ConnectivityService {
6 ■     ConnectivityService(this._connectivity);
7 ■
8 ■     final Connectivity _connectivity;
9 ■
10 ■     Stream<bool> get isOnline =>
11 ■         _connectivity.onConnectivityChanged.map(_hasConnection);
12 ■
13 ■     Future<bool> checkOnline() async =>
14 ■         _hasConnection(await _connectivity.checkConnectivity());
15 ■
16 ■     bool _hasConnection(List<ConnectivityResult> results) => results.any(
17 ■         (ConnectivityResult r) =>
18 ■             r != ConnectivityResult.none && r != ConnectivityResult.bluetooth,
19 ■     );
20 ■ }

```

```
app_providers.dart · data_providers.dart · app_router.dart
```

88 lines

Page 47


```

1 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
2 ■
3 ■ import '../core/constants/enums.dart';
4 ■ import '../models/app_notification.dart';
5 ■ import '../models/app_user.dart';
6 ■ import '../models/audit_log.dart';
7 ■ import '../models/bus.dart';
8 ■ import '../models/bus_route.dart';
9 ■ import '../models/bus_stop.dart';
10 ■ import '../models/live_location.dart';
11 ■ import '../models/trip.dart';
12 ■ import '../services/auth_service.dart';
13 ■ import 'app_providers.dart';
14 ■
15 ■ /// Profile of the signed-in user.
16 ■ final StreamProvider<AppUser?> myProfileProvider =
17 ■   StreamProvider<AppUser?>((Ref ref) {
18 ■     final AuthSession? session = ref.watch(currentSessionProvider);
19 ■     if (session == null) return Stream<AppUser?>.value(null);
20 ■     return ref.watch(firestoreServiceProvider).watchUser(session.uid);
21 ■   });
22 ■
23 ■ /// The signed-in user's assigned bus (students & drivers).
24 ■ /// Access is enforced by rules: the query only succeeds for the claim busId.
25 ■ final StreamProvider<Bus?> myBusProvider = StreamProvider<Bus?>((Ref ref) {
26 ■   final AuthSession? session = ref.watch(currentSessionProvider);
27 ■   final String? busId = session?.busId;
28 ■   if (busId == null || busId.isEmpty) return Stream<Bus?>.value(null);
29 ■   return ref.watch(firestoreServiceProvider).watchBus(busId);
30 ■ });
31 ■
32 ■ final StreamProvider<BusRoute?> myRouteProvider =
33 ■   StreamProvider<BusRoute?>((Ref ref) {
34 ■     final String? busId = ref.watch(currentSessionProvider)?.busId;
35 ■     if (busId == null || busId.isEmpty) return Stream<BusRoute?>.value(null);
36 ■     return ref.watch(firestoreServiceProvider).watchRouteOfBus(busId);
37 ■   });
38 ■
39 ■ final StreamProvider<List<BusStop>> myStopsProvider =
40 ■   StreamProvider<List<BusStop>>((Ref ref) {
41 ■     final String? busId = ref.watch(currentSessionProvider)?.busId;
42 ■     if (busId == null || busId.isEmpty) {
43 ■       return Stream<List<BusStop>>.value(const <BusStop>[]);
44 ■     }
45 ■     return ref.watch(firestoreServiceProvider).watchStopsOfBus(busId);
46 ■   });
47 ■
48 ■ /// The student's own assigned stop.
49 ■ final Provider<BusStop?> myAssignedStopProvider = Provider<BusStop?>((Ref ref) {
50 ■   final AppUser? me = ref.watch(myProfileProvider).valueOrNull;
51 ■   final List<BusStop> stops =
52 ■     ref.watch(myStopsProvider).valueOrNull ?? const <BusStop>[];
53 ■   if (me?.stopId == null) return null;
54 ■   for (final BusStop s in stops) {
55 ■     if (s.id == me!.stopId) return s;
56 ■   }
57 ■   return null;
58 ■ });

```

```

59 ■
60 ■ /// Active trip of my bus (null → "Bus has no active trip").
61 ■ final StreamProvider<Trip?> myBusActiveTripProvider =
62 ■     StreamProvider<Trip?>((Ref ref) {
63 ■         final String? busId = ref.watch(currentSessionProvider)?.busId;
64 ■         if (busId == null || busId.isEmpty) return Stream<Trip?>.value(null);
65 ■         return ref.watch(firestoreServiceProvider).watchActiveTripOfBus(busId);
66 ■     });
67 ■
68 ■ /// Live location of my bus. Rules restrict this to the assigned bus.
69 ■ final StreamProvider<LiveLocation?> myBusLiveLocationProvider =
70 ■     StreamProvider<LiveLocation?>((Ref ref) {
71 ■         final String? busId = ref.watch(currentSessionProvider)?.busId;
72 ■         if (busId == null || busId.isEmpty) {
73 ■             return Stream<LiveLocation?>.value(null);
74 ■         }
75 ■         return ref.watch(liveLocationServiceProvider).watchBusLocation(busId);
76 ■     });
77 ■
78 ■ /// Freshness ticker: re-evaluates freshness every 10s even without new data,
79 ■ /// so a silent GPS flips LIVE → stale → unavailable honestly.
80 ■ final StreamProvider<DateTime> freshnessTickProvider =
81 ■     StreamProvider<DateTime>((Ref ref) => Stream<DateTime>.periodic(
82 ■         const Duration(seconds: 10),
83 ■         (_) => DateTime.now(),
84 ■     ),
85 ■ );
86 ■
87 ■
88 ■ final Provider<LocationFreshness> myBusFreshnessProvider =
89 ■     Provider<LocationFreshness>((Ref ref) {
90 ■         ref.watch(freshnessTickProvider); // tick
91 ■         final LiveLocation? live = ref.watch(myBusLiveLocationProvider).valueOrNull;
92 ■         final Trip? trip = ref.watch(myBusActiveTripProvider).valueOrNull;
93 ■         if (live == null || trip == null) return LocationFreshness.unavailable;
94 ■         return live.freshness();
95 ■     });
96 ■
97 ■ /// Notification inbox.
98 ■ final StreamProvider<List<AppNotification>> inboxProvider =
99 ■     StreamProvider<List<AppNotification>>((Ref ref) {
100 ■         final AuthSession? session = ref.watch(currentSessionProvider);
101 ■         if (session == null) {
102 ■             return Stream<List<AppNotification>>.value(const <AppNotification>[]);
103 ■         }
104 ■         return ref.watch(firestoreServiceProvider).watchInbox(session.uid);
105 ■     });
106 ■
107 ■ final Provider<int> unreadCountProvider = Provider<int>((Ref ref) {
108 ■     final List<AppNotification> inbox =
109 ■         ref.watch(inboxProvider).valueOrNull ?? const <AppNotification>[];
110 ■     return inbox.where((AppNotification n) => !n.read).length;
111 ■ });
112 ■
113 ■ /// School contact/config.
114 ■ final StreamProvider<Map<String, dynamic>> schoolConfigProvider =
115 ■     StreamProvider<Map<String, dynamic>>((Ref ref) => ref.watch(firestoreServiceProvider).watchSchoolConfig(),

```

[illegible]

lib/router/app_router.dart

193 lines

```

1 ■
2 ■ import 'package:flutter/material.dart';
3 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
4 ■ import 'package:go_router/go_router.dart';
5 ■
6 ■ import '../core/constants/enums.dart';
7 ■ import '../features/admin/admin_shell.dart';
8 ■ import '../features/admin/screens/admin_audit_logs_screen.dart';
9 ■ import '../features/admin/screens/admin_buses_screen.dart';
10 ■ import '../features/admin/screens/admin_dashboard_screen.dart';
11 ■ import '../features/admin/screens/admin_drivers_screen.dart';
12 ■ import '../features/admin/screens/admin_monitoring_screen.dart';
13 ■ import '../features/admin/screens/admin_notifications_screen.dart';
14 ■ import '../features/admin/screens/admin_routes_stops_screen.dart';
15 ■ import '../features/admin/screens/admin_settings_screen.dart';
16 ■ import '../features/admin/screens/admin_students_screen.dart';
17 ■ import '../features/auth/login_screen.dart';
18 ■ import '../features/auth/splash_screen.dart';
19 ■ import '../features/driver/driver_shell.dart';
20 ■ import '../features/driver/screens/driver_home_screen.dart';
21 ■ import '../features/driver/screens/driver_profile_screen.dart';
22 ■ import '../features/driver/screens/driver_route_screen.dart';
23 ■ import '../features/student/student_shell.dart';
24 ■ import '../features/student/screens/student_alerts_screen.dart';
25 ■ import '../features/student/screens/student_bus_details_screen.dart';
26 ■ import '../features/student/screens/student_home_screen.dart';
27 ■ import '../features/student/screens/student_map_screen.dart';
28 ■ import '../features/student/screens/student_profile_screen.dart';
29 ■ import '../features/student/screens/student_settings_screen.dart';
30 ■ import '../features/student/screens/student_stops_screen.dart';
31 ■ import '../providers/app_providers.dart';
32 ■ import '../services/auth_service.dart';
33 ■
34 ■ /// Role-aware router. The redirect runs on EVERY navigation and every auth
35 ■ /// change, so:
36 ■ /// - signed-out users can only reach /login,
37 ■ /// - each role is fenced into its own subtree (a student typing an /admin
38 ■ ///   URL is bounced home),
39 ■ /// - expired/revoked sessions bounce to /login automatically.
40 ■ /// This is UX-level fencing only – real enforcement lives in backend rules.
41 ■ final Provider<GoRouter> appRouterProvider = Provider<GoRouter>((Ref ref) {
42 ■   final ValueNotifier<int> refresh = ValueNotifier<int>(0);
43 ■   ref.listen(sessionProvider, (Object? _, Object? __) => refresh.value++);
44 ■   ref.onDispose(refresh.dispose);
45 ■
46 ■   return GoRouter(
47 ■     initialLocation: '/splash',
48 ■     refreshListenable: refresh,
49 ■     redirect: (BuildContext context, GoRouterState state) {
50 ■       final AsyncValue<AuthSession?> async = ref.read(sessionProvider);
51 ■       if (async.isLoading) {
52 ■         return state.matchedLocation == '/splash' ? null : '/splash';
53 ■       }
54 ■       final AuthSession? session = async.valueOrNull;
55 ■       final String loc = state.matchedLocation;
56 ■       final bool onAuthScreen = loc == '/login' || loc == '/splash';
57 ■
58 ■       if (session == null) return onAuthScreen ? (loc == '/splash' ? '/login' : null) : '/login'

```

```

59  ;
60
61  final String home = switch (session.role) {
62    UserRole.student => '/student/home',
63    UserRole.driver => '/driver/home',
64    UserRole.admin => '/admin/dashboard',
65  };
66  if (onAuthScreen) return home;
67
68  final String fence = '/${session.role.name}';
69  if (!loc.startsWith(fence)) return home;
70  return null;
71 },
72 routes: <RouteBase>[
73   GoRoute(path: '/splash', builder: (_, __) => const SplashScreen()),
74   GoRoute(path: '/login', builder: (_, __) => const LoginScreen()),
75
76   // Student shell
77   StatefulShellRoute.indexedStack(
78     builder: (_, __, StatefulNavigationShell shell) =>
79       StudentShell(shell: shell),
80     branches: <StatefulShellBranch>[
81       StatefulShellBranch(routes: <RouteBase>[
82         GoRoute(
83           path: '/student/home',
84           builder: (_, __) => const StudentHomeScreen(),
85           routes: <RouteBase>[
86             GoRoute(
87               path: 'bus',
88               builder: (_, __) => const StudentBusDetailsScreen(),
89             ),
90             GoRoute(
91               path: 'stops',
92               builder: (_, __) => const StudentStopsScreen(),
93             ),
94           ],
95         ),
96       ],),
97       StatefulShellBranch(routes: <RouteBase>[
98         GoRoute(
99           path: '/student/map',
100           builder: (_, __) => const StudentMapScreen(),
101         ),
102       ],),
103       StatefulShellBranch(routes: <RouteBase>[
104         GoRoute(
105           path: '/student/alerts',
106           builder: (_, __) => const StudentAlertsScreen(),
107         ),
108       ],),
109       StatefulShellBranch(routes: <RouteBase>[
110         GoRoute(
111           path: '/student/profile',
112           builder: (_, __) => const StudentProfileScreen(),
113           routes: <RouteBase>[
114             GoRoute(
115               path: 'settings',
116               builder: (_, __) => const StudentSettingsScreen(),

```

[illegible]

7. Auth Screens

Splash and the single role-aware login.

splash_screen.dart · login_screen.dart

lib/features/auth/splash_screen.dart

68 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■
3 ■ import '../core/constants/nmb_constants.dart';
4 ■ import '../core/theme/nmb_colors.dart';
5 ■ import '../core/theme/nmb_typography.dart';
6 ■
7 ■ /// Branded splash while the session is being restored. Routing away happens
8 ■ /// automatically via the router's redirect when sessionProvider resolves.
9 ■ class SplashScreen extends StatelessWidget {
10 ■   const SplashScreen({super.key});
11 ■
12 ■   @override
13 ■   Widget build(BuildContext context) {
14 ■     return Scaffold(
15 ■       backgroundColor: NmbColors.primary,
16 ■       body: Center(
17 ■         child: Column(
18 ■           mainAxisAlignment: MainAxisAlignment.center,
19 ■           children: <Widget>[
20 ■             Container(
21 ■               width: 108,
22 ■               height: 108,
23 ■               decoration: BoxDecoration(
24 ■                 color: Colors.white,
25 ■                 borderRadius: BorderRadius.circular(30),
26 ■                 boxShadow: const <BoxShadow>[
27 ■                   BoxShadow(
28 ■                     color: Color(0x33000000),
29 ■                     blurRadius: 30,
30 ■                     offset: Offset(0, 12),
31 ■                   ),
32 ■                 ],
33 ■               ),
34 ■               child: const Icon(
35 ■                 Icons.directions_bus_rounded,
36 ■                 size: 60,
37 ■                 color: NmbColors.accent,
38 ■               ),
39 ■             ),
40 ■             const SizedBox(height: 28),
41 ■             Text(
42 ■               NmbConstants.appName.toUpperCase(),
43 ■               style: NmbTypography.displayTitle.copyWith(
44 ■                 color: Colors.white,
45 ■                 letterSpacing: 1.5,
46 ■               ),
47 ■             ),
48 ■             const SizedBox(height: 6),
49 ■             Text(
50 ■               NmbConstants.schoolName,
51 ■               style: NmbTypography.bodySecondary
52 ■                 .copyWith(color: Colors.white70),
53 ■             ),
54 ■             const SizedBox(height: 48),
55 ■             const SizedBox(
56 ■               width: 26,
57 ■               height: 26,
58 ■               child: CircularProgressIndicator(
59 ■
60 ■                 strokeWidth: 2.6,
61 ■                 color: Colors.white,
62 ■               ),
63 ■             ),
64 ■           ],
65 ■         ),
66 ■       );
67 ■   }
68 ■ }

```

lib/features/auth/login_screen.dart

202 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/constants/nmb_constants.dart';
5 ■ import '../core/theme/nmb_colors.dart';
6 ■ import '../core/theme/nmb_typography.dart';
7 ■ import '../core/utils/result.dart';
8 ■ import '../core/utils/validators.dart';
9 ■ import '../core/widgets/nmb_dialogs.dart';
10 ■ import '../core/widgets/responsive_scaffold_body.dart';
11 ■ import '../providers/app_providers.dart';
12 ■ import '../services/auth_service.dart';
13 ■
14 ■ /// One login for all roles – the account's custom claims decide where the
15 ■ /// router sends the user. Students/drivers/admins cannot pick a role.
16 ■ class LoginScreen extends ConsumerStatefulWidget {
17 ■   const LoginScreen({super.key});
18 ■
19 ■   @override
20 ■   ConsumerState<LoginScreen> createState() => _LoginScreenState();
21 ■ }
22 ■
23 ■ class _LoginScreenState extends ConsumerState<LoginScreen> {
24 ■   final GlobalKey<FormState> _formKey = GlobalKey<FormState>();
25 ■   final TextEditingController _emailController = TextEditingController();
26 ■   final TextEditingController _passwordController = TextEditingController();
27 ■   bool _obscure = true;
28 ■   bool _loading = false;
29 ■
30 ■   @override
31 ■   void dispose() {
32 ■     _emailController.dispose();
33 ■     _passwordController.dispose();
34 ■     super.dispose();
35 ■   }
36 ■
37 ■   Future<void> _signIn() async {
38 ■     if (!(_formKey.currentState?.validate() ?? false)) return;
39 ■     setState(() => _loading = true);
40 ■     final Result<AuthSession> result =
41 ■       await ref.read(authServiceProvider).signIn(
42 ■         email: _emailController.text,
43 ■         password: _passwordController.text,
44 ■       );
45 ■     if (!mounted) return;
46 ■     setState(() => _loading = false);
47 ■     result.when(
48 ■       ok: (_) {}, // router redirect handles navigation
49 ■       err: (AppFailure f) => showNmbSnack(context, f.message, isError: true),
50 ■     );
51 ■   }
52 ■
53 ■   Future<void> _forgotPassword() async {
54 ■     final String email = _emailController.text.trim();
55 ■     if (Validators.email(email) != null) {
56 ■       showNmbSnack(
57 ■         context,
58 ■         'Enter your school email above first, then tap "Forgot password".',

```

```

59 ■      );
60 ■      return;
61 ■    }
62 ■    final Result<void> result =
63 ■      await ref.read(authServiceProvider).sendPasswordReset(email);
64 ■    if (!mounted) return;
65 ■    result.when(
66 ■      ok: (_) => showNmbSnack(
67 ■        context,
68 ■        'Password reset email sent to $email.',
69 ■        isSuccess: true,
70 ■      ),
71 ■      err: (AppFailure f) => showNmbSnack(context, f.message, isError: true),
72 ■    );
73 ■  }
74 ■
75 ■  @override
76 ■  Widget build(BuildContext context) {
77 ■    return Scaffold(
78 ■      body: ResponsiveBody(
79 ■        child: Column(
80 ■          crossAxisAlignment: CrossAxisAlignment.stretch,
81 ■          children: <Widget>[
82 ■            const SizedBox(height: 48),
83 ■            Center(
84 ■              child: Container(
85 ■                width: 84,
86 ■                height: 84,
87 ■                decoration: BoxDecoration(
88 ■                  color: NmbColors.accentSoft,
89 ■                  borderRadius: BorderRadius.circular(24),
90 ■                ),
91 ■                child: const Icon(
92 ■                  Icons.directions_bus_rounded,
93 ■                  size: 46,
94 ■                  color: NmbColors.accentDark,
95 ■                ),
96 ■              ),
97 ■            ),
98 ■            const SizedBox(height: 24),
99 ■            const Text(
100 ■              'Welcome back!',
101 ■              style: NmbTypography.displayTitle,
102 ■              textAlign: TextAlign.center,
103 ■            ),
104 ■            const SizedBox(height: 8),
105 ■            const Text(
106 ■              'Sign in with your ${NmbConstants.schoolName} account',
107 ■              style: NmbTypography.bodySecondary,
108 ■              textAlign: TextAlign.center,
109 ■            ),
110 ■            const SizedBox(height: 36),
111 ■            Form(
112 ■              key: _formKey,
113 ■              child: Column(
114 ■                children: <Widget>[
115 ■                  TextFormField(
116 ■                    controller: _emailController,

```

```

117 ■         keyboardType: TextInputType.emailAddress,
118 ■         autofillHints: const <String>[AutofillHints.username],
119 ■         textInputAction: TextInputAction.next,
120 ■         decoration: const InputDecoration(
121 ■             hintText: 'School email',
122 ■             prefixIcon: Icon(Icons.mail_outline_rounded),
123 ■         ),
124 ■         validator: Validators.email,
125 ■     ),
126 ■     const SizedBox(height: 14),
127 ■     TextFormField(
128 ■         controller: _passwordController,
129 ■         obscureText: _obscure,
130 ■         autofillHints: const <String>[AutofillHints.password],
131 ■         onFieldSubmitted: (_) => _signIn(),
132 ■         decoration: InputDecoration(
133 ■             hintText: 'Password',
134 ■             prefixIcon: const Icon(Icons.lock_outline_rounded),
135 ■             suffixIcon: IconButton(
136 ■                 tooltip:
137 ■                     _obscure ? 'Show password' : 'Hide password',
138 ■                 icon: Icon(
139 ■                     _obscure
140 ■                         ? Icons.visibility_outlined
141 ■                         : Icons.visibility_off_outlined,
142 ■                 ),
143 ■                 onPressed: () =>
144 ■                     setState(() => _obscure = !_obscure),
145 ■             ),
146 ■         ),
147 ■         validator: Validators.password,
148 ■     ),
149 ■ ],
150 ■ ),
151 ■ ),
152 ■ const SizedBox(height: 10),
153 ■ Align(
154 ■     alignment: Alignment.centerRight,
155 ■     child: TextButton(
156 ■         onPressed: _loading ? null : _forgotPassword,
157 ■         child: const Text('Forgot password?'),
158 ■     ),
159 ■ ),
160 ■ const SizedBox(height: 12),
161 ■ FilledButton(
162 ■     onPressed: _loading ? null : _signIn,
163 ■     child: _loading
164 ■         ? const SizedBox(
165 ■             width: 22,
166 ■             height: 22,
167 ■             child: CircularProgressIndicator(
168 ■                 strokeWidth: 2.4,
169 ■                 color: Colors.white,
170 ■             ),
171 ■         )
172 ■         : const Text('Sign in'),
173 ■ ),
174 ■ const SizedBox(height: 28),

```

```
175 ■      Container(  
176 ■        padding: const EdgeInsets.all(14),  
177 ■        decoration: BoxDecoration(  
178 ■          color: NmbColors.infoSoft,  
179 ■          borderRadius: BorderRadius.circular(14),  
180 ■        ),  
181 ■        child: Row(  
182 ■          children: <Widget>[  
183 ■            const Icon(Icons.school_outlined,  
184 ■              color: NmbColors.info, size: 22,),  
185 ■            const SizedBox(width: 10),  
186 ■            Expanded(  
187 ■              child: Text(  
188 ■                'Accounts are created by the school. If you don\'t '  
189 ■                'have one, please contact the school office.',  
190 ■                style: NmbTypography.caption  
191 ■                  .copyWith(color: NmbColors.info),  
192 ■              ),  
193 ■            ),  
194 ■          ],  
195 ■        ),  
196 ■      ),  
197 ■    ],  
198 ■  ),  
199 ■ ),  
200 ■ );  
201 ■ }  
202 ■ }
```

8. Student App

Shell, home, live map, bus details, stops, alerts, profile, settings + shared widgets.

student_shell.dart · student_home_screen.dart · tracking_status_card.dart ·
 student_map_screen.dart · bus_marker_icon.dart · student_bus_details_screen.dart ·
 student_stops_screen.dart · student_alerts_screen.dart · student_profile_screen.dart ·
 student_settings_screen.dart

lib/features/student/student_shell.dart

66 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■ import 'package:go_router/go_router.dart';
4 ■
5 ■ import '../core/widgets/state_views.dart';
6 ■ import '../providers/app_providers.dart';
7 ■ import '../providers/data_providers.dart';
8 ■
9 ■ /// Student bottom-navigation shell: Home · Live Map · Alerts · Profile.
10 ■ class StudentShell extends ConsumerWidget {
11 ■   const StudentShell({super.key, required this.shell});
12 ■
13 ■   final StatefulNavigationShell shell;
14 ■
15 ■   @override
16 ■   Widget build(BuildContext context, WidgetRef ref) {
17 ■     final bool online = ref.watch(isOnlineProvider).valueOrNull ?? true;
18 ■     final int unread = ref.watch(unreadCountProvider);
19 ■
20 ■     return Scaffold(
21 ■       body: Column(
22 ■         children: <Widget>[
23 ■           if (!online) const OfflineBanner(),
24 ■           Expanded(child: shell),
25 ■         ],
26 ■       ),
27 ■       bottomNavigationBar: NavigationBar(
28 ■         selectedIndex: shell.currentIndex,
29 ■         onDestinationSelected: (int index) => shell.goBranch(
30 ■           index,
31 ■           initialLocation: index == shell.currentIndex,
32 ■         ),
33 ■         destinations: <Widget>[
34 ■           const NavigationDestination(
35 ■             icon: Icon(Icons.home_outlined),
36 ■             selectedIcon: Icon(Icons.home_rounded),
37 ■             label: 'Home',
38 ■           ),
39 ■           const NavigationDestination(
40 ■             icon: Icon(Icons.map_outlined),
41 ■             selectedIcon: Icon(Icons.map_rounded),
42 ■             label: 'Live Map',
43 ■           ),
44 ■           NavigationDestination(
45 ■             icon: Badge(
46 ■               isLabelVisible: unread > 0,
47 ■               label: Text('$unread'),
48 ■               child: const Icon(Icons.notifications_outlined),
49 ■             ),
50 ■             selectedIcon: Badge(
51 ■               isLabelVisible: unread > 0,
52 ■               label: Text('$unread'),
53 ■               child: const Icon(Icons.notifications_rounded),
54 ■             ),
55 ■             label: 'Alerts',
56 ■           ),
57 ■           const NavigationDestination(
58 ■             icon: Icon(Icons.person_outline_rounded),

```

```

59 ■         selectedIcon: Icon(Icons.person_rounded),
60 ■         label: 'Profile',
61 ■       ),
62 ■     ],
63 ■   ),
64 ■ );
65 ■ }
66 ■ }

```

lib/features/student/screens/student_home_screen.dart

375 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■ import 'package:go_router/go_router.dart';
4 ■
5 ■ import '../../../core/constants/enums.dart';
6 ■ import '../../../core/constants/nmb_constants.dart';
7 ■ import '../../../core/theme/nmb_colors.dart';
8 ■ import '../../../core/theme/nmb_typography.dart';
9 ■ import '../../../core/utils/formatters.dart';
10 ■ import '../../../core/widgets/nmb_card.dart';
11 ■ import '../../../core/widgets/responsive_scaffold_body.dart';
12 ■ import '../../../core/widgets/state_views.dart';
13 ■ import '../../../models/app_notification.dart';
14 ■ import '../../../models/app_user.dart';
15 ■ import '../../../models/bus.dart';
16 ■ import '../../../models/bus_stop.dart';
17 ■ import '../../../models/live_location.dart';
18 ■ import '../../../models/trip.dart';
19 ■ import '../../../providers/app_providers.dart';
20 ■ import '../../../providers/data_providers.dart';
21 ■ import '../../../services/eta_service.dart';
22 ■ import '../../../widgets/tracking_status_card.dart';
23 ■
24 ■ /// Student Home: greeting, assigned bus card, live status, big
25 ■ /// "Track My Bus" button, assigned stop, quick actions, latest alert.
26 ■ class StudentHomeScreen extends ConsumerWidget {
27 ■   const StudentHomeScreen({super.key});
28 ■
29 ■   @override
30 ■   Widget build(BuildContext context, WidgetRef ref) {
31 ■     final AsyncValue<AppUser?> profileAsync = ref.watch(myProfileProvider);
32 ■
33 ■     return Scaffold(
34 ■       appBar: AppBar(
35 ■         title: const Text(NmbConstants.appName),
36 ■         actions: <Widget>[
37 ■           IconButton(
38 ■             tooltip: 'Settings',
39 ■             icon: const Icon(Icons.settings_outlined),
40 ■             onPressed: () => context.go('/student/profile/settings'),
41 ■           ),
42 ■         ],
43 ■       ),
44 ■       body: profileAsync.when(
45 ■         loading: () => const LoadingView(message: 'Loading your dashboard...'),
46 ■         error: (Object e, _) => ErrorView(
47 ■           message: 'We couldn\'t load your dashboard. '
48 ■             'Please check your connection.',
49 ■           onRetry: () => ref.invalidate(myProfileProvider),
50 ■         ),
51 ■         data: (AppUser? me) {
52 ■           if (me == null) {
53 ■             return const ErrorView(
54 ■               title: 'Account not ready',
55 ■               message: 'Your account is not fully set up yet. '
56 ■                 'Please contact the school office.',
57 ■             );
58 ■           }

```

```

59 ■         return _HomeContent(me: me);
60 ■     },
61 ■     ),
62 ■ );
63 ■ }
64 ■ }
65 ■
66 ■ class _HomeContent extends ConsumerWidget {
67 ■     const _HomeContent({required this.me});
68 ■
69 ■     final AppUser me;
70 ■
71 ■     @override
72 ■     Widget build(BuildContext context, WidgetRef ref) {
73 ■         final Bus? bus = ref.watch(myBusProvider).valueOrNull;
74 ■         final Trip? trip = ref.watch(myBusActiveTripProvider).valueOrNull;
75 ■         final LiveLocation? live =
76 ■             ref.watch(myBusLiveLocationProvider).valueOrNull;
77 ■         final LocationFreshness freshness = ref.watch(myBusFreshnessProvider);
78 ■         final BusStop? myStop = ref.watch(myAssignedStopProvider);
79 ■         final List<BusStop> stops =
80 ■             ref.watch(myStopsProvider).valueOrNull ?? const <BusStop>[];
81 ■         final List<AppNotification> inbox =
82 ■             ref.watch(inboxProvider).valueOrNull ?? const <AppNotification>[];
83 ■
84 ■         // ETA (only when data is reliable).
85 ■         String? etaText;
86 ■         if (myStop != null && freshness == LocationFreshness.live) {
87 ■             final EtaService etaService = ref.watch(etaServiceProvider);
88 ■             if (live != null) etaService.recordSpeed(live.speedKmh);
89 ■             final EtaResult eta = etaService.etaToStop(
90 ■                 live: live,
91 ■                 targetStop: myStop,
92 ■                 orderedStops: stops,
93 ■             );
94 ■             if (eta is EtaAvailable) etaText = Formatters.eta(eta.duration);
95 ■         }
96 ■
97 ■         return ResponsiveBody(
98 ■             child: Column(
99 ■                 crossAxisAlignment: CrossAxisAlignment.stretch,
100 ■                 children: <Widget>[
101 ■                     // Greeting
102 ■                     Text(
103 ■                         Formatters.greetingForNow(firstName: me.firstName),
104 ■                         style: NmbTypography.displayTitle,
105 ■                     ),
106 ■                     const SizedBox(height: 4),
107 ■                     const Text(
108 ■                         'Here\'s your bus for today',
109 ■                         style: NmbTypography.bodySecondary,
110 ■                     ),
111 ■                     const SizedBox(height: 20),
112 ■
113 ■                     // Assigned bus card
114 ■                     if (me.busId == null)
115 ■                         const NmbCard(
116 ■                             color: NmbColors.warningSoft,

```

```

117 ■         child: Row(
118 ■           children: <Widget>[
119 ■             Icon(Icons.no_transfer_rounded, color: NmbColors.warning),
120 ■             SizedBox(width: 12),
121 ■             Expanded(
122 ■               child: Text(
123 ■                 'No bus has been assigned to your account yet. '
124 ■                 'Please contact the school office.',
125 ■                 style: NmbTypography.bodySecondary,
126 ■               ),
127 ■             ),
128 ■           ],
129 ■         ),
130 ■       )
131 ■     else ...<Widget>[
132 ■       _AssignedBusCard(bus: bus, trip: trip),
133 ■       const SizedBox(height: NmbConstants.cardGap),
134 ■       TrackingStatusCard(
135 ■         freshness: freshness,
136 ■         live: live,
137 ■         trip: trip,
138 ■         etaText: etaText,
139 ■       ),
140 ■       const SizedBox(height: NmbConstants.cardGap),
141 ■
142 ■       // Big Track My Bus button
143 ■       FilledButton.icon(
144 ■         style: FilledButton.styleFrom(
145 ■           minimumSize: const Size.fromHeight(60),
146 ■           backgroundColor: NmbColors.primary,
147 ■         ),
148 ■         onPressed: () => context.go('/student/map'),
149 ■         icon: const Icon(Icons.my_location_rounded),
150 ■         label: const Text('Track My Bus'),
151 ■       ),
152 ■       const SizedBox(height: NmbConstants.cardGap),
153 ■
154 ■       // My stop
155 ■       if (myStop != null)
156 ■         NmbCard(
157 ■           onTap: () => context.go('/student/home/stops'),
158 ■           child: Row(
159 ■             children: <Widget>[
160 ■               Container(
161 ■                 width: 44,
162 ■                 height: 44,
163 ■                 decoration: BoxDecoration(
164 ■                   color: NmbColors.accentSoft,
165 ■                   borderRadius: BorderRadius.circular(13),
166 ■                 ),
167 ■                 child: const Icon(
168 ■                   Icons.pin_drop_rounded,
169 ■                   color: NmbColors.accentDark,
170 ■                 ),
171 ■               ),
172 ■               const SizedBox(width: 12),
173 ■               Expanded(
174 ■                 child: Column(

```

```

175 ■         crossAxisAlignment: CrossAxisAlignment.start,
176 ■         children: <Widget>[
177 ■             const Text('My stop', style: NmbTypography.caption),
178 ■             const SizedBox(height: 2),
179 ■             Text(myStop.name, style: NmbTypography.cardTitle),
180 ■             if (myStop.scheduledTime != null)
181 ■                 Text(
182 ■                     'Scheduled: ${myStop.scheduledTime}',
183 ■                     style: NmbTypography.bodySecondary,
184 ■                 ),
185 ■         ],
186 ■     ),
187 ■ ),
188 ■     const Icon(Icons.chevron_right_rounded,
189 ■         color: NmbColors.textTertiary, ),
190 ■ ],
191 ■ ),
192 ■ ),
193 ■ const SizedBox(height: NmbConstants.cardGap),
194 ■
195 ■ // Quick actions
196 ■ Row(
197 ■     children: <Widget>[
198 ■         Expanded(
199 ■             child: _QuickAction(
200 ■                 icon: Icons.route_rounded,
201 ■                 label: 'Bus & Route',
202 ■                 onTap: () => context.go('/student/home/bus'),
203 ■             ),
204 ■         ),
205 ■         const SizedBox(width: 12),
206 ■         Expanded(
207 ■             child: _QuickAction(
208 ■                 icon: Icons.pin_drop_outlined,
209 ■                 label: 'Bus Stops',
210 ■                 onTap: () => context.go('/student/home/stops'),
211 ■             ),
212 ■         ),
213 ■         const SizedBox(width: 12),
214 ■         Expanded(
215 ■             child: _QuickAction(
216 ■                 icon: Icons.notifications_outlined,
217 ■                 label: 'Alerts',
218 ■                 onTap: () => context.go('/student/alerts'),
219 ■             ),
220 ■         ),
221 ■     ],
222 ■ ),
223 ■ ],
224 ■
225 ■ // Latest important alert
226 ■ if (inbox.isNotEmpty) ...<Widget>[
227 ■     const SizedBox(height: 22),
228 ■     const Text('Latest alert', style: NmbTypography.sectionTitle),
229 ■     const SizedBox(height: 10),
230 ■     NmbCard(
231 ■         onTap: () => context.go('/student/alerts'),
232 ■         borderColor:

```

```

233 ■         inbox.first.read ? null : NmbColors.primary.withOpacity(.3),
234 ■     child: Row(
235 ■       crossAxisAlignment: CrossAxisAlignment.start,
236 ■       children: <Widget>[
237 ■         const Icon(Icons.campaign_rounded,
238 ■           color: NmbColors.primary,),
239 ■         const SizedBox(width: 12),
240 ■         Expanded(
241 ■           child: Column(
242 ■             crossAxisAlignment: CrossAxisAlignment.start,
243 ■             children: <Widget>[
244 ■               Text(inbox.first.title,
245 ■                 style: NmbTypography.cardTitle,),
246 ■               const SizedBox(height: 2),
247 ■               Text(
248 ■                 inbox.first.body,
249 ■                 style: NmbTypography.bodySecondary,
250 ■                 maxLines: 2,
251 ■                 overflow: TextOverflow.ellipsis,
252 ■               ),
253 ■               const SizedBox(height: 4),
254 ■               Text(
255 ■                 Formatters.relativeTime(inbox.first.receivedAt),
256 ■                 style: NmbTypography.caption,
257 ■               ),
258 ■             ],
259 ■           ),
260 ■         ),
261 ■       ],
262 ■     ),
263 ■   ),
264 ■ ],
265 ■   const SizedBox(height: 24),
266 ■ ],
267 ■ ),
268 ■ );
269 ■ }
270 ■ }
271 ■
272 ■ class _AssignedBusCard extends StatelessWidget {
273 ■   const _AssignedBusCard({required this.bus, required this.trip});
274 ■
275 ■   final Bus? bus;
276 ■   final Trip? trip;
277 ■
278 ■   @override
279 ■   Widget build(BuildContext context) {
280 ■     final bool onTrip = trip != null;
281 ■     return NmbCard(
282 ■       color: NmbColors.primary,
283 ■       child: Row(
284 ■         children: <Widget>[
285 ■           Container(
286 ■             width: 56,
287 ■             height: 56,
288 ■             decoration: BoxDecoration(
289 ■               color: NmbColors.accent,
290 ■               borderRadius: BorderRadius.circular(16),

```

```

291 ■      ),
292 ■      child: const Icon(
293 ■        Icons.directions_bus_rounded,
294 ■        color: Colors.white,
295 ■        size: 32,
296 ■      ),
297 ■    ),
298 ■    const SizedBox(width: 14),
299 ■    Expanded(
300 ■      child: Column(
301 ■        crossAxisAlignment: CrossAxisAlignment.start,
302 ■        children: <Widget>[
303 ■          Text(
304 ■            'My bus',
305 ■            style:
306 ■              NmbTypography.caption.copyWith(color: Colors.white70),
307 ■          ),
308 ■          const SizedBox(height: 2),
309 ■          Text(
310 ■            bus?.busNumber ?? 'Loading...',
311 ■            style: NmbTypography.screenTitle
312 ■              .copyWith(color: Colors.white),
313 ■          ),
314 ■          if (bus != null)
315 ■            Text(
316 ■              bus!.plateNumber,
317 ■              style: NmbTypography.bodySecondary
318 ■                .copyWith(color: Colors.white70),
319 ■            ),
320 ■        ],
321 ■      ),
322 ■    ),
323 ■    Container(
324 ■      padding:
325 ■        const EdgeInsets.symmetric(horizontal: 10, vertical: 6),
326 ■      decoration: BoxDecoration(
327 ■        color: onTrip ? NmbColors.success : Colors.white24,
328 ■        borderRadius: BorderRadius.circular(999),
329 ■      ),
330 ■      child: Text(
331 ■        onTrip ? 'ON TRIP' : 'IDLE',
332 ■        style: NmbTypography.caption.copyWith(
333 ■          color: Colors.white,
334 ■          fontWeight: FontWeight.w800,
335 ■          letterSpacing: 0.6,
336 ■        ),
337 ■      ),
338 ■    ),
339 ■  ],
340 ■ ),
341 ■ );
342 ■ }
343 ■ }
344 ■
345 ■ class _QuickAction extends StatelessWidget {
346 ■   const _QuickAction({
347 ■     required this.icon,
348 ■     required this.label,

```

```
349 ■     required this.onTap,
350 ■   });
351 ■
352 ■   final IconData icon;
353 ■   final String label;
354 ■   final VoidCallback onTap;
355 ■
356 ■   @override
357 ■   Widget build(BuildContext context) {
358 ■     return NmbCard(
359 ■       onTap: onTap,
360 ■       padding: const EdgeInsets.symmetric(vertical: 16),
361 ■       child: Column(
362 ■         children: <Widget>[
363 ■           Icon(icon, color: NmbColors.primary, size: 26),
364 ■           const SizedBox(height: 8),
365 ■           Text(
366 ■             label,
367 ■             style: NmbTypography.caption
368 ■               .copyWith(color: NmbColors.textSecondary),
369 ■             textAlign: TextAlign.center,
370 ■           ),
371 ■         ],
372 ■       ),
373 ■     );
374 ■   }
375 ■ }
```

lib/features/student/widgets/tracking_status_card.dart

85 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■
3 ■ import '../core/constants/enums.dart';
4 ■ import '../core/theme/nmb_colors.dart';
5 ■ import '../core/theme/nmb_typography.dart';
6 ■ import '../core/utils/formatters.dart';
7 ■ import '../core/widgets/nmb_card.dart';
8 ■ import '../core/widgets/status_pill.dart';
9 ■ import '../models/live_location.dart';
10 ■ import '../models/trip.dart';
11 ■
12 ■ /// Honest tracking status: LIVE / last-updated / unavailable.
13 ■ /// Never shows an old fix as live.
14 ■ class TrackingStatusCard extends StatelessWidget {
15 ■   const TrackingStatusCard({
16 ■     super.key,
17 ■     required this.freshness,
18 ■     required this.live,
19 ■     required this.trip,
20 ■     this.etaText,
21 ■   });
22 ■
23 ■   final LocationFreshness freshness;
24 ■   final LiveLocation? live;
25 ■   final Trip? trip;
26 ■   final String? etaText;
27 ■
28 ■   @override
29 ■   Widget build(BuildContext context) {
30 ■     final (String title, String subtitle, IconData icon, Color color) info =
31 ■       switch (freshness) {
32 ■         LocationFreshness.live => (
33 ■           'Bus is being tracked live',
34 ■           etaText ?? 'Follow it on the Live Map',
35 ■           Icons.gps_fixed_rounded,
36 ■           NmbColors.live,
37 ■         ),
38 ■         LocationFreshness.stale => (
39 ■           'Location temporarily unavailable',
40 ■           live != null
41 ■             ? 'Last updated ${Formatters.relativeTime(live!.updatedAt)}'
42 ■             : 'Waiting for the bus signal...',
43 ■           Icons.gps_not_fixed_rounded,
44 ■           NmbColors.stale,
45 ■         ),
46 ■         LocationFreshness.unavailable => (
47 ■           trip == null ? 'No trip in progress' : 'Location unavailable',
48 ■           trip == null
49 ■             ? 'Tracking starts when the driver begins the trip.'
50 ■             : 'We\'ll show the bus as soon as the signal returns.',
51 ■           Icons.location_off_rounded,
52 ■           NmbColors.offline,
53 ■         ),
54 ■       };
55 ■
56 ■     return NmbCard(
57 ■       child: Row(
58 ■         children: <Widget>[

```

```
59 ■      Container(  
60 ■        width: 48,  
61 ■        height: 48,  
62 ■        decoration: BoxDecoration(  
63 ■          color: info.$4.withOpacity(0.12),  
64 ■          borderRadius: BorderRadius.circular(14),  
65 ■        ),  
66 ■        child: Icon(info.$3, color: info.$4),  
67 ■      ),  
68 ■      const SizedBox(width: 14),  
69 ■      Expanded(  
70 ■        child: Column(  
71 ■          crossAxisAlignment: CrossAxisAlignment.start,  
72 ■          children: <Widget>[  
73 ■            Text(info.$1, style: NmbTypography.cardTitle),  
74 ■            const SizedBox(height: 3),  
75 ■            Text(info.$2, style: NmbTypography.bodySecondary),  
76 ■          ],  
77 ■        ),  
78 ■      ),  
79 ■      const SizedBox(width: 8),  
80 ■      StatusPill.freshness(freshness),  
81 ■    ],  
82 ■  ),  
83 ■ );  
84 ■ }  
85 ■ }
```

lib/features/student/screens/student_map_screen.dart

266 lines

```

1 ■ import 'dart:async';
2 ■
3 ■ import 'package:flutter/material.dart';
4 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
5 ■ import 'package:google_maps_flutter/google_maps_flutter.dart';
6 ■
7 ■ import '../core/constants/enums.dart';
8 ■ import '../core/theme/nmb_colors.dart';
9 ■ import '../core/theme/nmb_typography.dart';
10 ■ import '../core/utils/formatters.dart';
11 ■ import '../core/widgets/state_views.dart';
12 ■ import '../core/widgets/status_pill.dart';
13 ■ import '../models/bus_route.dart';
14 ■ import '../models/bus_stop.dart';
15 ■ import '../models/live_location.dart';
16 ■ import '../models/trip.dart';
17 ■ import '../providers/data_providers.dart';
18 ■ import '../shared/bus_marker_icon.dart';
19 ■
20 ■ /// Full live map: bus marker (school-bus icon), route line, all stops,
21 ■ /// the student's own stop highlighted, school marker, Center-on-Bus FAB.
22 ■ ///
23 ■ /// The map only ever receives THIS student's bus location – the backend
24 ■ /// rules make any other subscription impossible.
25 ■ class StudentMapScreen extends ConsumerStatefulWidget {
26 ■   const StudentMapScreen({super.key});
27 ■
28 ■   @override
29 ■   ConsumerState<StudentMapScreen> createState() => _StudentMapScreenState();
30 ■ }
31 ■
32 ■ class _StudentMapScreenState extends ConsumerState<StudentMapScreen> {
33 ■   GoogleMapController? _controller;
34 ■   BitmapDescriptor? _busIcon;
35 ■   bool _followBus = true;
36 ■
37 ■   @override
38 ■   void initState() {
39 ■     super.initState();
40 ■     BusMarkerIcon.load().then((BitmapDescriptor icon) {
41 ■       if (mounted) setState(() => _busIcon = icon);
42 ■     });
43 ■   }
44 ■
45 ■   Future<void> _centerOnBus(LiveLocation live) async {
46 ■     await _controller?.animateCamera(
47 ■       CameraUpdate.newLatLng(LatLng(live.position.lat, live.position.lng)),
48 ■     );
49 ■   }
50 ■
51 ■   @override
52 ■   Widget build(BuildContext context) {
53 ■     final LiveLocation? live =
54 ■       ref.watch(myBusLiveLocationProvider).valueOrNull;
55 ■     final LocationFreshness freshness = ref.watch(myBusFreshnessProvider);
56 ■     final Trip? trip = ref.watch(myBusActiveTripProvider).valueOrNull;
57 ■     final BusRoute? route = ref.watch(myRouteProvider).valueOrNull;
58 ■     final List<BusStop> stops =

```

```

59 ■         ref.watch(myStopsProvider).valueOrNull ?? const <BusStop>[];
60 ■     final BusStop? myStop = ref.watch(myAssignedStopProvider);
61 ■
62 ■     // Auto-follow the bus while fresh.
63 ■     ref.listen(myBusLiveLocationProvider,
64 ■         (Object? _, AsyncValue<LiveLocation?> next) {
65 ■         final LiveLocation? l = next.valueOrNull;
66 ■         if (l != null && _followBus && freshness != LocationFreshness.unavailable) {
67 ■             _centerOnBus(l);
68 ■         }
69 ■     });
70 ■
71 ■     if (route == null && stops.isEmpty) {
72 ■         return Scaffold(
73 ■             appBar: AppBar(title: const Text('Live Map')),
74 ■             body: const EmptyState(
75 ■                 icon: Icons.map_outlined,
76 ■                 title: 'Route not available yet',
77 ■                 message:
78 ■                     'Your bus route hasn\'t been published. Please check back '
79 ■                     'later or contact the school office.',
80 ■             ),
81 ■         );
82 ■     }
83 ■
84 ■     final Set<Marker> markers = <Marker>{};
85 ■     final Set<Polyline> polylines = <Polyline>{};
86 ■
87 ■     // School marker.
88 ■     if (route != null) {
89 ■         markers.add(
90 ■             Marker(
91 ■                 markerId: const MarkerId('school'),
92 ■                 position: LatLng(
93 ■                     route.schoolLocation.lat,
94 ■                     route.schoolLocation.lng,
95 ■                 ),
96 ■                 icon: BitmapDescriptor.defaultMarkerWithHue(
97 ■                     BitmapDescriptor.hueAzure,
98 ■                 ),
99 ■                 infoWindow: const InfoWindow(title: 'SRBS International School'),
100 ■             ),
101 ■         );
102 ■     }
103 ■
104 ■     // Stop markers; student's own stop highlighted.
105 ■     for (final BusStop stop in stops) {
106 ■         final bool isMine = stop.id == myStop?.id;
107 ■         markers.add(
108 ■             Marker(
109 ■                 markerId: MarkerId('stop_${stop.id}'),
110 ■                 position: LatLng(stop.location.lat, stop.location.lng),
111 ■                 icon: BitmapDescriptor.defaultMarkerWithHue(
112 ■                     isMine
113 ■                         ? BitmapDescriptor.hueOrange
114 ■                         : BitmapDescriptor.hueViolet,
115 ■                 ),
116 ■                 infoWindow: InfoWindow(

```

```

117 ■         title: isMine ? '■ ${stop.name} (My stop)' : stop.name,
118 ■         snippet: stop.scheduledTime != null
119 ■             ? 'Scheduled ${stop.scheduledTime}'
120 ■             : null,
121 ■     ),
122 ■ ),
123 ■ );
124 ■ }
125 ■
126 ■ // Route polyline: stop-to-stop fallback (encoded polyline drawn when
127 ■ // provided by admin).
128 ■ if (stops.length >= 2) {
129 ■     polylines.add(
130 ■         Polyline(
131 ■             polylineId: const PolylineId('route'),
132 ■             width: 4,
133 ■             color: NmbColors.primary.withOpacity(0.75),
134 ■             points: <LatLng>[
135 ■                 for (final BusStop s in stops)
136 ■                     LatLng(s.location.lat, s.location.lng),
137 ■                 if (route != null)
138 ■                     LatLng(route.schoolLocation.lat, route.schoolLocation.lng),
139 ■             ],
140 ■         ),
141 ■     );
142 ■ }
143 ■
144 ■ // Bus marker – only when the fix is live or stale; NEVER when
145 ■ // unavailable (an old point must not pretend to be the bus).
146 ■ if (live != null && freshness != LocationFreshness.unavailable) {
147 ■     markers.add(
148 ■         Marker(
149 ■             markerId: const MarkerId('bus'),
150 ■             position: LatLng(live.position.lat, live.position.lng),
151 ■             icon: _busIcon ??
152 ■                 BitmapDescriptor.defaultMarkerWithHue(
153 ■                     BitmapDescriptor.hueYellow,
154 ■                 ),
155 ■             rotation: live.heading,
156 ■             anchor: const Offset(0.5, 0.5),
157 ■             flat: true,
158 ■             alpha: freshness == LocationFreshness.live ? 1.0 : 0.5,
159 ■             infoWindow: InfoWindow(
160 ■                 title: 'School bus',
161 ■                 snippet: freshness == LocationFreshness.live
162 ■                     ? 'Live'
163 ■                     : 'Last updated ${Formatters.relativeTime(live.updatedAt)}',
164 ■             ),
165 ■         ),
166 ■     );
167 ■ }
168 ■
169 ■ final LatLng initialTarget = live != null
170 ■     ? LatLng(live.position.lat, live.position.lng)
171 ■     : stops.isNotEmpty
172 ■         ? LatLng(stops.first.location.lat, stops.first.location.lng)
173 ■         : LatLng(
174 ■             route!.schoolLocation.lat,

```

```

175 ■         route.schoolLocation.lng,
176 ■     );
177 ■
178 ■     return Scaffold(
179 ■       appBar: AppBar(
180 ■         title: const Text('Live Map'),
181 ■         actions: <Widget>[
182 ■           Padding(
183 ■             padding: const EdgeInsets.only(right: 16),
184 ■             child: Center(child: StatusPill.freshness(freshness)),
185 ■           ),
186 ■         ],
187 ■       ),
188 ■       body: Stack(
189 ■         children: <Widget>[
190 ■           GoogleMap(
191 ■             initialCameraPosition:
192 ■               CameraPosition(target: initialTarget, zoom: 14),
193 ■             markers: markers,
194 ■             polylines: polylines,
195 ■             myLocationButtonEnabled: false,
196 ■             zoomControlsEnabled: false,
197 ■             compassEnabled: true,
198 ■             onMapCreated: (GoogleMapController c) => _controller = c,
199 ■             // User dragging disables auto-follow until re-centered.
200 ■             onCameraMoveStarted: () => _followBus = false,
201 ■           ),
202 ■
203 ■           // Honest status banner overlaying the map.
204 ■           if (freshness != LocationFreshness.live)
205 ■             Positioned(
206 ■               top: 12,
207 ■               left: 16,
208 ■               right: 16,
209 ■               child: Material(
210 ■                 borderRadius: BorderRadius.circular(14),
211 ■                 color: freshness == LocationFreshness.stale
212 ■                   ? NmbColors.warningSoft
213 ■                   : NmbColors.surface,
214 ■                 elevation: 3,
215 ■                 child: Padding(
216 ■                   padding: const EdgeInsets.all(12),
217 ■                   child: Row(
218 ■                     children: <Widget>[
219 ■                       Icon(
220 ■                         freshness == LocationFreshness.stale
221 ■                           ? Icons.history_rounded
222 ■                           : Icons.location_off_rounded,
223 ■                         color: freshness == LocationFreshness.stale
224 ■                           ? NmbColors.warning
225 ■                           : NmbColors.textTertiary,
226 ■                         size: 20,
227 ■                       ),
228 ■                       const SizedBox(width: 10),
229 ■                       Expanded(
230 ■                         child: Text(
231 ■                           freshness == LocationFreshness.stale &&
232 ■                             live != null

```

```

233 ■          ? 'Location temporarily unavailable – last '
234 ■            'updated ${Formatters.relativeTime(live.updatedAt))}'
235 ■          : trip == null
236 ■            ? 'No trip in progress. Tracking starts '
237 ■              'when the driver begins the trip.'
238 ■            : 'Location temporarily unavailable.',
239 ■          style: NmbTypography.caption.copyWith(
240 ■            color: NmbColors.textSecondary,
241 ■          ),
242 ■        ),
243 ■      ),
244 ■    ],
245 ■  ),
246 ■ ),
247 ■ ),
248 ■ ),
249 ■ ],
250 ■ ),
251 ■ floatingActionButton: live != null &&
252 ■   freshness != LocationFreshness.unavailable
253 ■   ? FloatingActionButton.extended(
254 ■     onPressed: () {
255 ■       _followBus = true;
256 ■       _centerOnBus(live);
257 ■     },
258 ■     backgroundColor: NmbColors.primary,
259 ■     foregroundColor: Colors.white,
260 ■     icon: const Icon(Icons.center_focus_strong_rounded),
261 ■     label: const Text('Center on Bus'),
262 ■   )
263 ■   : null,
264 ■ );
265 ■ }
266 ■ }

```

lib/features/shared/bus_marker_icon.dart

85 lines

```

1 ■ import 'dart:async';
2 ■ import 'dart:typed_data';
3 ■ import 'dart:ui' as ui;
4 ■
5 ■ import 'package:flutter/material.dart';
6 ■ import 'package:google_maps_flutter/google_maps_flutter.dart';
7 ■
8 ■ import '../core/theme/nmb_colors.dart';
9 ■
10 ■ /// Draws a school-bus map marker at runtime (no asset pipeline needed):
11 ■ /// amber rounded bus body with windows and wheels on a white circular badge.
12 ■ abstract final class BusMarkerIcon {
13 ■   static BitmapDescriptor? _cached;
14 ■
15 ■   static Future<BitmapDescriptor> load({double size = 120}) async {
16 ■     if (_cached != null) return _cached!;
17 ■
18 ■     final ui.PictureRecorder recorder = ui.PictureRecorder();
19 ■     final Canvas canvas = Canvas(recorder);
20 ■     final double s = size;
21 ■     final Offset center = Offset(s / 2, s / 2);
22 ■
23 ■     // White badge with subtle shadow ring.
24 ■     canvas.drawCircle(
25 ■       center,
26 ■       s * 0.48,
27 ■       Paint()..color = Colors.black.withOpacity(0.15),
28 ■     );
29 ■     canvas.drawCircle(center, s * 0.45, Paint()..color = Colors.white);
30 ■
31 ■     // Bus body.
32 ■     final RRect body = RRect.fromRectAndRadius(
33 ■       Rect.fromCenter(center: center, width: s * 0.56, height: s * 0.40),
34 ■       Radius.circular(s * 0.09),
35 ■     );
36 ■     canvas.drawRRect(body, Paint()..color = NmbColors.accent);
37 ■
38 ■     // Windows.
39 ■     final Paint windowPaint = Paint()..color = const Color(0xFFE3F2FD);
40 ■     final double winW = s * 0.115;
41 ■     final double winH = s * 0.13;
42 ■     final double winY = center.dy - s * 0.09;
43 ■     for (int i = 0; i < 3; i++) {
44 ■       final double winX = center.dx - s * 0.21 + i * (winW + s * 0.04);
45 ■       canvas.drawRRect(
46 ■         RRect.fromRectAndRadius(
47 ■           Rect.fromLTWH(winX, winY, winW, winH),
48 ■           Radius.circular(s * 0.025),
49 ■         ),
50 ■         windowPaint,
51 ■       );
52 ■     }
53 ■
54 ■     // Stripe.
55 ■     canvas.drawRect(
56 ■       Rect.fromCenter(
57 ■         center: Offset(center.dx, center.dy + s * 0.075),
58 ■         width: s * 0.56,

```

```
59 ■      height: s * 0.025,  
60 ■    ),  
61 ■    Paint()..color = Colors.white,  
62 ■  );  
63 ■  
64 ■  // Wheels.  
65 ■  final Paint wheelPaint = Paint()..color = const Color(0xFF37474F);  
66 ■  canvas.drawCircle(  
67 ■    Offset(center.dx - s * 0.16, center.dy + s * 0.20),  
68 ■    s * 0.055,  
69 ■    wheelPaint,  
70 ■  );  
71 ■  canvas.drawCircle(  
72 ■    Offset(center.dx + s * 0.16, center.dy + s * 0.20),  
73 ■    s * 0.055,  
74 ■    wheelPaint,  
75 ■  );  
76 ■  
77 ■  final ui.Image image = await recorder  
78 ■    .endRecording()  
79 ■    .toImage(s.toInt(), s.toInt());  
80 ■  final ByteData? bytes =  
81 ■    await image.toByteData(format: ui.ImageByteFormat.png);  
82 ■  _cached = BitmapDescriptor.bytes(bytes!.buffer.asUint8List());  
83 ■  return _cached!  
84 ■ }  
85 ■ }
```

lib/features/student/screens/student_bus_details_screen.dart

159 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/theme/nmb_colors.dart';
5 ■ import '../core/theme/nmb_typography.dart';
6 ■ import '../core/widgets/nmb_card.dart';
7 ■ import '../core/widgets/responsive_scaffold_body.dart';
8 ■ import '../core/widgets/state_views.dart';
9 ■ import '../models/bus.dart';
10 ■ import '../models/bus_route.dart';
11 ■ import '../models/bus_stop.dart';
12 ■ import '../models/trip.dart';
13 ■ import '../providers/data_providers.dart';
14 ■
15 ■ /// Bus & route details for the student's assigned bus. Read-only:
16 ■ /// students cannot change anything here by design.
17 ■ class StudentBusDetailsScreen extends ConsumerWidget {
18 ■   const StudentBusDetailsScreen({super.key});
19 ■
20 ■   @override
21 ■   Widget build(BuildContext context, WidgetRef ref) {
22 ■     final AsyncValue<Bus?> busAsync = ref.watch(myBusProvider);
23 ■     final BusRoute? route = ref.watch(myRouteProvider).valueOrNull;
24 ■     final List<BusStop> stops =
25 ■       ref.watch(myStopsProvider).valueOrNull ?? const <BusStop>[];
26 ■     final Trip? trip = ref.watch(myBusActiveTripProvider).valueOrNull;
27 ■
28 ■     return Scaffold(
29 ■       appBar: AppBar(title: const Text('My Bus & Route')),
30 ■       body: busAsync.when(
31 ■         loading: () => const LoadingView(),
32 ■         error: (Object e, _) => ErrorView(
33 ■           message: 'Couldn\'t load bus details.',
34 ■           onRetry: () => ref.invalidate(myBusProvider),
35 ■         ),
36 ■         data: (Bus? bus) {
37 ■           if (bus == null) {
38 ■             return const EmptyState(
39 ■               icon: Icons.no_transfer_rounded,
40 ■               title: 'No bus assigned',
41 ■               message:
42 ■                 'A bus hasn\'t been assigned to your account yet. Please '
43 ■                 'contact the school office.',
44 ■             );
45 ■           }
46 ■           return ResponsiveBody(
47 ■             child: Column(
48 ■               crossAxisAlignment: CrossAxisAlignment.stretch,
49 ■               children: <Widget>[
50 ■                 NmbCard(
51 ■                   child: Column(
52 ■                     children: <Widget>[
53 ■                       _InfoRow(
54 ■                         icon: Icons.directions_bus_rounded,
55 ■                         label: 'Bus number',
56 ■                         value: bus.busNumber,
57 ■                       ),
58 ■                       const Divider(height: 22),

```

```

59 ■      _InfoRow(
60 ■          icon: Icons.pin_outlined,
61 ■          label: 'Plate number',
62 ■          value: bus.plateNumber,
63 ■      ),
64 ■      const Divider(height: 22),
65 ■      _InfoRow(
66 ■          icon: Icons.event_seat_outlined,
67 ■          label: 'Capacity',
68 ■          value: '${bus.capacity} seats',
69 ■      ),
70 ■      const Divider(height: 22),
71 ■      _InfoRow(
72 ■          icon: Icons.play_circle_outline_rounded,
73 ■          label: 'Current status',
74 ■          value: trip != null
75 ■              ? 'On a ${trip.direction.label.toLowerCase()} trip'
76 ■              : 'No trip in progress',
77 ■          valueColor: trip != null
78 ■              ? NmbColors.success
79 ■              : NmbColors.textSecondary,
80 ■      ),
81 ■    ],
82 ■  ),
83 ■),
84 ■const SizedBox(height: 16),
85 ■if (route != null) ...<Widget>[
86 ■  const Text('Route', style: NmbTypography.sectionTitle),
87 ■  const SizedBox(height: 10),
88 ■  NmbCard(
89 ■    child: Column(
90 ■      children: <Widget>[
91 ■        _InfoRow(
92 ■          icon: Icons.route_rounded,
93 ■          label: 'Route name',
94 ■          value: route.name,
95 ■        ),
96 ■        const Divider(height: 22),
97 ■        _InfoRow(
98 ■          icon: Icons.pin_drop_outlined,
99 ■          label: 'Stops on route',
100 ■          value: '${stops.length}',
101 ■        ),
102 ■        const Divider(height: 22),
103 ■        _InfoRow(
104 ■          icon: Icons.swap_vert_rounded,
105 ■          label: 'Direction',
106 ■          value: route.direction.label,
107 ■        ),
108 ■      ],
109 ■    ),
110 ■  ),
111 ■] else
112 ■  const NmbCard(
113 ■    child: Text(
114 ■      'Route details haven\'t been published yet.',
115 ■      style: NmbTypography.bodySecondary,
116 ■    ),

```

```

117 ■         ),
118 ■         const SizedBox(height: 24),
119 ■       ],
120 ■     ),
121 ■   );
122 ■ },
123 ■ ),
124 ■ );
125 ■ }
126 ■ }
127 ■
128 ■ class _InfoRow extends StatelessWidget {
129 ■   const _InfoRow({
130 ■     required this.icon,
131 ■     required this.label,
132 ■     required this.value,
133 ■     this.valueColor,
134 ■   });
135 ■
136 ■   final IconData icon;
137 ■   final String label;
138 ■   final String value;
139 ■   final Color? valueColor;
140 ■
141 ■   @override
142 ■   Widget build(BuildContext context) {
143 ■     return Row(
144 ■       children: <Widget>[
145 ■         Icon(icon, size: 22, color: NmbColors.primary),
146 ■         const SizedBox(width: 12),
147 ■         Expanded(
148 ■           child: Text(label, style: NmbTypography.bodySecondary),
149 ■         ),
150 ■         Text(
151 ■           value,
152 ■           style: NmbTypography.cardTitle.copyWith(
153 ■             color: valueColor ?? NmbColors.textPrimary,
154 ■           ),
155 ■         ),
156 ■       ],
157 ■     );
158 ■   }
159 ■ }

```

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/theme/nmb_colors.dart';
5 ■ import '../core/theme/nmb_typography.dart';
6 ■ import '../core/widgets/nmb_card.dart';
7 ■ import '../core/widgets/responsive_scaffold_body.dart';
8 ■ import '../core/widgets/state_views.dart';
9 ■ import '../models/bus_stop.dart';
10 ■ import '../providers/data_providers.dart';
11 ■
12 ■ /// Ordered list of the route's stops as a friendly timeline; the student's
13 ■ /// assigned stop is highlighted with a star.
14 ■ class StudentStopsScreen extends ConsumerWidget {
15 ■   const StudentStopsScreen({super.key});
16 ■
17 ■   @override
18 ■   Widget build(BuildContext context, WidgetRef ref) {
19 ■     final AsyncValue<List<BusStop>> stopsAsync = ref.watch(myStopsProvider);
20 ■     final BusStop? myStop = ref.watch(myAssignedStopProvider);
21 ■
22 ■     return Scaffold(
23 ■       appBar: AppBar(title: const Text('Bus Stops')),
24 ■       body: stopsAsync.when(
25 ■         loading: () => const LoadingView(),
26 ■         error: (Object e, _) => ErrorView(
27 ■           message: 'Couldn\'t load the stop list.',
28 ■           onRetry: () => ref.invalidate(myStopsProvider),
29 ■         ),
30 ■         data: (List<BusStop> stops) {
31 ■           if (stops.isEmpty) {
32 ■             return const EmptyState(
33 ■               icon: Icons.pin_drop_outlined,
34 ■               title: 'No stops yet',
35 ■               message: 'The stop list for your route hasn\'t been '
36 ■                 'published yet.',
37 ■             );
38 ■           }
39 ■           return ResponsiveBody(
40 ■             child: Column(
41 ■               crossAxisAlignment: CrossAxisAlignment.stretch,
42 ■               children: <Widget>[
43 ■                 for (int i = 0; i < stops.length; i++) ...<Widget>[
44 ■                   _StopTile(
45 ■                     stop: stops[i],
46 ■                     index: i,
47 ■                     isLast: i == stops.length - 1,
48 ■                     isMine: stops[i].id == myStop?.id,
49 ■                   ),
50 ■                 ],
51 ■                 const SizedBox(height: 16),
52 ■                 const NmbCard(
53 ■                   color: NmbColors.infoSoft,
54 ■                   child: Row(
55 ■                     children: <Widget>[
56 ■                       Icon(Icons.info_outline_rounded,
57 ■                         color: NmbColors.info,),
58 ■                       SizedBox(width: 10),

```

```

59 ■         Expanded(
60 ■             child: Text(
61 ■                 'Your pickup stop is set by the school. To change '
62 ■                 'it, please contact the school office.',
63 ■                 style: NmbTypography.bodySecondary,
64 ■             ),
65 ■         ),
66 ■     ],
67 ■ ),
68 ■ ),
69 ■     const SizedBox(height: 24),
70 ■ ],
71 ■ ),
72 ■ );
73 ■ },
74 ■ ),
75 ■ );
76 ■ }
77 ■ }
78 ■
79 ■ class _StopTile extends StatelessWidget {
80 ■     const _StopTile({
81 ■         required this.stop,
82 ■         required this.index,
83 ■         required this.isLast,
84 ■         required this.isMine,
85 ■     });
86 ■
87 ■     final BusStop stop;
88 ■     final int index;
89 ■     final bool isLast;
90 ■     final bool isMine;
91 ■
92 ■     @override
93 ■     Widget build(BuildContext context) {
94 ■         return IntrinsicHeight(
95 ■             child: Row(
96 ■                 crossAxisAlignment: CrossAxisAlignment.stretch,
97 ■                 children: <Widget>[
98 ■                     // Timeline rail
99 ■                     SizedBox(
100 ■                         width: 36,
101 ■                         child: Column(
102 ■                             children: <Widget>[
103 ■                                 Container(
104 ■                                     width: 26,
105 ■                                     height: 26,
106 ■                                     decoration: BoxDecoration(
107 ■                                         color: isMine ? NmbColors.accent : NmbColors.primarySoft,
108 ■                                         shape: BoxShape.circle,
109 ■                                     ),
110 ■                                     child: Center(
111 ■                                         child: isMine
112 ■                                             ? const Icon(Icons.star_rounded,
113 ■                                                 size: 16, color: Colors.white,)
114 ■                                             : Text(
115 ■                                                 '${index + 1}',
116 ■                                                 style: NmbTypography.caption.copyWith(

```

```

117 ■                                     color: NmbColors.primaryDark,
118 ■                                     fontWeight: FontWeight.w800,
119 ■                                     ),
120 ■                                 ),
121 ■                             ),
122 ■                         ),
123 ■                     if (!isLast)
124 ■                         Expanded(
125 ■                             child: Container(
126 ■                                 width: 2.4,
127 ■                                 color: NmbColors.divider,
128 ■                             ),
129 ■                         ),
130 ■                     ],
131 ■                 ),
132 ■             ),
133 ■             const SizedBox(width: 10),
134 ■             Expanded(
135 ■                 child: Padding(
136 ■                     padding: const EdgeInsets.only(bottom: 12),
137 ■                     child: NmbCard(
138 ■                         color: isMine ? NmbColors.accentSoft : NmbColors.surface,
139 ■                         borderColor: isMine ? NmbColors.accent : null,
140 ■                         padding: const EdgeInsets.all(14),
141 ■                         child: Row(
142 ■                             children: <Widget>[
143 ■                                 Expanded(
144 ■                                     child: Column(
145 ■                                         crossAxisAlignment: CrossAxisAlignment.start,
146 ■                                         children: <Widget>[
147 ■                                             Text(stop.name, style: NmbTypography.cardTitle),
148 ■                                             if (isMine)
149 ■                                                 Text(
150 ■                                                     'My assigned stop',
151 ■                                                     style: NmbTypography.caption.copyWith(
152 ■                                                         color: NmbColors.accentDark,
153 ■                                                         fontWeight: FontWeight.w700,
154 ■                                                     ),
155 ■                                                 ),
156 ■                                         ],
157 ■                                     ),
158 ■                                 ),
159 ■                                 if (stop.scheduledTime != null)
160 ■                                     Container(
161 ■                                         padding: const EdgeInsets.symmetric(
162 ■                                             horizontal: 10,
163 ■                                             vertical: 5,
164 ■                                         ),
165 ■                                         decoration: BoxDecoration(
166 ■                                             color: NmbColors.surface,
167 ■                                             borderRadius: BorderRadius.circular(10),
168 ■                                             border: Border.all(color: NmbColors.divider),
169 ■                                         ),
170 ■                                         child: Text(
171 ■                                             stop.scheduledTime!,
172 ■                                             style: NmbTypography.caption.copyWith(
173 ■                                                 color: NmbColors.textSecondary,
174 ■                                                 fontWeight: FontWeight.w700,
175 ■                                             ),
176 ■                                         ),
177 ■                                     ),
178 ■                                 ],
179 ■                             ),
180 ■                         ),
181 ■                     ),
182 ■                 ),
183 ■             ],
184 ■         ),
185 ■     );
186 ■ }
187 ■ }

```

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/constants/enums.dart';
5 ■ import '../core/theme/nmb_colors.dart';
6 ■ import '../core/theme/nmb_typography.dart';
7 ■ import '../core/utils/formatters.dart';
8 ■ import '../core/widgets/nmb_card.dart';
9 ■ import '../core/widgets/responsive_scaffold_body.dart';
10 ■ import '../core/widgets/state_views.dart';
11 ■ import '../models/app_notification.dart';
12 ■ import '../providers/app_providers.dart';
13 ■ import '../providers/data_providers.dart';
14 ■ import '../services/auth_service.dart';
15 ■
16 ■ /// Notification inbox. Items are fanned out server-side per user, so this
17 ■ /// list can only ever contain notifications addressed to this student.
18 ■ class StudentAlertsScreen extends ConsumerWidget {
19 ■   const StudentAlertsScreen({super.key});
20 ■
21 ■   @override
22 ■   Widget build(BuildContext context, WidgetRef ref) {
23 ■     final AsyncValue<List<AppNotification>> inboxAsync =
24 ■       ref.watch(inboxProvider);
25 ■
26 ■     return Scaffold(
27 ■       appBar: AppBar(title: const Text('Alerts')),
28 ■       body: inboxAsync.when(
29 ■         loading: () => const LoadingView(),
30 ■         error: (Object e, _) => ErrorView(
31 ■           message: 'Couldn\'t load alerts.',
32 ■           onRetry: () => ref.invalidate(inboxProvider),
33 ■         ),
34 ■         data: (List<AppNotification> items) {
35 ■           if (items.isEmpty) {
36 ■             return const EmptyState(
37 ■               icon: Icons.notifications_none_rounded,
38 ■               title: 'No alerts yet',
39 ■               message: 'Bus updates and school announcements will '
40 ■                 'appear here.',
41 ■             );
42 ■           }
43 ■           return ResponsiveBody(
44 ■             child: Column(
45 ■               crossAxisAlignment: CrossAxisAlignment.stretch,
46 ■               children: <Widget>[
47 ■                 for (final AppNotification n in items) ...<Widget>[
48 ■                   _AlertTile(notification: n),
49 ■                   const SizedBox(height: 10),
50 ■                 ],
51 ■                 const SizedBox(height: 16),
52 ■               ],
53 ■             ),
54 ■           );
55 ■         },
56 ■       ),
57 ■     );
58 ■   }

```

```

59 ■ }
60 ■
61 ■ class _AlertTile extends ConsumerWidget {
62 ■   const _AlertTile({required this.notification});
63 ■
64 ■   final AppNotification notification;
65 ■
66 ■   (IconData, Color, Color) get _style => switch (notification.type) {
67 ■     NotificationType.tripStarted => (
68 ■       Icons.play_circle_rounded,
69 ■       NmbColors.success,
70 ■       NmbColors.successSoft
71 ■     ),
72 ■     NotificationType.busApproaching => (
73 ■       Icons.near_me_rounded,
74 ■       NmbColors.primary,
75 ■       NmbColors.primarySoft
76 ■     ),
77 ■     NotificationType.busReachedStop => (
78 ■       Icons.pin_drop_rounded,
79 ■       NmbColors.accentDark,
80 ■       NmbColors.accentSoft
81 ■     ),
82 ■     NotificationType.busReachedSchool => (
83 ■       Icons.school_rounded,
84 ■       NmbColors.info,
85 ■       NmbColors.infoSoft
86 ■     ),
87 ■     NotificationType.busDelayed => (
88 ■       Icons.schedule_rounded,
89 ■       NmbColors.warning,
90 ■       NmbColors.warningSoft
91 ■     ),
92 ■     NotificationType.trackingUnavailable => (
93 ■       Icons.location_off_rounded,
94 ■       NmbColors.textTertiary,
95 ■       NmbColors.divider
96 ■     ),
97 ■     NotificationType.announcement => (
98 ■       Icons.campaign_rounded,
99 ■       NmbColors.primary,
100 ■       NmbColors.primarySoft
101 ■     ),
102 ■   };
103 ■
104 ■   @override
105 ■   Widget build(BuildContext context, WidgetRef ref) {
106 ■     final (IconData icon, Color color, Color bg) = _style;
107 ■     final AuthSession? session = ref.watch(currentSessionProvider);
108 ■
109 ■     return NmbCard(
110 ■       onTap: notification.read || session == null
111 ■         ? null
112 ■         : () => ref
113 ■           .read(firestoreServiceProvider)
114 ■           .markInboxRead(session.uid, notification.id),
115 ■       borderColor:
116 ■         notification.read ? null : NmbColors.primary.withOpacity(0.35),

```

```

117 ■      child: Row(
118 ■        crossAxisAlignment: CrossAxisAlignment.start,
119 ■        children: <Widget>[
120 ■          Container(
121 ■            width: 42,
122 ■            height: 42,
123 ■            decoration: BoxDecoration(
124 ■              color: bg,
125 ■              borderRadius: BorderRadius.circular(12),
126 ■            ),
127 ■            child: Icon(icon, color: color, size: 22),
128 ■          ),
129 ■          const SizedBox(width: 12),
130 ■          Expanded(
131 ■            child: Column(
132 ■              crossAxisAlignment: CrossAxisAlignment.start,
133 ■              children: <Widget>[
134 ■                Row(
135 ■                  children: <Widget>[
136 ■                    Expanded(
137 ■                      child: Text(
138 ■                        notification.title,
139 ■                        style: NmbTypography.cardTitle.copyWith(
140 ■                          fontWeight: notification.read
141 ■                            ? FontWeight.w500
142 ■                            : FontWeight.w700,
143 ■                        ),
144 ■                    ),
145 ■                  ],
146 ■                  if (!notification.read)
147 ■                    Container(
148 ■                      width: 9,
149 ■                      height: 9,
150 ■                      decoration: const BoxDecoration(
151 ■                        color: NmbColors.primary,
152 ■                        shape: BoxShape.circle,
153 ■                      ),
154 ■                    ),
155 ■                ],
156 ■              ),
157 ■              const SizedBox(height: 3),
158 ■              Text(notification.body, style: NmbTypography.bodySecondary),
159 ■              const SizedBox(height: 6),
160 ■              Text(
161 ■                Formatters.relativeTime(notification.receivedAt),
162 ■                style: NmbTypography.caption,
163 ■              ),
164 ■            ],
165 ■          ),
166 ■        ],
167 ■      ),
168 ■    ),
169 ■  );
170 ■ }
171 ■ }

```

lib/features/student/screens/student_profile_screen.dart

231 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■ import 'package:go_router/go_router.dart';
4 ■
5 ■ import '../core/theme/nmb_colors.dart';
6 ■ import '../core/theme/nmb_typography.dart';
7 ■ import '../core/widgets/nmb_card.dart';
8 ■ import '../core/widgets/nmb_dialogs.dart';
9 ■ import '../core/widgets/responsive_scaffold_body.dart';
10 ■ import '../core/widgets/state_views.dart';
11 ■ import '../models/app_user.dart';
12 ■ import '../models/bus.dart';
13 ■ import '../models/bus_stop.dart';
14 ■ import '../providers/app_providers.dart';
15 ■ import '../providers/data_providers.dart';
16 ■ import '../services/auth_service.dart';
17 ■
18 ■ /// Student profile: personal info (read-only), school contact card,
19 ■ /// settings entry, and logout with confirmation.
20 ■ class StudentProfileScreen extends ConsumerWidget {
21 ■   const StudentProfileScreen({super.key});
22 ■
23 ■   Future<void> _logout(BuildContext context, WidgetRef ref) async {
24 ■     final bool confirmed = await showNmbConfirmDialog(
25 ■       context,
26 ■       title: 'Log out?',
27 ■       message: 'You\'ll need your school email and password to sign '
28 ■         'back in.',
29 ■       confirmLabel: 'Log out',
30 ■       destructive: true,
31 ■       icon: Icons.logout_rounded,
32 ■     );
33 ■     if (!confirmed) return;
34 ■     final AuthSession? session = ref.read(currentSessionProvider);
35 ■     if (session != null) {
36 ■       await ref
37 ■         .read(notificationServiceProvider)
38 ■         .unregisterDevice(session.uid);
39 ■     }
40 ■     await ref.read(authServiceProvider).signOut();
41 ■   }
42 ■
43 ■   @override
44 ■   Widget build(BuildContext context, WidgetRef ref) {
45 ■     final AsyncValue<AppUser?> profileAsync = ref.watch(myProfileProvider);
46 ■     final Bus? bus = ref.watch(myBusProvider).valueOrNull;
47 ■     final BusStop? stop = ref.watch(myAssignedStopProvider);
48 ■     final Map<String, dynamic>? school =
49 ■       ref.watch(schoolConfigProvider).valueOrNull;
50 ■
51 ■     return Scaffold(
52 ■       appBar: AppBar(title: const Text('Profile')),
53 ■       body: profileAsync.when(
54 ■         loading: () => const LoadingView(),
55 ■         error: (Object e, _) => ErrorView(
56 ■           message: 'Couldn\'t load your profile.',
57 ■           onRetry: () => ref.invalidate(myProfileProvider),
58 ■         ),

```

```

59 ■ data: (AppUser? me) {
60 ■   if (me == null) {
61 ■     return const ErrorView(
62 ■       title: 'Profile unavailable',
63 ■       message: 'Please contact the school office.',
64 ■     );
65 ■   }
66 ■   return ResponsiveBody(
67 ■     child: Column(
68 ■       crossAxisAlignment: CrossAxisAlignment.stretch,
69 ■       children: <Widget>[
70 ■         // Identity card
71 ■         NmbCard(
72 ■           child: Row(
73 ■             children: <Widget>[
74 ■               CircleAvatar(
75 ■                 radius: 30,
76 ■                 backgroundColor: NmbColors.primarySoft,
77 ■                 child: Text(
78 ■                   me.firstName.isNotEmpty
79 ■                     ? me.firstName[0].toUpperCase()
80 ■                     : '?',
81 ■                   style: NmbTypography.screenTitle
82 ■                     .copyWith(color: NmbColors.primary),
83 ■                 ),
84 ■               ),
85 ■               const SizedBox(width: 14),
86 ■               Expanded(
87 ■                 child: Column(
88 ■                   crossAxisAlignment: CrossAxisAlignment.start,
89 ■                   children: <Widget>[
90 ■                     Text(me.fullName,
91 ■                       style: NmbTypography.sectionTitle,),
92 ■                     const SizedBox(height: 2),
93 ■                     Text(me.email,
94 ■                       style: NmbTypography.bodySecondary,),
95 ■                     if (me.classSection != null)
96 ■                       Text('Class ${me.classSection}',
97 ■                         style: NmbTypography.caption,),
98 ■                   ],
99 ■                 ),
100 ■               ),
101 ■             ],
102 ■           ),
103 ■         ),
104 ■         const SizedBox(height: 14),
105 ■
106 ■         // Assignment (read-only, admin-controlled)
107 ■         NmbCard(
108 ■           child: Column(
109 ■             children: <Widget>[
110 ■               ListTile(
111 ■                 contentPadding: EdgeInsets.zero,
112 ■                 leading: const Icon(Icons.directions_bus_rounded,
113 ■                   color: NmbColors.primary,),
114 ■                 title: const Text('Assigned bus'),
115 ■                 trailing: Text(
116 ■                   bus?.busNumber ?? 'Not assigned',

```

```

117 ■         style: NmbTypography.cardTitle,
118 ■     ),
119 ■ ),
120 ■ const Divider(),
121 ■ ListTile(
122 ■     contentPadding: EdgeInsets.zero,
123 ■     leading: const Icon(Icons.pin_drop_rounded,
124 ■         color: NmbColors.accentDark,),
125 ■     title: const Text('Assigned stop'),
126 ■     trailing: Text(
127 ■         stop?.name ?? 'Not assigned',
128 ■         style: NmbTypography.cardTitle,
129 ■     ),
130 ■ ),
131 ■ const SizedBox(height: 6),
132 ■ const Text(
133 ■     'Bus and stop assignments are managed by the school '
134 ■     'and can\'t be changed in the app.',
135 ■     style: NmbTypography.caption,
136 ■ ),
137 ■ ],
138 ■ ),
139 ■ ),
140 ■ const SizedBox(height: 14),
141 ■
142 ■ // School contact
143 ■ NmbCard(
144 ■     child: Column(
145 ■         crossAxisAlignment: CrossAxisAlignment.start,
146 ■         children: <Widget>[
147 ■             const Text('School contact',
148 ■                 style: NmbTypography.sectionTitle,),
149 ■             const SizedBox(height: 10),
150 ■             _ContactRow(
151 ■                 icon: Icons.school_rounded,
152 ■                 text: (school?['name'] as String?) ??
153 ■                     'SRBS International School',
154 ■             ),
155 ■             if (school?['phone'] != null)
156 ■                 _ContactRow(
157 ■                     icon: Icons.call_rounded,
158 ■                     text: school!['phone'] as String,
159 ■                 ),
160 ■             if (school?['email'] != null)
161 ■                 _ContactRow(
162 ■                     icon: Icons.mail_rounded,
163 ■                     text: school!['email'] as String,
164 ■                 ),
165 ■             if (school?['address'] != null)
166 ■                 _ContactRow(
167 ■                     icon: Icons.location_on_rounded,
168 ■                     text: school!['address'] as String,
169 ■                 ),
170 ■         ],
171 ■     ),
172 ■ ),
173 ■ const SizedBox(height: 14),
174 ■

```

```

175 ■      NmbCard(
176 ■          onTap: () => context.go('/student/profile/settings'),
177 ■          child: const Row(
178 ■              children: <Widget>[
179 ■                  Icon(Icons.settings_outlined,
180 ■                      color: NmbColors.textSecondary,),
181 ■                  SizedBox(width: 12),
182 ■                  Expanded(
183 ■                      child:
184 ■                          Text('Settings', style: NmbTypography.cardTitle),
185 ■                      ),
186 ■                  Icon(Icons.chevron_right_rounded,
187 ■                      color: NmbColors.textTertiary,),
188 ■              ],
189 ■          ),
190 ■      ),
191 ■      const SizedBox(height: 20),
192 ■      OutlinedButton.icon(
193 ■          style: OutlinedButton.styleFrom(
194 ■              foregroundColor: NmbColors.danger,
195 ■              side: const BorderSide(color: NmbColors.danger),
196 ■          ),
197 ■          onPressed: () => _logout(context, ref),
198 ■          icon: const Icon(Icons.logout_rounded),
199 ■          label: const Text('Log out'),
200 ■      ),
201 ■      const SizedBox(height: 24),
202 ■  ],
203 ■ ),
204 ■ );
205 ■ },
206 ■ ),
207 ■ );
208 ■ };
209 ■ }
210 ■ }
211 ■
212 ■ class _ContactRow extends StatelessWidget {
213 ■     const _ContactRow({required this.icon, required this.text});
214 ■
215 ■     final IconData icon;
216 ■     final String text;
217 ■
218 ■     @override
219 ■     Widget build(BuildContext context) {
220 ■         return Padding(
221 ■             padding: const EdgeInsets.only(bottom: 8),
222 ■             child: Row(
223 ■                 children: <Widget>[
224 ■                     Icon(icon, size: 18, color: NmbColors.textTertiary),
225 ■                     const SizedBox(width: 10),
226 ■                     Expanded(child: Text(text, style: NmbTypography.bodySecondary)),
227 ■                 ],
228 ■             ),
229 ■         );
230 ■     }
231 ■ }

```

lib/features/student/screens/student_settings_screen.dart

184 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■ import 'package:permission_handler/permission_handler.dart';
4 ■
5 ■ import '../core/theme/nmb_colors.dart';
6 ■ import '../core/theme/nmb_typography.dart';
7 ■ import '../core/widgets/nmb_card.dart';
8 ■ import '../core/widgets/nmb_dialogs.dart';
9 ■ import '../core/widgets/responsive_scaffold_body.dart';
10 ■ import '../core/utils/result.dart';
11 ■ import '../providers/app_providers.dart';
12 ■ import '../services/auth_service.dart';
13 ■
14 ■ /// Notification preferences + permission helper + password reset.
15 ■ class StudentSettingsScreen extends ConsumerStatefulWidget {
16 ■   const StudentSettingsScreen({super.key});
17 ■
18 ■   @override
19 ■   ConsumerState<StudentSettingsScreen> createState() =>
20 ■     _StudentSettingsScreenState();
21 ■ }
22 ■
23 ■ class _StudentSettingsScreenState
24 ■   extends ConsumerState<StudentSettingsScreen> {
25 ■     bool _tripAlerts = true;
26 ■     bool _approachingAlerts = true;
27 ■     bool _announcementAlerts = true;
28 ■     PermissionStatus? _notifPermission;
29 ■
30 ■     @override
31 ■     void initState() {
32 ■       super.initState();
33 ■       Permission.notification.status.then((PermissionStatus s) {
34 ■         if (mounted) setState(() => _notifPermission = s);
35 ■       });
36 ■     }
37 ■
38 ■     Future<void> _saveSettings() async {
39 ■       final AuthSession? session = ref.read(currentSessionProvider);
40 ■       if (session == null) return;
41 ■       await ref.read(firestoreServiceProvider).saveUserSettings(
42 ■         session.uid,
43 ■         <String, dynamic>{
44 ■           'tripAlerts': _tripAlerts,
45 ■           'approachingAlerts': _approachingAlerts,
46 ■           'announcementAlerts': _announcementAlerts,
47 ■         },
48 ■       );
49 ■       if (mounted) showNmbSnack(context, 'Settings saved', isSuccess: true);
50 ■     }
51 ■
52 ■     Future<void> _requestNotifPermission() async {
53 ■       final PermissionStatus status = await Permission.notification.request();
54 ■       if (!mounted) return;
55 ■       setState(() => _notifPermission = status);
56 ■       if (status.isPermanentlyDenied) {
57 ■         final bool open = await showNmbConfirmDialog(
58 ■           context,

```

```

59 ■         title: 'Enable notifications',
60 ■         message: 'Notifications are turned off for this app. Open system '
61 ■             'settings to enable bus alerts?',
62 ■         confirmLabel: 'Open Settings',
63 ■     );
64 ■     if (open) await openAppSettings();
65 ■ }
66 ■ }
67 ■
68 ■ Future<void> _resetPassword() async {
69 ■     final AuthSession? session = ref.read(currentSessionProvider);
70 ■     if (session == null) return;
71 ■     final Result<void> result = await ref
72 ■         .read(authServiceProvider)
73 ■         .sendPasswordReset(session.email);
74 ■     if (!mounted) return;
75 ■     result.when(
76 ■         ok: (_) => showNmbSnack(
77 ■             context,
78 ■             'Password reset email sent to ${session.email}',
79 ■             isSuccess: true,
80 ■         ),
81 ■         err: (AppFailure f) => showNmbSnack(context, f.message, isError: true),
82 ■     );
83 ■ }
84 ■
85 ■ @override
86 ■ Widget build(BuildContext context) {
87 ■     final bool notifDenied =
88 ■         _notifPermission != null && !_notifPermission!.isGranted;
89 ■
90 ■     return Scaffold(
91 ■         appBar: AppBar(title: const Text('Settings')),
92 ■         body: ResponsiveBody(
93 ■             child: Column(
94 ■                 crossAxisAlignment: CrossAxisAlignment.stretch,
95 ■                 children: <Widget>[
96 ■                     if (notifDenied) ...<Widget>[
97 ■                         NmbCard(
98 ■                             color: NmbColors.warningSoft,
99 ■                             child: Column(
100 ■                                 crossAxisAlignment: CrossAxisAlignment.start,
101 ■                                 children: <Widget>[
102 ■                                     const Row(
103 ■                                         children: <Widget>[
104 ■                                             Icon(Icons.notifications_off_rounded,
105 ■                                                 color: NmbColors.warning,),
106 ■                                             SizedBox(width: 10),
107 ■                                             Expanded(
108 ■                                                 child: Text(
109 ■                                                     'Notifications are off – you won\'t receive '
110 ■                                                     'bus alerts.',
111 ■                                                     style: NmbTypography.bodySecondary,
112 ■                                                 ),
113 ■                                             ),
114 ■                                         ],
115 ■                                     ),
116 ■                                     const SizedBox(height: 10),

```

```

117 ■          OutlinedButton(
118 ■              onPressed: _requestNotifPermission,
119 ■              child: const Text('Enable notifications'),
120 ■          ),
121 ■      ],
122 ■  ),
123 ■ ),
124 ■   const SizedBox(height: 14),
125 ■ ],
126 ■ const Text('Notification preferences',
127 ■     style: NmbTypography.sectionTitle,),
128 ■ const SizedBox(height: 10),
129 ■ NmbCard(
130 ■     padding: const EdgeInsets.symmetric(
131 ■         horizontal: 12, vertical: 4,),
132 ■     child: Column(
133 ■         children: <Widget>[
134 ■             SwitchListTile(
135 ■                 title: const Text('Trip started / ended'),
136 ■                 value: _tripAlerts,
137 ■                 onChanged: (bool v) =>
138 ■                     setState(() => _tripAlerts = v),
139 ■             ),
140 ■             SwitchListTile(
141 ■                 title: const Text('Bus approaching my stop'),
142 ■                 value: _approachingAlerts,
143 ■                 onChanged: (bool v) =>
144 ■                     setState(() => _approachingAlerts = v),
145 ■             ),
146 ■             SwitchListTile(
147 ■                 title: const Text('School announcements'),
148 ■                 value: _announcementAlerts,
149 ■                 onChanged: (bool v) =>
150 ■                     setState(() => _announcementAlerts = v),
151 ■             ),
152 ■         ],
153 ■     ),
154 ■ ),
155 ■ const SizedBox(height: 12),
156 ■ FilledButton(
157 ■     onPressed: _saveSettings,
158 ■     child: const Text('Save preferences'),
159 ■ ),
160 ■ const SizedBox(height: 24),
161 ■ const Text('Account', style: NmbTypography.sectionTitle),
162 ■ const SizedBox(height: 10),
163 ■ NmbCard(
164 ■     onTap: _resetPassword,
165 ■     child: const Row(
166 ■         children: <Widget>[
167 ■             Icon(Icons.lock_reset_rounded, color: NmbColors.primary),
168 ■             SizedBox(width: 12),
169 ■             Expanded(
170 ■                 child: Text('Send password reset email',
171 ■                     style: NmbTypography.cardTitle,),
172 ■             ),
173 ■             Icon(Icons.chevron_right_rounded,
174 ■                 color: NmbColors.textTertiary,),
175 ■         ],
176 ■     ),
177 ■ ),
178 ■ const SizedBox(height: 24),
179 ■ ],
180 ■ ),
181 ■ ),
182 ■ );
183 ■ }
184 ■ }

```

9. Driver App

Shell, one-tap trip dashboard, route list, profile.

driver_shell.dart · driver_home_screen.dart · driver_route_screen.dart ·
driver_profile_screen.dart

lib/features/driver/driver_shell.dart

52 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■ import 'package:go_router/go_router.dart';
4 ■
5 ■ import '../core/widgets/state_views.dart';
6 ■ import '../providers/app_providers.dart';
7 ■
8 ■ /// Driver shell: three tabs only – deliberately minimal so the interface is
9 ■ /// safe to use around a vehicle.
10 ■ class DriverShell extends ConsumerWidget {
11 ■   const DriverShell({super.key, required this.shell});
12 ■
13 ■   final StatefulNavigationShell shell;
14 ■
15 ■   @override
16 ■   Widget build(BuildContext context, WidgetRef ref) {
17 ■     final bool online = ref.watch(isOnlineProvider).valueOrNull ?? true;
18 ■
19 ■     return Scaffold(
20 ■       body: Column(
21 ■         children: <Widget>[
22 ■           if (!online) const OfflineBanner(),
23 ■           Expanded(child: shell),
24 ■         ],
25 ■       ),
26 ■       bottomNavigationBar: NavigationBar(
27 ■         selectedIndex: shell.currentIndex,
28 ■         onDestinationSelected: (int index) => shell.goBranch(
29 ■           index,
30 ■           initialLocation: index == shell.currentIndex,
31 ■         ),
32 ■         destinations: const <Widget>[
33 ■           NavigationDestination(
34 ■             icon: Icon(Icons.directions_bus_outlined),
35 ■             selectedIcon: Icon(Icons.directions_bus_rounded),
36 ■             label: 'Trip',
37 ■           ),
38 ■           NavigationDestination(
39 ■             icon: Icon(Icons.route_outlined),
40 ■             selectedIcon: Icon(Icons.route_rounded),
41 ■             label: 'Route',
42 ■           ),
43 ■           NavigationDestination(
44 ■             icon: Icon(Icons.person_outline_rounded),
45 ■             selectedIcon: Icon(Icons.person_rounded),
46 ■             label: 'Profile',
47 ■           ),
48 ■         ],
49 ■       ),
50 ■     );
51 ■   }
52 ■ }

```

lib/features/driver/screens/driver_home_screen.dart

418 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■ import 'package:geolocator/geolocator.dart';
4 ■
5 ■ import '../core/constants/enums.dart';
6 ■ import '../core/theme/nmb_colors.dart';
7 ■ import '../core/theme/nmb_typography.dart';
8 ■ import '../core/utils/formatters.dart';
9 ■ import '../core/utils/result.dart';
10 ■ import '../core/widgets/nmb_card.dart';
11 ■ import '../core/widgets/nmb_dialogs.dart';
12 ■ import '../core/widgets/responsive_scaffold_body.dart';
13 ■ import '../core/widgets/state_views.dart';
14 ■ import '../models/app_user.dart';
15 ■ import '../models/bus.dart';
16 ■ import '../models/bus_route.dart';
17 ■ import '../models/live_location.dart';
18 ■ import '../models/trip.dart';
19 ■ import '../providers/app_providers.dart';
20 ■ import '../providers/data_providers.dart';
21 ■ import '../services/auth_service.dart';
22 ■ import '../services/driver_trip_service.dart';
23 ■
24 ■ /// Driver home: giant Start/End trip control, GPS + connection status, and
25 ■ /// assigned bus info. Everything is one-tap; nothing requires interaction
26 ■ /// while driving.
27 ■ class DriverHomeScreen extends ConsumerStatefulWidget {
28 ■   const DriverHomeScreen({super.key});
29 ■
30 ■   @override
31 ■   ConsumerState<DriverHomeScreen> createState() => _DriverHomeScreenState();
32 ■ }
33 ■
34 ■ class _DriverHomeScreenState extends ConsumerState<DriverHomeScreen> {
35 ■   bool _busy = false;
36 ■   bool _gpsServiceOn = false;
37 ■   bool _resumedCheckDone = false;
38 ■
39 ■   @override
40 ■   void initState() {
41 ■     super.initState();
42 ■     _refreshGpsStatus();
43 ■   }
44 ■
45 ■   Future<void> _refreshGpsStatus() async {
46 ■     final bool on = await Geolocator.isLocationServiceEnabled();
47 ■     if (mounted) setState(() => _gpsServiceOn = on);
48 ■   }
49 ■
50 ■   /// If the app restarted while a trip is still active on the server,
51 ■   /// offer to resume publishing GPS.
52 ■   Future<void> _maybeResume(Trip trip, AuthSession session) async {
53 ■     if (_resumedCheckDone) return;
54 ■     _resumedCheckDone = true;
55 ■     final DriverTripService svc = ref.read(driverTripServiceProvider);
56 ■     if (svc.hasLocalActiveTrip) return;
57 ■     if (trip.driverId != session.uid) return;
58 ■     await svc.resumePublishing(busId: trip.busId, tripId: trip.id);

```

```

59 ■ }
60 ■
61 ■ Future<void> _startTrip({
62 ■   required AuthSession session,
63 ■   required Bus bus,
64 ■   required BusRoute? route,
65 ■ }) async {
66 ■   final TripDirection? direction = await showModalBottomSheet<TripDirection>(
67 ■     context: context,
68 ■     shape: const RoundedRectangleBorder(
69 ■       borderRadius: BorderRadius.vertical(top: Radius.circular(24)),
70 ■     ),
71 ■     builder: (BuildContext ctx) => SafeArea(
72 ■       child: Padding(
73 ■         padding: const EdgeInsets.all(20),
74 ■         child: Column(
75 ■           mainAxisAlignment: MainAxisAlignment.min,
76 ■           crossAxisAlignment: CrossAxisAlignment.stretch,
77 ■           children: <Widget>[
78 ■             const Text('Start which trip?',
79 ■               style: NmbTypography.sectionTitle,
80 ■               textAlign: TextAlign.center,)),
81 ■             const SizedBox(height: 18),
82 ■             FilledButton.icon(
83 ■               icon: const Icon(Icons.wb_sunny_rounded),
84 ■               label: const Text('Morning pickup – towards school'),
85 ■               onPressed: () =>
86 ■                 Navigator.of(ctx).pop(TripDirection.pickup),
87 ■             ),
88 ■             const SizedBox(height: 12),
89 ■             FilledButton.icon(
90 ■               style: FilledButton.styleFrom(
91 ■                 backgroundColor: NmbColors.accentDark,)),
92 ■               icon: const Icon(Icons.home_rounded),
93 ■               label: const Text('Afternoon drop – towards home'),
94 ■               onPressed: () => Navigator.of(ctx).pop(TripDirection.drop),
95 ■             ),
96 ■           ],
97 ■         ),
98 ■       ),
99 ■     ),
100 ■   );
101 ■   if (direction == null || !mounted) return;
102 ■
103 ■   setState(() => _busy = true);
104 ■   final Result<String> result =
105 ■     await ref.read(driverTripServiceProvider).startTrip(
106 ■       driverId: session.uid,
107 ■       busId: bus.id,
108 ■       routeId: route?.id ?? '',
109 ■       direction: direction.name,
110 ■     );
111 ■   if (!mounted) return;
112 ■   setState(() => _busy = false);
113 ■   result.when(
114 ■     ok: (_) => showNmbSnack(
115 ■       context,
116 ■       'Trip started. Location is now shared with your students.',

```

```

117 ■         isSuccess: true,
118 ■     ),
119 ■     err: (AppFailure f) => showNmbSnack(context, f.message, isError: true),
120 ■ );
121 ■ _refreshGpsStatus();
122 ■ }
123 ■
124 ■ Future<void> _endTrip() async {
125 ■     final bool confirmed = await showNmbConfirmDialog(
126 ■         context,
127 ■         title: 'End trip?',
128 ■         message: 'Live tracking will stop and students will no longer see '
129 ■             'the bus location.',
130 ■         confirmLabel: 'End trip',
131 ■         destructive: true,
132 ■         icon: Icons.stop_circle_outlined,
133 ■     );
134 ■     if (!confirmed || !mounted) return;
135 ■
136 ■     setState(() => _busy = true);
137 ■     final Result<void> result =
138 ■         await ref.read(driverTripServiceProvider).endTrip();
139 ■     if (!mounted) return;
140 ■     setState(() => _busy = false);
141 ■     result.when(
142 ■         ok: (_) =>
143 ■             showNmbSnack(context, 'Trip ended. Thank you!', isSuccess: true),
144 ■         err: (AppFailure f) => showNmbSnack(context, f.message, isError: true),
145 ■     );
146 ■ }
147 ■
148 ■ @override
149 ■ Widget build(BuildContext context) {
150 ■     final AuthSession? session = ref.watch(currentSessionProvider);
151 ■     final AppUser? me = ref.watch(myProfileProvider).valueOrNull;
152 ■     final AsyncValue<Bus?> busAsync = ref.watch(myBusProvider);
153 ■     final BusRoute? route = ref.watch(myRouteProvider).valueOrNull;
154 ■     final Trip? trip = ref.watch(myBusActiveTripProvider).valueOrNull;
155 ■     final LiveLocation? live =
156 ■         ref.watch(myBusLiveLocationProvider).valueOrNull;
157 ■     final bool online = ref.watch(isOnlineProvider).valueOrNull ?? true;
158 ■
159 ■     if (session == null) return const LoadingView();
160 ■
161 ■     if (trip != null && trip.isActive) {
162 ■         _maybeResume(trip, session);
163 ■     }
164 ■
165 ■     return Scaffold(
166 ■         appBar: AppBar(title: const Text('Driver Dashboard')),
167 ■         body: busAsync.when(
168 ■             loading: () => const LoadingView(),
169 ■             error: (Object e, _) => ErrorView(
170 ■                 message: 'Couldn\'t load your bus. Check your connection.',
171 ■                 onRetry: () => ref.invalidate(myBusProvider),
172 ■             ),
173 ■         data: (Bus? bus) {
174 ■             if (bus == null) {

```

```

175 ■         return const EmptyState(
176 ■             icon: Icons.no_transfer_rounded,
177 ■             title: 'No bus assigned',
178 ■             message: 'The school hasn\'t assigned a bus to your account '
179 ■                 'yet. Please contact the transport office.',
180 ■         );
181 ■     }
182 ■
183 ■     final bool onTrip = trip != null && trip.isActive;
184 ■
185 ■     return ResponsiveBody(
186 ■         child: Column(
187 ■             crossAxisAlignment: CrossAxisAlignment.stretch,
188 ■             children: <Widget>[
189 ■                 Text(
190 ■                     Formatters.greetingForNow(
191 ■                         firstName: me?.firstName ?? 'Driver',),
192 ■                     style: NmbTypography.displayTitle,
193 ■                 ),
194 ■                 const SizedBox(height: 16),
195 ■
196 ■                 // Assigned bus
197 ■                 NmbCard(
198 ■                     color: onTrip ? NmbColors.successSoft : NmbColors.surface,
199 ■                     borderColor: onTrip ? NmbColors.success : null,
200 ■                     child: Row(
201 ■                         children: <Widget>[
202 ■                             Container(
203 ■                                 width: 54,
204 ■                                 height: 54,
205 ■                                 decoration: BoxDecoration(
206 ■                                     color: NmbColors.accent,
207 ■                                     borderRadius: BorderRadius.circular(16),
208 ■                                 ),
209 ■                                 child: const Icon(Icons.directions_bus_rounded,
210 ■                                     color: Colors.white, size: 30,),
211 ■                             ),
212 ■                             const SizedBox(width: 14),
213 ■                             Expanded(
214 ■                                 child: Column(
215 ■                                     crossAxisAlignment: CrossAxisAlignment.start,
216 ■                                     children: <Widget>[
217 ■                                         Text(bus.busNumber,
218 ■                                             style: NmbTypography.sectionTitle,),
219 ■                                         Text(bus.plateNumber,
220 ■                                             style: NmbTypography.bodySecondary,),
221 ■                                         if (route != null)
222 ■                                             Text(route.name,
223 ■                                                 style: NmbTypography.caption,),
224 ■                                     ],
225 ■                                 ),
226 ■                             ),
227 ■                             Column(
228 ■                                 crossAxisAlignment: CrossAxisAlignment.end,
229 ■                                 children: <Widget>[
230 ■                                     Text(
231 ■                                         onTrip ? 'TRIP ACTIVE' : 'IDLE',
232 ■                                         style: NmbTypography.caption.copyWith(

```

```

233 ■         color: onTrip
234 ■             ? NmbColors.success
235 ■             : NmbColors.textTertiary,
236 ■         fontWeight: FontWeight.w800,
237 ■     ),
238 ■ ),
239 ■     if (onTrip && trip.startedAt != null)
240 ■         Text(
241 ■             'since '
242 ■             '${Formatters.timeOfDay(trip.startedAt!)}',
243 ■             style: NmbTypography.caption,
244 ■         ),
245 ■     ],
246 ■ ),
247 ■ ],
248 ■ ),
249 ■ ),
250 ■ const SizedBox(height: 14),
251 ■
252 ■ // GPS + connection status
253 ■ Row(
254 ■     children: <Widget>[
255 ■         Expanded(
256 ■             child: _StatusTile(
257 ■                 icon: _gpsServiceOn
258 ■                     ? Icons.gps_fixed_rounded
259 ■                     : Icons.gps_off_rounded,
260 ■                 label: 'GPS',
261 ■                 value: _gpsServiceOn ? 'On' : 'Off',
262 ■                 ok: _gpsServiceOn,
263 ■                 onTap: () async {
264 ■                     await Geolocator.openLocationSettings();
265 ■                     _refreshGpsStatus();
266 ■                 },
267 ■             ),
268 ■         ),
269 ■         const SizedBox(width: 12),
270 ■         Expanded(
271 ■             child: _StatusTile(
272 ■                 icon: online
273 ■                     ? Icons.wifi_rounded
274 ■                     : Icons.wifi_off_rounded,
275 ■                 label: 'Internet',
276 ■                 value: online ? 'Connected' : 'Offline',
277 ■                 ok: online,
278 ■             ),
279 ■         ),
280 ■         const SizedBox(width: 12),
281 ■         Expanded(
282 ■             child: _StatusTile(
283 ■                 icon: onTrip && live != null
284 ■                     ? Icons.podcasts_rounded
285 ■                     : Icons.portable_wifi_off_rounded,
286 ■                 label: 'Sharing',
287 ■                 value: onTrip
288 ■                     ? (live != null ? 'Live' : 'Waiting...')
289 ■                     : 'Stopped',
290 ■                 ok: onTrip && live != null,

```

```

291 ■         ),
292 ■       ),
293 ■     ],
294 ■   ),
295 ■   const SizedBox(height: 14),
296 ■
297 ■   // Current location (while on trip)
298 ■   if (onTrip && live != null)
299 ■     NmbCard(
300 ■       child: Row(
301 ■         children: <Widget>[
302 ■           const Icon(Icons.my_location_rounded,
303 ■             color: NmbColors.primary, ),
304 ■           const SizedBox(width: 12),
305 ■           Expanded(
306 ■             child: Column(
307 ■               crossAxisAlignment: CrossAxisAlignment.start,
308 ■               children: <Widget>[
309 ■                 Text(
310 ■                   'Sharing location - '
311 ■                   '${live.speedKmh.toStringAsFixed(0)} km/h',
312 ■                   style: NmbTypography.cardTitle,
313 ■                 ),
314 ■                 Text(
315 ■                   'Last sent '
316 ■                   '${Formatters.relativeTime(live.updatedAt)}',
317 ■                   style: NmbTypography.bodySecondary,
318 ■                 ),
319 ■               ],
320 ■             ),
321 ■           ),
322 ■         ],
323 ■       ),
324 ■     ),
325 ■     const SizedBox(height: 20),
326 ■
327 ■     // THE control: one giant button.
328 ■     if (_busy)
329 ■       const Center(
330 ■         child: Padding(
331 ■           padding: EdgeInsets.all(12),
332 ■           child: CircularProgressIndicator(),
333 ■         ),
334 ■       )
335 ■     else if (!onTrip)
336 ■       FilledButton.icon(
337 ■         style: FilledButton.styleFrom(
338 ■           minimumSize: const Size.fromHeight(72),
339 ■           backgroundColor: NmbColors.success,
340 ■           textStyle: NmbTypography.button.copyWith(fontSize: 19),
341 ■         ),
342 ■         onPressed: () => _startTrip(
343 ■           session: session,
344 ■           bus: bus,
345 ■           route: route,
346 ■         ),
347 ■         icon: const Icon(Icons.play_arrow_rounded, size: 30),
348 ■         label: const Text('START TRIP'),
349 ■       )

```

```

349 ■         else
350 ■             FilledButton.icon(
351 ■                 style: FilledButton.styleFrom(
352 ■                     minimumSize: const Size.fromHeight(72),
353 ■                     backgroundColor: NmbColors.danger,
354 ■                     textStyle: NmbTypography.button.copyWith(fontSize: 19),
355 ■                 ),
356 ■                 onPressed: _endTrip,
357 ■                 icon: const Icon(Icons.stop_rounded, size: 30),
358 ■                 label: const Text('END TRIP'),
359 ■             ),
360 ■         const SizedBox(height: 12),
361 ■         Text(
362 ■             onTrip
363 ■                 ? 'Keep the phone connected to power. Location is '
364 ■                 'shared automatically – no further action needed.'
365 ■                 : 'Start the trip when you begin the route. Students '
366 ■                 'will see the bus live on their map.',
367 ■             style: NmbTypography.caption,
368 ■             textAlign: TextAlign.center,
369 ■         ),
370 ■         const SizedBox(height: 24),
371 ■     ],
372 ■ ),
373 ■ );
374 ■ },
375 ■ ),
376 ■ );
377 ■ }
378 ■ }
379 ■
380 ■ class _StatusTile extends StatelessWidget {
381 ■     const _StatusTile({
382 ■         required this.icon,
383 ■         required this.label,
384 ■         required this.value,
385 ■         required this.ok,
386 ■         this.onTap,
387 ■     });
388 ■
389 ■     final IconData icon;
390 ■     final String label;
391 ■     final String value;
392 ■     final bool ok;
393 ■     final VoidCallback? onTap;
394 ■
395 ■     @override
396 ■     Widget build(BuildContext context) {
397 ■         return NmbCard(
398 ■             onTap: onTap,
399 ■             padding: const EdgeInsets.symmetric(vertical: 14, horizontal: 8),
400 ■             color: ok ? NmbColors.surface : NmbColors.dangerSoft,
401 ■             child: Column(
402 ■                 children: <Widget>[
403 ■                     Icon(icon,
404 ■                         color: ok ? NmbColors.success : NmbColors.danger, size: 26,),
405 ■                     const SizedBox(height: 6),
406 ■                     Text(label, style: NmbTypography.caption),
407 ■
408 ■                     Text(
409 ■                         value,
410 ■                         style: NmbTypography.caption.copyWith(
411 ■                             fontWeight: FontWeight.w800,
412 ■                             color: ok ? NmbColors.textPrimary : NmbColors.danger,
413 ■                         ),
414 ■                     ),
415 ■                 ],
416 ■             ),
417 ■         );
418 ■     }

```

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/theme/nmb_colors.dart';
5 ■ import '../core/theme/nmb_typography.dart';
6 ■ import '../core/widgets/nmb_card.dart';
7 ■ import '../core/widgets/responsive_scaffold_body.dart';
8 ■ import '../core/widgets/state_views.dart';
9 ■ import '../models/bus_route.dart';
10 ■ import '../models/bus_stop.dart';
11 ■ import '../providers/data_providers.dart';
12 ■
13 ■ /// Read-only route/stop list for the driver – big text, glanceable.
14 ■ class DriverRouteScreen extends ConsumerWidget {
15 ■   const DriverRouteScreen({super.key});
16 ■
17 ■   @override
18 ■   Widget build(BuildContext context, WidgetRef ref) {
19 ■     final BusRoute? route = ref.watch(myRouteProvider).valueOrNull;
20 ■     final AsyncValue<List<BusStop>> stopsAsync = ref.watch(myStopsProvider);
21 ■
22 ■     return Scaffold(
23 ■       appBar: AppBar(title: const Text('My Route')),
24 ■       body: stopsAsync.when(
25 ■         loading: () => const LoadingView(),
26 ■         error: (Object e, _) => ErrorView(
27 ■           message: 'Couldn\'t load the route.',
28 ■           onRetry: () => ref.invalidate(myStopsProvider),
29 ■         ),
30 ■         data: (List<BusStop> stops) {
31 ■           if (stops.isEmpty) {
32 ■             return const EmptyState(
33 ■               icon: Icons.route_outlined,
34 ■               title: 'No route published',
35 ■               message: 'The transport office hasn\'t published your route '
36 ■                 'yet.',
37 ■             );
38 ■           }
39 ■           return ResponsiveBody(
40 ■             child: Column(
41 ■               crossAxisAlignment: CrossAxisAlignment.stretch,
42 ■               children: <Widget>[
43 ■                 if (route != null) ...<Widget>[
44 ■                   Text(route.name, style: NmbTypography.screenTitle),
45 ■                   const SizedBox(height: 4),
46 ■                   Text(
47 ■                     '${stops.length} stops • ${route.direction.label}',
48 ■                     style: NmbTypography.bodySecondary,
49 ■                   ),
50 ■                   const SizedBox(height: 16),
51 ■                 ],
52 ■                 for (int i = 0; i < stops.length; i++) ...<Widget>[
53 ■                   NmbCard(
54 ■                     padding: const EdgeInsets.all(16),
55 ■                     child: Row(
56 ■                       children: <Widget>[
57 ■                         Container(
58 ■                           width: 34,

```

```

59 ■         height: 34,
60 ■         decoration: const BoxDecoration(
61 ■             color: NmbColors.primarySoft,
62 ■             shape: BoxShape.circle,
63 ■         ),
64 ■         child: Center(
65 ■             child: Text(
66 ■                 '${i + 1}',
67 ■                 style: NmbTypography.cardTitle
68 ■                     .copyWith(color: NmbColors.primaryDark),
69 ■             ),
70 ■         ),
71 ■     ),
72 ■     const SizedBox(width: 14),
73 ■     Expanded(
74 ■         child: Text(stops[i].name,
75 ■             style: NmbTypography.cardTitle,),
76 ■     ),
77 ■     if (stops[i].scheduledTime != null)
78 ■         Text(
79 ■             stops[i].scheduledTime!,
80 ■             style: NmbTypography.cardTitle
81 ■                 .copyWith(color: NmbColors.primary),
82 ■         ),
83 ■     ],
84 ■ ),
85 ■ ),
86 ■     const SizedBox(height: 10),
87 ■ ],
88 ■     const SizedBox(height: 16),
89 ■ ],
90 ■ ),
91 ■ );
92 ■ },
93 ■ ),
94 ■ );
95 ■ }
96 ■ }

```

lib/features/driver/screens/driver_profile_screen.dart

154 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/theme/nmb_colors.dart';
5 ■ import '../core/theme/nmb_typography.dart';
6 ■ import '../core/widgets/nmb_card.dart';
7 ■ import '../core/widgets/nmb_dialogs.dart';
8 ■ import '../core/widgets/responsive_scaffold_body.dart';
9 ■ import '../core/widgets/state_views.dart';
10 ■ import '../models/app_user.dart';
11 ■ import '../models/bus.dart';
12 ■ import '../models/trip.dart';
13 ■ import '../providers/app_providers.dart';
14 ■ import '../providers/data_providers.dart';
15 ■ import '../services/auth_service.dart';
16 ■
17 ■ class DriverProfileScreen extends ConsumerWidget {
18 ■   const DriverProfileScreen({super.key});
19 ■
20 ■   Future<void> _logout(BuildContext context, WidgetRef ref) async {
21 ■     final Trip? trip = ref.read(myBusActiveTripProvider).valueOrNull;
22 ■     if (trip != null && trip.isActive) {
23 ■       showNmbSnack(
24 ■         context,
25 ■         'Please end your active trip before logging out.',
26 ■         isError: true,
27 ■       );
28 ■       return;
29 ■     }
30 ■     final bool confirmed = await showNmbConfirmDialog(
31 ■       context,
32 ■       title: 'Log out?',
33 ■       message: 'You\'ll need your driver account credentials to sign '
34 ■         'back in.',
35 ■       confirmLabel: 'Log out',
36 ■       destructive: true,
37 ■       icon: Icons.logout_rounded,
38 ■     );
39 ■     if (!confirmed) return;
40 ■     final AuthSession? session = ref.read(currentSessionProvider);
41 ■     if (session != null) {
42 ■       await ref
43 ■         .read(notificationServiceProvider)
44 ■         .unregisterDevice(session.uid);
45 ■     }
46 ■     await ref.read(authServiceProvider).signOut();
47 ■   }
48 ■
49 ■   @override
50 ■   Widget build(BuildContext context, WidgetRef ref) {
51 ■     final AsyncValue<AppUser?> profileAsync = ref.watch(myProfileProvider);
52 ■     final Bus? bus = ref.watch(myBusProvider).valueOrNull;
53 ■     final Map<String, dynamic>? school =
54 ■       ref.watch(schoolConfigProvider).valueOrNull;
55 ■
56 ■     return Scaffold(
57 ■       appBar: AppBar(title: const Text('Profile')),
58 ■       body: profileAsync.when(

```

```

59 ■ loading: () => const LoadingView(),
60 ■ error: (Object e, _) => ErrorView(
61 ■   message: 'Couldn\'t load your profile.',
62 ■   onRetry: () => ref.invalidate(myProfileProvider),
63 ■ ),
64 ■ data: (AppUser? me) {
65 ■   if (me == null) {
66 ■     return const ErrorView(
67 ■       title: 'Profile unavailable',
68 ■       message: 'Please contact the transport office.',
69 ■     );
70 ■   }
71 ■   return ResponsiveBody(
72 ■     child: Column(
73 ■       crossAxisAlignment: CrossAxisAlignment.stretch,
74 ■       children: <Widget>[
75 ■         NmbCard(
76 ■           child: Row(
77 ■             children: <Widget>[
78 ■               const CircleAvatar(
79 ■                 radius: 30,
80 ■                 backgroundColor: NmbColors.accentSoft,
81 ■                 child: Icon(Icons.badge_rounded,
82 ■                   color: NmbColors.accentDark, size: 30,),
83 ■               ),
84 ■               const SizedBox(width: 14),
85 ■               Expanded(
86 ■                 child: Column(
87 ■                   crossAxisAlignment: CrossAxisAlignment.start,
88 ■                   children: <Widget>[
89 ■                     Text(me.fullName,
90 ■                       style: NmbTypography.sectionTitle,),
91 ■                     Text(me.email,
92 ■                       style: NmbTypography.bodySecondary,),
93 ■                     const Text('School bus driver',
94 ■                       style: NmbTypography.caption,),
95 ■                   ],
96 ■                 ),
97 ■               ),
98 ■             ],
99 ■           ),
100 ■         ),
101 ■         const SizedBox(height: 14),
102 ■         NmbCard(
103 ■           child: ListTile(
104 ■             contentPadding: EdgeInsets.zero,
105 ■             leading: const Icon(Icons.directions_bus_rounded,
106 ■               color: NmbColors.primary,),
107 ■             title: const Text('Assigned bus'),
108 ■             trailing: Text(
109 ■               bus?.busNumber ?? 'Not assigned',
110 ■               style: NmbTypography.cardTitle,
111 ■             ),
112 ■           ),
113 ■         ),
114 ■         const SizedBox(height: 14),
115 ■         if (school?['phone'] != null)
116 ■           NmbCard(

```

```

117 ■         child: Row(
118 ■           children: <Widget>[
119 ■             const Icon(Icons.support_agent_rounded,
120 ■               color: NmbColors.info,),
121 ■             const SizedBox(width: 12),
122 ■             Expanded(
123 ■               child: Column(
124 ■                 crossAxisAlignment: CrossAxisAlignment.start,
125 ■                 children: <Widget>[
126 ■                   const Text('Transport office',
127 ■                     style: NmbTypography.cardTitle,),
128 ■                   Text(school!['phone'] as String,
129 ■                     style: NmbTypography.bodySecondary,),
130 ■                 ],
131 ■               ),
132 ■             ),
133 ■           ],
134 ■         ),
135 ■       ),
136 ■     const SizedBox(height: 20),
137 ■     OutlinedButton.icon(
138 ■       style: OutlinedButton.styleFrom(
139 ■         foregroundColor: NmbColors.danger,
140 ■         side: const BorderSide(color: NmbColors.danger),
141 ■       ),
142 ■       onPressed: () => _logout(context, ref),
143 ■       icon: const Icon(Icons.logout_rounded),
144 ■       label: const Text('Log out'),
145 ■     ),
146 ■     const SizedBox(height: 24),
147 ■   ],
148 ■ ),
149 ■ );
150 ■ },
151 ■ ),
152 ■ );
153 ■ }
154 ■ }

```

10. Admin App

Shell + all management screens.

admin_shell.dart · admin_dashboard_screen.dart · user_editor_sheet.dart ·
 admin_students_screen.dart · admin_drivers_screen.dart · admin_buses_screen.dart ·
 admin_routes_stops_screen.dart · admin_monitoring_screen.dart ·
 admin_notifications_screen.dart · admin_audit_logs_screen.dart ·
 admin_settings_screen.dart

lib/features/admin/admin_shell.dart

148 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■ import 'package:go_router/go_router.dart';
4 ■
5 ■ import '../core/theme/nmb_colors.dart';
6 ■ import '../core/theme/nmb_typography.dart';
7 ■ import '../core/widgets/nmb_dialogs.dart';
8 ■ import '../core/widgets/state_views.dart';
9 ■ import '../providers/app_providers.dart';
10 ■ import '../services/auth_service.dart';
11 ■
12 ■ /// Admin shell with a navigation drawer (admin has more sections than fit a
13 ■ /// bottom bar, but the design stays friendly, not "corporate dashboard").
14 ■ class AdminShell extends ConsumerWidget {
15 ■   const AdminShell({super.key, required this.child});
16 ■
17 ■   final Widget child;
18 ■
19 ■   static const List<(String, String, IconData)> _items =
20 ■     <(String, String, IconData)>[
21 ■       ('/admin/dashboard', 'Dashboard', Icons.dashboard_rounded),
22 ■       ('/admin/students', 'Students', Icons.school_rounded),
23 ■       ('/admin/drivers', 'Drivers', Icons.badge_rounded),
24 ■       ('/admin/buses', 'Buses', Icons.directions_bus_rounded),
25 ■       ('/admin/routes', 'Routes & Stops', Icons.route_rounded),
26 ■       ('/admin/monitoring', 'Live Monitoring', Icons.radar_rounded),
27 ■       ('/admin/notifications', 'Notifications', Icons.campaign_rounded),
28 ■       ('/admin/audit', 'Activity Logs', Icons.receipt_long_rounded),
29 ■       ('/admin/settings', 'Settings', Icons.settings_rounded),
30 ■     ];
31 ■
32 ■   Future<void> _logout(BuildContext context, WidgetRef ref) async {
33 ■     final bool ok = await showNmbConfirmDialog(
34 ■       context,
35 ■       title: 'Log out?',
36 ■       message: 'End your admin session?',
37 ■       confirmLabel: 'Log out',
38 ■       destructive: true,
39 ■       icon: Icons.logout_rounded,
40 ■     );
41 ■     if (!ok) return;
42 ■     final AuthSession? session = ref.read(currentSessionProvider);
43 ■     if (session != null) {
44 ■       await ref
45 ■         .read(notificationServiceProvider)
46 ■         .unregisterDevice(session.uid);
47 ■     }
48 ■     await ref.read(authServiceProvider).signOut();
49 ■   }
50 ■
51 ■   @override
52 ■   Widget build(BuildContext context, WidgetRef ref) {
53 ■     final String location = GoRouterState.of(context).matchedLocation;
54 ■     final bool online = ref.watch(isOnlineProvider).valueOrNull ?? true;
55 ■
56 ■     return Scaffold(
57 ■       drawer: Drawer(
58 ■         child: SafeArea(

```

```

59 ■      child: Column(
60 ■        children: <Widget>[
61 ■          Padding(
62 ■            padding: const EdgeInsets.all(20),
63 ■            child: Row(
64 ■              children: <Widget>[
65 ■                Container(
66 ■                  width: 46,
67 ■                  height: 46,
68 ■                  decoration: BoxDecoration(
69 ■                    color: NmbColors.primary,
70 ■                    borderRadius: BorderRadius.circular(14),
71 ■                  ),
72 ■                  child: const Icon(Icons.directions_bus_rounded,
73 ■                    color: Colors.white,),
74 ■                ),
75 ■                const SizedBox(width: 12),
76 ■                const Expanded(
77 ■                  child: Column(
78 ■                    crossAxisAlignment: CrossAxisAlignment.start,
79 ■                    children: <Widget>[
80 ■                      Text('Never Miss Bus',
81 ■                        style: NmbTypography.sectionTitle,),
82 ■                      Text('Admin Console',
83 ■                        style: NmbTypography.caption,),
84 ■                    ],
85 ■                  ),
86 ■                ),
87 ■              ],
88 ■            ),
89 ■          ),
90 ■          const Divider(),
91 ■          Expanded(
92 ■            child: ListView(
93 ■              padding: const EdgeInsets.symmetric(
94 ■                horizontal: 10, vertical: 6,),
95 ■              children: <Widget>[
96 ■                for (final (String path, String label, IconData icon)
97 ■                  in _items)
98 ■                  ListTile(
99 ■                    shape: RoundedRectangleBorder(
100 ■                      borderRadius: BorderRadius.circular(12),
101 ■                    ),
102 ■                    selected: location.startsWith(path),
103 ■                    selectedTileColor: NmbColors.primarySoft,
104 ■                    leading: Icon(icon),
105 ■                    title: Text(
106 ■                      label,
107 ■                      style: NmbTypography.body.copyWith(
108 ■                        fontWeight: location.startsWith(path)
109 ■                          ? FontWeight.w700
110 ■                          : FontWeight.w500,
111 ■                    ),
112 ■                  ),
113 ■                onTap: () {
114 ■                  Navigator.of(context).pop();
115 ■                  context.go(path);
116 ■                },

```

```
117 ■         ),
118 ■       ],
119 ■     ),
120 ■   ),
121 ■   const Divider(),
122 ■   ListTile(
123 ■     leading:
124 ■       const Icon(Icons.logout_rounded, color: NmbColors.danger),
125 ■     title: Text(
126 ■       'Log out',
127 ■       style:
128 ■         NmbTypography.body.copyWith(color: NmbColors.danger),
129 ■     ),
130 ■     onTap: () {
131 ■       Navigator.of(context).pop();
132 ■       _logout(context, ref);
133 ■     },
134 ■   ),
135 ■   const SizedBox(height: 8),
136 ■ ],
137 ■ ),
138 ■ ),
139 ■ ),
140 ■ body: Column(
141 ■   children: <Widget>[
142 ■     if (!online) const OfflineBanner(),
143 ■     Expanded(child: child),
144 ■   ],
145 ■ ),
146 ■ );
147 ■ }
148 ■ }
```

lib/features/admin/screens/admin_dashboard_screen.dart

223 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■ import 'package:go_router/go_router.dart';
4 ■
5 ■ import '../core/theme/nmb_colors.dart';
6 ■ import '../core/theme/nmb_typography.dart';
7 ■ import '../core/widgets/nmb_card.dart';
8 ■ import '../core/widgets/responsive_scaffold_body.dart';
9 ■ import '../models/app_user.dart';
10 ■ import '../models/bus.dart';
11 ■ import '../models/trip.dart';
12 ■ import '../providers/data_providers.dart';
13 ■
14 ■ /// Admin dashboard: at-a-glance counts + active trips + shortcuts.
15 ■ class AdminDashboardScreen extends ConsumerWidget {
16 ■   const AdminDashboardScreen({super.key});
17 ■
18 ■   @override
19 ■   Widget build(BuildContext context, WidgetRef ref) {
20 ■     final List<AppUser> students =
21 ■       ref.watch(allStudentsProvider).valueOrNull ?? const <AppUser>[];
22 ■     final List<AppUser> drivers =
23 ■       ref.watch(allDriversProvider).valueOrNull ?? const <AppUser>[];
24 ■     final List<Bus> buses =
25 ■       ref.watch(allBusesProvider).valueOrNull ?? const <Bus>[];
26 ■     final List<Trip> activeTrips =
27 ■       ref.watch(activeTripsProvider).valueOrNull ?? const <Trip>[];
28 ■
29 ■     return Scaffold(
30 ■       appBar: AppBar(title: const Text('Dashboard')),
31 ■       body: ResponsiveBody(
32 ■         child: Column(
33 ■           crossAxisAlignment: CrossAxisAlignment.stretch,
34 ■           children: <Widget>[
35 ■             Row(
36 ■               children: <Widget>[
37 ■                 Expanded(
38 ■                   child: _StatCard(
39 ■                     label: 'Students',
40 ■                     value: '${students.length}',
41 ■                     icon: Icons.school_rounded,
42 ■                     color: NmbColors.primary,
43 ■                     onTap: () => context.go('/admin/students'),
44 ■                   ),
45 ■                 ),
46 ■                 const SizedBox(width: 12),
47 ■                 Expanded(
48 ■                   child: _StatCard(
49 ■                     label: 'Drivers',
50 ■                     value: '${drivers.length}',
51 ■                     icon: Icons.badge_rounded,
52 ■                     color: NmbColors.accentDark,
53 ■                     onTap: () => context.go('/admin/drivers'),
54 ■                   ),
55 ■                 ),
56 ■               ],
57 ■             ),
58 ■             const SizedBox(height: 12),

```

```

59 ■      Row(
60 ■        children: <Widget>[
61 ■          Expanded(
62 ■            child: _StatCard(
63 ■              label: 'Buses',
64 ■              value: '${buses.length}',
65 ■              icon: Icons.directions_bus_rounded,
66 ■              color: NmbColors.info,
67 ■              onTap: () => context.go('/admin/buses'),
68 ■            ),
69 ■          ),
70 ■          const SizedBox(width: 12),
71 ■          Expanded(
72 ■            child: _StatCard(
73 ■              label: 'Active trips',
74 ■              value: '${activeTrips.length}',
75 ■              icon: Icons.radar_rounded,
76 ■              color: NmbColors.success,
77 ■              onTap: () => context.go('/admin/monitoring'),
78 ■            ),
79 ■          ),
80 ■        ],
81 ■      ),
82 ■      const SizedBox(height: 22),
83 ■      const Text('Active trips', style: NmbTypography.sectionTitle),
84 ■      const SizedBox(height: 10),
85 ■      if (activeTrips.isEmpty)
86 ■        const NmbCard(
87 ■          child: Row(
88 ■            children: <Widget>[
89 ■              Icon(Icons.nightlight_round,
90 ■                color: NmbColors.textTertiary,),
91 ■              SizedBox(width: 12),
92 ■              Expanded(
93 ■                child: Text(
94 ■                  'No buses are on a trip right now.',
95 ■                  style: NmbTypography.bodySecondary,
96 ■                ),
97 ■              ),
98 ■            ],
99 ■          ),
100 ■        )
101 ■      else
102 ■        for (final Trip trip in activeTrips) ...<Widget>[
103 ■          _ActiveTripTile(trip: trip, buses: buses),
104 ■          const SizedBox(height: 10),
105 ■        ],
106 ■      const SizedBox(height: 22),
107 ■      const Text('Quick actions', style: NmbTypography.sectionTitle),
108 ■      const SizedBox(height: 10),
109 ■      NmbCard(
110 ■        padding: const EdgeInsets.symmetric(vertical: 6),
111 ■        child: Column(
112 ■          children: <Widget>[
113 ■            ListTile(
114 ■              leading: const Icon(Icons.person_add_alt_1_rounded,
115 ■                color: NmbColors.primary,),
116 ■              title: const Text('Add a student or driver'),

```

```

117 ■         trailing: const Icon(Icons.chevron_right_rounded),
118 ■         onTap: () => context.go('/admin/students'),
119 ■     ),
120 ■     const Divider(height: 1),
121 ■     ListTile(
122 ■         leading: const Icon(Icons.campaign_rounded),
123 ■         color: NmbColors.accentDark,),
124 ■         title: const Text('Send an announcement'),
125 ■         trailing: const Icon(Icons.chevron_right_rounded),
126 ■         onTap: () => context.go('/admin/notifications'),
127 ■     ),
128 ■     const Divider(height: 1),
129 ■     ListTile(
130 ■         leading: const Icon(Icons.receipt_long_rounded),
131 ■         color: NmbColors.info,),
132 ■         title: const Text('Review activity logs'),
133 ■         trailing: const Icon(Icons.chevron_right_rounded),
134 ■         onTap: () => context.go('/admin/audit'),
135 ■     ),
136 ■ ],
137 ■ ),
138 ■ ),
139 ■     const SizedBox(height: 24),
140 ■ ],
141 ■ ),
142 ■ ),
143 ■ );
144 ■ }
145 ■ }
146 ■
147 ■ class _StatCard extends StatelessWidget {
148 ■   const _StatCard({
149 ■     required this.label,
150 ■     required this.value,
151 ■     required this.icon,
152 ■     required this.color,
153 ■     required this.onTap,
154 ■   });
155 ■
156 ■   final String label;
157 ■   final String value;
158 ■   final IconData icon;
159 ■   final Color color;
160 ■   final VoidCallback onTap;
161 ■
162 ■   @override
163 ■   Widget build(BuildContext context) {
164 ■     return NmbCard(
165 ■       onTap: onTap,
166 ■       child: Column(
167 ■         crossAxisAlignment: CrossAxisAlignment.start,
168 ■         children: <Widget>[
169 ■           Icon(icon, color: color, size: 26),
170 ■           const SizedBox(height: 10),
171 ■           Text(value, style: NmbTypography.statLarge),
172 ■           Text(label, style: NmbTypography.bodySecondary),
173 ■         ],
174 ■       ),

```

```

175 ■   );
176 ■   }
177 ■ }
178 ■
179 ■ class _ActiveTripTile extends StatelessWidget {
180 ■   const _ActiveTripTile({required this.trip, required this.buses});
181 ■
182 ■   final Trip trip;
183 ■   final List<Bus> buses;
184 ■
185 ■   @override
186 ■   Widget build(BuildContext context) {
187 ■     Bus? bus;
188 ■     for (final Bus b in buses) {
189 ■       if (b.id == trip.busId) bus = b;
190 ■     }
191 ■     return NmbCard(
192 ■       onTap: () => context.go('/admin/monitoring'),
193 ■       child: Row(
194 ■         children: <Widget>[
195 ■           Container(
196 ■             width: 42,
197 ■             height: 42,
198 ■             decoration: BoxDecoration(
199 ■               color: NmbColors.successSoft,
200 ■               borderRadius: BorderRadius.circular(12),
201 ■             ),
202 ■             child: const Icon(Icons.directions_bus_rounded,
203 ■               color: NmbColors.success,),
204 ■           ),
205 ■           const SizedBox(width: 12),
206 ■           Expanded(
207 ■             child: Column(
208 ■               crossAxisAlignment: CrossAxisAlignment.start,
209 ■               children: <Widget>[
210 ■                 Text(bus?.busNumber ?? trip.busId,
211 ■                   style: NmbTypography.cardTitle,),
212 ■                 Text('${trip.direction.label} trip in progress',
213 ■                   style: NmbTypography.bodySecondary,),
214 ■               ],
215 ■             ),
216 ■           ),
217 ■           const Icon(Icons.chevron_right_rounded,
218 ■             color: NmbColors.textTertiary,),
219 ■         ],
220 ■       ),
221 ■     );
222 ■   }
223 ■ }

```

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/theme/nmb_typography.dart';
5 ■ import '../core/utils/result.dart';
6 ■ import '../core/utils/validators.dart';
7 ■ import '../core/widgets/nmb_dialogs.dart';
8 ■ import '../models/app_user.dart';
9 ■ import '../models/bus.dart';
10 ■ import '../models/bus_stop.dart';
11 ■ import '../providers/app_providers.dart';
12 ■ import '../providers/data_providers.dart';
13 ■
14 ■ /// Bottom sheet to provision a new student/driver account.
15 ■ /// Provisioning goes through the admin-only `provisionUser` Cloud Function –
16 ■ /// the app cannot create privileged accounts by itself.
17 ■ class UserEditorSheet extends ConsumerStatefulWidget {
18 ■   const UserEditorSheet({super.key, required this.role});
19 ■
20 ■   final String role; // 'student' | 'driver'
21 ■
22 ■   static Future<void> show(BuildContext context, {required String role}) =>
23 ■     showModalBottomSheet<void>(
24 ■       context: context,
25 ■       isScrollControlled: true,
26 ■       shape: const RoundedRectangleBorder(
27 ■         borderRadius: BorderRadius.vertical(top: Radius.circular(24)),
28 ■       ),
29 ■       builder: (_) => UserEditorSheet(role: role),
30 ■     );
31 ■
32 ■   @override
33 ■   ConsumerState<UserEditorSheet> createState() => _UserEditorSheetState();
34 ■ }
35 ■
36 ■ class _UserEditorSheetState extends ConsumerState<UserEditorSheet> {
37 ■   final GlobalKey<FormState> _formKey = GlobalKey<FormState>();
38 ■   final TextEditingController _name = TextEditingController();
39 ■   final TextEditingController _email = TextEditingController();
40 ■   final TextEditingController _password = TextEditingController();
41 ■   final TextEditingController _classSection = TextEditingController();
42 ■   final TextEditingController _phone = TextEditingController();
43 ■   String? _busId;
44 ■   String? _stopId;
45 ■   bool _saving = false;
46 ■
47 ■   bool get isStudent => widget.role == 'student';
48 ■
49 ■   @override
50 ■   void dispose() {
51 ■     _name.dispose();
52 ■     _email.dispose();
53 ■     _password.dispose();
54 ■     _classSection.dispose();
55 ■     _phone.dispose();
56 ■     super.dispose();
57 ■   }
58 ■

```

```

59 ■ Future<void> _save() async {
60 ■   if (!(_formKey.currentState?.validate() ?? false)) return;
61 ■   setState(() => _saving = true);
62 ■   final Result<String> result =
63 ■     await ref.read(adminFunctionsServiceProvider).provisionUser(
64 ■       email: _email.text.trim(),
65 ■       fullName: _name.text.trim(),
66 ■       role: widget.role,
67 ■       temporaryPassword: _password.text,
68 ■       busId: _busId,
69 ■       stopId: isStudent ? _stopId : null,
70 ■       classSection:
71 ■         isStudent && _classSection.text.trim().isNotEmpty
72 ■           ? _classSection.text.trim()
73 ■           : null,
74 ■       phone: !isStudent && _phone.text.trim().isNotEmpty
75 ■         ? _phone.text.trim()
76 ■         : null,
77 ■     );
78 ■   if (!mounted) return;
79 ■   setState(() => _saving = false);
80 ■   result.when(
81 ■     ok: (_) {
82 ■       Navigator.of(context).pop();
83 ■       showNmbSnack(
84 ■         context,
85 ■         '${isStudent ? 'Student' : 'Driver'} account created.',
86 ■         isSuccess: true,
87 ■       );
88 ■     },
89 ■     err: (AppFailure f) => showNmbSnack(context, f.message, isError: true),
90 ■   );
91 ■ }
92 ■
93 ■ @override
94 ■ Widget build(BuildContext context) {
95 ■   final List<Bus> buses =
96 ■     ref.watch(allBusesProvider).valueOrNull ?? const <Bus>[];
97 ■   final List<BusStop> stops = _busId == null
98 ■     ? const <BusStop>[]
99 ■     : ref.watch(stopsOfBusProvider(_busId!)).valueOrNull ??
100 ■       const <BusStop>[];
101 ■
102 ■   return Padding(
103 ■     padding: EdgeInsets.only(
104 ■       bottom: MediaQuery.of(context).viewInsets.bottom,
105 ■     ),
106 ■     child: SafeArea(
107 ■       child: SingleChildScrollView(
108 ■         padding: const EdgeInsets.all(20),
109 ■         child: Form(
110 ■           key: _formKey,
111 ■           child: Column(
112 ■             mainAxisAlignment: MainAxisAlignment.min,
113 ■             crossAxisAlignment: CrossAxisAlignment.stretch,
114 ■             children: <Widget>[
115 ■               Text(
116 ■                 isStudent ? 'Add student' : 'Add driver',

```

```

117 ■         style: NmbTypography.screenTitle,
118 ■     ),
119 ■     const SizedBox(height: 16),
120 ■     TextFormField(
121 ■         controller: _name,
122 ■         decoration:
123 ■             const InputDecoration(hintText: 'Full name'),
124 ■         validator: (String? v) =>
125 ■             Validators.requiredField(v, label: 'Name'),
126 ■     ),
127 ■     const SizedBox(height: 12),
128 ■     TextFormField(
129 ■         controller: _email,
130 ■         keyboardType: TextInputType.emailAddress,
131 ■         decoration:
132 ■             const InputDecoration(hintText: 'School email'),
133 ■         validator: Validators.email,
134 ■     ),
135 ■     const SizedBox(height: 12),
136 ■     TextFormField(
137 ■         controller: _password,
138 ■         decoration: const InputDecoration(
139 ■             hintText: 'Temporary password',
140 ■             helperText:
141 ■                 'User should change it after first sign-in.',
142 ■         ),
143 ■         validator: Validators.strongPassword,
144 ■     ),
145 ■     const SizedBox(height: 12),
146 ■     if (isStudent) ...<Widget>[
147 ■         TextFormField(
148 ■             controller: _classSection,
149 ■             decoration: const InputDecoration(
150 ■                 hintText: 'Class & section (e.g. 7-B)',
151 ■             ),
152 ■             const SizedBox(height: 12),
153 ■         ] else ...<Widget>[
154 ■             TextFormField(
155 ■                 controller: _phone,
156 ■                 keyboardType: TextInputType.phone,
157 ■                 decoration:
158 ■                     const InputDecoration(hintText: 'Phone number'),
159 ■             ),
160 ■             const SizedBox(height: 12),
161 ■         ],
162 ■     DropdownButtonFormField<String>(
163 ■         value: _busId,
164 ■         decoration:
165 ■             const InputDecoration(hintText: 'Assign to bus'),
166 ■         items: <DropdownMenuItem<String>>[
167 ■             const DropdownMenuItem<String>(
168 ■                 value: null,
169 ■                 child: Text('No bus (assign later)'),
170 ■             ),
171 ■             for (final Bus b in buses)
172 ■                 DropdownMenuItem<String>(
173 ■                     value: b.id,
174 ■                     child: Text('${b.busNumber} – ${b.plateNumber}'),

```

```

175 ■         ),
176 ■     ],
177 ■     onChanged: (String? v) => setState(() {
178 ■         _busId = v;
179 ■         _stopId = null;
180 ■     })),
181 ■ ),
182 ■ if (isStudent && _busId != null) ...<Widget>[
183 ■     const SizedBox(height: 12),
184 ■     DropdownButtonFormField<String>(
185 ■         value: _stopId,
186 ■         decoration: const InputDecoration(
187 ■             hintText: 'Assigned bus stop',),
188 ■         items: <DropdownMenuItem<String>>[
189 ■             const DropdownMenuItem<String>(
190 ■                 value: null,
191 ■                 child: Text('No stop (assign later)'),
192 ■             ),
193 ■             for (final BusStop s in stops)
194 ■                 DropdownMenuItem<String>(
195 ■                     value: s.id,
196 ■                     child: Text(s.name),
197 ■                 ),
198 ■         ],
199 ■         onChanged: (String? v) => setState(() => _stopId = v),
200 ■     ),
201 ■ ],
202 ■ const SizedBox(height: 18),
203 ■ FilledButton(
204 ■     onPressed: _saving ? null : _save,
205 ■     child: _saving
206 ■         ? const SizedBox(
207 ■             width: 22,
208 ■             height: 22,
209 ■             child: CircularProgressIndicator(
210 ■                 strokeWidth: 2.4,
211 ■                 color: Colors.white,
212 ■             ),
213 ■         )
214 ■         : const Text('Create account'),
215 ■ ),
216 ■ const SizedBox(height: 8),
217 ■ ],
218 ■ ),
219 ■ ),
220 ■ ),
221 ■ ),
222 ■ );
223 ■ }
224 ■ }
225 ■
226 ■ /// Bottom sheet to reassign an existing user's bus/stop or toggle access.
227 ■ /// All changes go through admin-only Cloud Functions (claims + audit log).
228 ■ class UserManageSheet extends ConsumerStatefulWidget {
229 ■     const UserManageSheet({super.key, required this.user});
230 ■
231 ■     final AppUser user;
232 ■

```

```

233 ■ static Future<void> show(BuildContext context, AppUser user) =>
234 ■   showModalBottomSheet<void>(<
235 ■     context: context,
236 ■     isScrollControlled: true,
237 ■     shape: const RoundedRectangleBorder(
238 ■       borderRadius: BorderRadius.vertical(top: Radius.circular(24)),
239 ■     ),
240 ■     builder: (_) => UserManageSheet(user: user),
241 ■   );
242 ■
243 ■ @override
244 ■ ConsumerState<UserManageSheet> createState() => _UserManageSheetState();
245 ■ }
246 ■
247 ■ class _UserManageSheetState extends ConsumerState<UserManageSheet> {
248 ■   late String? _busId = widget.user.busId;
249 ■   late String? _stopId = widget.user.stopId;
250 ■   bool _saving = false;
251 ■
252 ■   bool get isStudent => widget.user.role.name == 'student';
253 ■
254 ■   Future<void> _saveAssignment() async {
255 ■     setState(() => _saving = true);
256 ■     final Result<void> result = isStudent
257 ■       ? await ref.read(adminFunctionsServiceProvider).assignUserToBus(
258 ■         uid: widget.user.uid,
259 ■         busId: _busId,
260 ■         stopId: _stopId,
261 ■       )
262 ■       : await ref.read(adminFunctionsServiceProvider).assignDriverToBus(
263 ■         driverUid: widget.user.uid,
264 ■         busId: _busId,
265 ■       );
266 ■     if (!mounted) return;
267 ■     setState(() => _saving = false);
268 ■     result.when(
269 ■       ok: (_) {
270 ■         Navigator.of(context).pop();
271 ■         showNmbSnack(context, 'Assignment updated.', isSuccess: true);
272 ■       },
273 ■       err: (AppFailure f) => showNmbSnack(context, f.message, isError: true),
274 ■     );
275 ■   }
276 ■
277 ■   Future<void> _toggleActive() async {
278 ■     final bool disable = widget.user.isActive;
279 ■     final bool confirmed = await showNmbConfirmDialog(
280 ■       context,
281 ■       title: disable ? 'Disable account?' : 'Enable account?',
282 ■       message: disable
283 ■         ? '${widget.user.fullName} will be signed out and blocked from '
284 ■           'the app until re-enabled.'
285 ■         : '${widget.user.fullName} will regain access to the app.',
286 ■       confirmLabel: disable ? 'Disable' : 'Enable',
287 ■       destructive: disable,
288 ■       icon: disable ? Icons.block_rounded : Icons.check_circle_outline,
289 ■     );
290 ■     if (!confirmed || !mounted) return;

```

```

291 ■     setState(() => _saving = true);
292 ■     final Result<void> result =
293 ■         await ref.read(adminFunctionsServiceProvider).setAccountActive(
294 ■             uid: widget.user.uid,
295 ■             active: !widget.user.isActive,
296 ■         );
297 ■     if (!mounted) return;
298 ■     setState(() => _saving = false);
299 ■     result.when(
300 ■         ok: (_) {
301 ■             Navigator.of(context).pop();
302 ■             showNmbSnack(context, 'Account updated.', isSuccess: true);
303 ■         },
304 ■         err: (AppFailure f) => showNmbSnack(context, f.message, isError: true),
305 ■     );
306 ■ }
307 ■
308 ■ @override
309 ■ Widget build(BuildContext context) {
310 ■     final List<Bus> buses =
311 ■         ref.watch(allBusesProvider).valueOrNull ?? const <Bus>[];
312 ■     final List<BusStop> stops = _busId == null
313 ■         ? const <BusStop>[]
314 ■         : ref.watch(stopsOfBusProvider(_busId!)).valueOrNull ??
315 ■             const <BusStop>[];
316 ■
317 ■     return SafeArea(
318 ■         child: SingleChildScrollView(
319 ■             padding: const EdgeInsets.all(20),
320 ■             child: Column(
321 ■                 mainAxisAlignment: MainAxisAlignment.min,
322 ■                 crossAxisAlignment: CrossAxisAlignment.stretch,
323 ■                 children: <Widget>[
324 ■                     Text(widget.user.fullName, style: NmbTypography.screenTitle),
325 ■                     Text(widget.user.email, style: NmbTypography.bodySecondary),
326 ■                     const SizedBox(height: 18),
327 ■                     DropdownButtonFormField<String>(
328 ■                         value: _busId,
329 ■                         decoration: const InputDecoration(hintText: 'Assigned bus'),
330 ■                         items: <DropdownMenuItem<String>>[
331 ■                             const DropdownMenuItem<String>(
332 ■                                 value: null,
333 ■                                 child: Text('No bus'),
334 ■                             ),
335 ■                             for (final Bus b in buses)
336 ■                                 DropdownMenuItem<String>(
337 ■                                     value: b.id,
338 ■                                     child: Text('${b.busNumber} – ${b.plateNumber}'),
339 ■                                 ),
340 ■                         ],
341 ■                         onChanged: (String? v) => setState(() {
342 ■                             _busId = v;
343 ■                             _stopId = null;
344 ■                         })),
345 ■                 ],
346 ■                 if (isStudent && _busId != null) ...<Widget>[
347 ■                     const SizedBox(height: 12),
348 ■                     DropdownButtonFormField<String>(

```

```

349 ■         value: stops.any((BusStop s) => s.id == _stopId)
350 ■         ? _stopId
351 ■         : null,
352 ■     decoration:
353 ■         const InputDecoration(hintText: 'Assigned stop'),
354 ■     items: <DropDownMenuItem<String>>[
355 ■         const DropDownMenuItem<String>{
356 ■             value: null,
357 ■             child: Text('No stop'),
358 ■         },
359 ■         for (final BusStop s in stops)
360 ■             DropDownMenuItem<String>{
361 ■                 value: s.id,
362 ■                 child: Text(s.name),
363 ■             },
364 ■     ],
365 ■     onChanged: (String? v) => setState(() => _stopId = v),
366 ■ ),
367 ■ ],
368 ■ const SizedBox(height: 18),
369 ■ FilledButton(
370 ■     onPressed: _saving ? null : _saveAssignment,
371 ■     child: const Text('Save assignment'),
372 ■ ),
373 ■ const SizedBox(height: 10),
374 ■ OutlinedButton.icon(
375 ■     onPressed: _saving ? null : _toggleActive,
376 ■     icon: Icon(
377 ■         widget.user.isActive
378 ■         ? Icons.block_rounded
379 ■         : Icons.check_circle_outline_rounded,
380 ■     ),
381 ■     label: Text(
382 ■         widget.user.isActive
383 ■         ? 'Disable account'
384 ■         : 'Enable account',
385 ■     ),
386 ■ ),
387 ■ const SizedBox(height: 8),
388 ■ ],
389 ■ ),
390 ■ ),
391 ■ );
392 ■ }
393 ■ }

```

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/theme/nmb_colors.dart';
5 ■ import '../core/theme/nmb_typography.dart';
6 ■ import '../core/widgets/nmb_card.dart';
7 ■ import '../core/widgets/responsive_scaffold_body.dart';
8 ■ import '../core/widgets/state_views.dart';
9 ■ import '../core/widgets/status_pill.dart';
10 ■ import '../models/app_user.dart';
11 ■ import '../models/bus.dart';
12 ■ import '../providers/data_providers.dart';
13 ■ import '../widgets/user_editor_sheet.dart';
14 ■
15 ■ /// Manage students: list, search, add, and per-student detail sheet
16 ■ /// (assign bus/stop, enable/disable). All mutations go via Cloud Functions.
17 ■ class AdminStudentsScreen extends ConsumerStatefulWidget {
18 ■   const AdminStudentsScreen({super.key});
19 ■
20 ■   @override
21 ■   ConsumerState<AdminStudentsScreen> createState() =>
22 ■     _AdminStudentsScreenState();
23 ■ }
24 ■
25 ■ class _AdminStudentsScreenState extends ConsumerState<AdminStudentsScreen> {
26 ■   String _query = '';
27 ■
28 ■   @override
29 ■   Widget build(BuildContext context) {
30 ■     final AsyncValue<List<AppUser>> studentsAsync =
31 ■       ref.watch(allStudentsProvider);
32 ■     final List<Bus> buses =
33 ■       ref.watch(allBusesProvider).valueOrNull ?? const <Bus>[];
34 ■
35 ■     return Scaffold(
36 ■       appBar: AppBar(title: const Text('Students')),
37 ■       floatingActionButton: FloatingActionButton.extended(
38 ■         onPressed: () => UserEditorSheet.show(context, role: 'student'),
39 ■         icon: const Icon(Icons.person_add_alt_1_rounded),
40 ■         label: const Text('Add student'),
41 ■       ),
42 ■       body: studentsAsync.when(
43 ■         loading: () => const LoadingView(),
44 ■         error: (Object e, _) => ErrorView(
45 ■           message: 'Couldn\'t load students.',
46 ■           onRetry: () => ref.invalidate(allStudentsProvider),
47 ■         ),
48 ■         data: (List<AppUser> students) {
49 ■           final List<AppUser> filtered = _query.isEmpty
50 ■             ? students
51 ■             : students
52 ■               .where((AppUser s) =>
53 ■                 s.fullName
54 ■                   .toLowerCase()
55 ■                     .contains(_query.toLowerCase()) ||
56 ■                     s.email.toLowerCase().contains(_query.toLowerCase()),)
57 ■               .toList();
58 ■

```

```

59 ■         return ResponsiveBody(
60 ■             child: Column(
61 ■                 crossAxisAlignment: CrossAxisAlignment.stretch,
62 ■                 children: <Widget>[
63 ■                     TextField(
64 ■                         decoration: const InputDecoration(
65 ■                             hintText: 'Search by name or email...',
66 ■                             prefixIcon: Icon(Icons.search_rounded),
67 ■                         ),
68 ■                         onChanged: (String v) => setState(() => _query = v),
69 ■                     ),
70 ■                     const SizedBox(height: 14),
71 ■                     if (filtered.isEmpty)
72 ■                         const Padding(
73 ■                             padding: EdgeInsets.only(top: 60),
74 ■                             child: EmptyState(
75 ■                                 icon: Icons.school_outlined,
76 ■                                 title: 'No students found',
77 ■                                 message:
78 ■                                     'Add students with the button below, or adjust '
79 ■                                     'your search.',
80 ■                             ),
81 ■                     )
82 ■                 ],
83 ■                 for (final AppUser s in filtered) ...<Widget>[
84 ■                     _StudentTile(student: s, buses: buses),
85 ■                     const SizedBox(height: 10),
86 ■                 ],
87 ■                 const SizedBox(height: 90),
88 ■             ],
89 ■         ),
90 ■     );
91 ■ },
92 ■ ),
93 ■ );
94 ■ }
95 ■ }
96 ■
97 ■ class _StudentTile extends StatelessWidget {
98 ■     const _StudentTile({required this.student, required this.buses});
99 ■
100 ■     final AppUser student;
101 ■     final List<Bus> buses;
102 ■
103 ■     @override
104 ■     Widget build(BuildContext context) {
105 ■         String busLabel = 'No bus';
106 ■         for (final Bus b in buses) {
107 ■             if (b.id == student.busId) busLabel = b.busNumber;
108 ■         }
109 ■         return NmbCard(
110 ■             onTap: () => UserManageSheet.show(context, student),
111 ■             child: Row(
112 ■                 children: <Widget>[
113 ■                     CircleAvatar(
114 ■                         backgroundColor: NmbColors.primarySoft,
115 ■                         child: Text(
116 ■                             student.firstName.isEmpty

```

```

117 ■         ? student.firstName[0].toUpperCase()
118 ■         : '?',
119 ■         style: NmbTypography.cardTitle
120 ■         .copyWith(color: NmbColors.primary),
121 ■     ),
122 ■ ),
123 ■     const SizedBox(width: 12),
124 ■     Expanded(
125 ■         child: Column(
126 ■             crossAxisAlignment: CrossAxisAlignment.start,
127 ■             children: <Widget>[
128 ■                 Text(student.fullName, style: NmbTypography.cardTitle),
129 ■                 Text(
130 ■                     <String>[
131 ■                         if (student.classSection != null)
132 ■                         'Class ${student.classSection}',
133 ■                         busLabel,
134 ■                     ].join(' • '),
135 ■                     style: NmbTypography.bodySecondary,
136 ■                 ),
137 ■             ],
138 ■         ),
139 ■     ),
140 ■     if (!student.isActive)
141 ■         const StatusPill(
142 ■             label: 'DISABLED',
143 ■             color: NmbColors.danger,
144 ■             background: NmbColors.dangerSoft,
145 ■         ),
146 ■     const Icon(Icons.chevron_right_rounded,
147 ■         color: NmbColors.textTertiary, ),
148 ■ ],
149 ■ ),
150 ■ );
151 ■ }
152 ■ }

```

lib/features/admin/screens/admin_drivers_screen.dart

108 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/theme/nmb_colors.dart';
5 ■ import '../core/theme/nmb_typography.dart';
6 ■ import '../core/widgets/nmb_card.dart';
7 ■ import '../core/widgets/responsive_scaffold_body.dart';
8 ■ import '../core/widgets/state_views.dart';
9 ■ import '../core/widgets/status_pill.dart';
10 ■ import '../models/app_user.dart';
11 ■ import '../models/bus.dart';
12 ■ import '../providers/data_providers.dart';
13 ■ import '../widgets/user_editor_sheet.dart';
14 ■
15 ■ /// Manage drivers: list, add, assign to bus, enable/disable.
16 ■ class AdminDriversScreen extends ConsumerWidget {
17 ■   const AdminDriversScreen({super.key});
18 ■
19 ■   @override
20 ■   Widget build(BuildContext context, WidgetRef ref) {
21 ■     final AsyncValue<List<AppUser>> driversAsync =
22 ■       ref.watch(allDriversProvider);
23 ■     final List<Bus> buses =
24 ■       ref.watch(allBusesProvider).valueOrNull ?? const <Bus>[];
25 ■
26 ■     return Scaffold(
27 ■       appBar: AppBar(title: const Text('Drivers')),
28 ■       floatingActionButton: FloatingActionButton.extended(
29 ■         onPressed: () => UserEditorSheet.show(context, role: 'driver'),
30 ■         icon: const Icon(Icons.person_add_alt_1_rounded),
31 ■         label: const Text('Add driver'),
32 ■       ),
33 ■       body: driversAsync.when(
34 ■         loading: () => const LoadingView(),
35 ■         error: (Object e, _) => ErrorView(
36 ■           message: 'Couldn\'t load drivers.',
37 ■           onRetry: () => ref.invalidate(allDriversProvider),
38 ■         ),
39 ■         data: (List<AppUser> drivers) {
40 ■           if (drivers.isEmpty) {
41 ■             return const EmptyState(
42 ■               icon: Icons.badge_outlined,
43 ■               title: 'No drivers yet',
44 ■               message: 'Add your first driver with the button below.',
45 ■             );
46 ■           }
47 ■           return ResponsiveBody(
48 ■             child: Column(
49 ■               crossAxisAlignment: CrossAxisAlignment.stretch,
50 ■               children: <Widget>[
51 ■                 for (final AppUser d in drivers) ...<Widget>[
52 ■                   _DriverTile(driver: d, buses: buses),
53 ■                   const SizedBox(height: 10),
54 ■                 ],
55 ■                 const SizedBox(height: 90),
56 ■               ],
57 ■             ),
58 ■           );

```

```

59 ■     },
60 ■   ),
61 ■ );
62 ■ }
63 ■ }
64 ■
65 ■ class _DriverTile extends StatelessWidget {
66 ■   const _DriverTile({required this.driver, required this.buses});
67 ■
68 ■   final AppUser driver;
69 ■   final List<Bus> buses;
70 ■
71 ■   @override
72 ■   Widget build(BuildContext context) {
73 ■     String busLabel = 'No bus assigned';
74 ■     for (final Bus b in buses) {
75 ■       if (b.id == driver.busId) busLabel = b.busNumber;
76 ■     }
77 ■     return NmbCard(
78 ■       onTap: () => UserManageSheet.show(context, driver),
79 ■       child: Row(
80 ■         children: <Widget>[
81 ■           const CircleAvatar(
82 ■             backgroundColor: NmbColors.accentSoft,
83 ■             child:
84 ■               Icon(Icons.badge_rounded, color: NmbColors.accentDark),
85 ■           ),
86 ■           const SizedBox(width: 12),
87 ■           Expanded(
88 ■             child: Column(
89 ■               crossAxisAlignment: CrossAxisAlignment.start,
90 ■               children: <Widget>[
91 ■                 Text(driver.fullName, style: NmbTypography.cardTitle),
92 ■                 Text(busLabel, style: NmbTypography.bodySecondary),
93 ■               ],
94 ■             ),
95 ■           ),
96 ■           if (!driver.isActive)
97 ■             const StatusPill(
98 ■               label: 'DISABLED',
99 ■               color: NmbColors.danger,
100 ■              background: NmbColors.dangerSoft,
101 ■            ),
102 ■           const Icon(Icons.chevron_right_rounded,
103 ■             color: NmbColors.textTertiary, ),
104 ■         ],
105 ■       ),
106 ■     );
107 ■   }
108 ■ }

```

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/theme/nmb_colors.dart';
5 ■ import '../core/theme/nmb_typography.dart';
6 ■ import '../core/utils/validators.dart';
7 ■ import '../core/widgets/nmb_card.dart';
8 ■ import '../core/widgets/nmb_dialogs.dart';
9 ■ import '../core/widgets/responsive_scaffold_body.dart';
10 ■ import '../core/widgets/state_views.dart';
11 ■ import '../models/app_user.dart';
12 ■ import '../models/bus.dart';
13 ■ import '../providers/app_providers.dart';
14 ■ import '../providers/data_providers.dart';
15 ■
16 ■ /// Manage buses: create/edit fleet. Structure: each bus gets a driver, a
17 ■ /// route, stops and students (assigned from their own screens).
18 ■ class AdminBusesScreen extends ConsumerWidget {
19 ■   const AdminBusesScreen({super.key});
20 ■
21 ■   @override
22 ■   Widget build(BuildContext context, WidgetRef ref) {
23 ■     final AsyncValue<List<Bus>> busesAsync = ref.watch(allBusesProvider);
24 ■     final List<AppUser> drivers =
25 ■       ref.watch(allDriversProvider).valueOrNull ?? const <AppUser>[];
26 ■     final List<AppUser> students =
27 ■       ref.watch(allStudentsProvider).valueOrNull ?? const <AppUser>[];
28 ■
29 ■     return Scaffold(
30 ■       appBar: AppBar(title: const Text('Buses')),
31 ■       floatingActionButton: FloatingActionButton.extended(
32 ■         onPressed: () => _BusEditorSheet.show(context),
33 ■         icon: const Icon(Icons.add_rounded),
34 ■         label: const Text('Add bus'),
35 ■       ),
36 ■       body: busesAsync.when(
37 ■         loading: () => const LoadingView(),
38 ■         error: (Object e, _) => ErrorView(
39 ■           message: 'Couldn\'t load buses.',
40 ■           onRetry: () => ref.invalidate(allBusesProvider),
41 ■         ),
42 ■         data: (List<Bus> buses) {
43 ■           if (buses.isEmpty) {
44 ■             return const EmptyState(
45 ■               icon: Icons.directions_bus_outlined,
46 ■               title: 'No buses yet',
47 ■               message: 'Create your first bus to start building the '
48 ■                 'transport structure.',
49 ■             );
50 ■           }
51 ■           return ResponsiveBody(
52 ■             child: Column(
53 ■               crossAxisAlignment: CrossAxisAlignment.stretch,
54 ■               children: <Widget>[
55 ■                 for (final Bus bus in buses) ...<Widget>[
56 ■                   _BusTile(
57 ■                     bus: bus,
58 ■                     driverName: _driverName(bus, drivers),

```

```

59 ■         studentCount: students
60 ■         .where((AppUser s) => s.busId == bus.id)
61 ■         .length,
62 ■     ),
63 ■     const SizedBox(height: 10),
64 ■ ],
65 ■     const SizedBox(height: 90),
66 ■ ],
67 ■ ),
68 ■ );
69 ■ },
70 ■ ),
71 ■ );
72 ■ }
73 ■
74 ■ String _driverName(Bus bus, List<AppUser> drivers) {
75 ■   for (final AppUser d in drivers) {
76 ■     if (d.busId == bus.id) return d.fullName;
77 ■   }
78 ■   return 'No driver';
79 ■ }
80 ■ }
81 ■
82 ■ class _BusTile extends ConsumerWidget {
83 ■   const _BusTile({
84 ■     required this.bus,
85 ■     required this.driverName,
86 ■     required this.studentCount,
87 ■   });
88 ■
89 ■   final Bus bus;
90 ■   final String driverName;
91 ■   final int studentCount;
92 ■
93 ■   @override
94 ■   Widget build(BuildContext context, WidgetRef ref) {
95 ■     return NmbCard(
96 ■       onTap: () => _BusEditorSheet.show(context, bus: bus),
97 ■       child: Row(
98 ■         children: <Widget>[
99 ■           Container(
100 ■             width: 48,
101 ■             height: 48,
102 ■             decoration: BoxDecoration(
103 ■               color: bus.isActive
104 ■                 ? NmbColors.accentSoft
105 ■                 : NmbColors.divider,
106 ■               borderRadius: BorderRadius.circular(14),
107 ■             ),
108 ■             child: Icon(
109 ■               Icons.directions_bus_rounded,
110 ■               color: bus.isActive
111 ■                 ? NmbColors.accentDark
112 ■                 : NmbColors.textTertiary,
113 ■             ),
114 ■           ),
115 ■           const SizedBox(width: 12),
116 ■           Expanded(

```

```

117 ■         child: Column(
118 ■             crossAxisAlignment: CrossAxisAlignment.start,
119 ■             children: <Widget>[
120 ■                 Text('${bus.busNumber} • ${bus.plateNumber}',
121 ■                     style: NmbTypography.cardTitle,),
122 ■                 Text(
123 ■                     '$driverName • $studentCount students • '
124 ■                     '${bus.capacity} seats',
125 ■                     style: NmbTypography.bodySecondary,
126 ■                 ),
127 ■             ],
128 ■         ),
129 ■     ),
130 ■     const Icon(Icons.chevron_right_rounded,
131 ■         color: NmbColors.textTertiary,),
132 ■ ],
133 ■ ),
134 ■ );
135 ■ }
136 ■ }
137 ■
138 ■ class _BusEditorSheet extends ConsumerStatefulWidget {
139 ■     const _BusEditorSheet({this.bus});
140 ■
141 ■     final Bus? bus;
142 ■
143 ■     static Future<void> show(BuildContext context, {Bus? bus}) =>
144 ■         showModalBottomSheet<void>(
145 ■             context: context,
146 ■             isScrollControlled: true,
147 ■             shape: const RoundedRectangleBorder(
148 ■                 borderRadius: BorderRadius.vertical(top: Radius.circular(24)),
149 ■             ),
150 ■             builder: (_) => _BusEditorSheet(bus: bus),
151 ■         );
152 ■
153 ■     @override
154 ■     ConsumerState<_BusEditorSheet> createState() => _BusEditorSheetState();
155 ■ }
156 ■
157 ■ class _BusEditorSheetState extends ConsumerState<_BusEditorSheet> {
158 ■     final GlobalKey<FormState> _formKey = GlobalKey<FormState>();
159 ■     late final TextEditingController _number =
160 ■         TextEditingController(text: widget.bus?.busNumber);
161 ■     late final TextEditingController _plate =
162 ■         TextEditingController(text: widget.bus?.plateNumber);
163 ■     late final TextEditingController _capacity = TextEditingController(
164 ■         text: widget.bus == null ? '' : '${widget.bus!.capacity}',);
165 ■     late bool _isActive = widget.bus?.isActive ?? true;
166 ■     bool _saving = false;
167 ■
168 ■     @override
169 ■     void dispose() {
170 ■         _number.dispose();
171 ■         _plate.dispose();
172 ■         _capacity.dispose();
173 ■         super.dispose();
174 ■     }

```

```

175 ■
176 ■ Future<void> _save() async {
177 ■   if (!(_formKey.currentState?.validate() ?? false)) return;
178 ■   setState(() => _saving = true);
179 ■   try {
180 ■     final Map<String, dynamic> data = <String, dynamic>{
181 ■       'busNumber': _number.text.trim(),
182 ■       'plateNumber': _plate.text.trim(),
183 ■       'capacity': int.tryParse(_capacity.text.trim()) ?? 0,
184 ■       'isActive': _isActive,
185 ■     };
186 ■     if (widget.bus == null) {
187 ■       await ref.read(firestoreServiceProvider).createBus(
188 ■         Bus(
189 ■           id: '',
190 ■           busNumber: data['busNumber'] as String,
191 ■           plateNumber: data['plateNumber'] as String,
192 ■           capacity: data['capacity'] as int,
193 ■           isActive: _isActive,
194 ■         ),
195 ■       );
196 ■     } else {
197 ■       await ref
198 ■         .read(firestoreServiceProvider)
199 ■         .updateBus(widget.bus!.id, data);
200 ■     }
201 ■     if (!mounted) return;
202 ■     Navigator.of(context).pop();
203 ■     showNmbSnack(context, 'Bus saved.', isSuccess: true);
204 ■   } catch (_) {
205 ■     if (!mounted) return;
206 ■     showNmbSnack(context, 'Couldn\'t save the bus. Try again.',
207 ■       isError: true,);
208 ■   } finally {
209 ■     if (mounted) setState(() => _saving = false);
210 ■   }
211 ■ }
212 ■
213 ■ @override
214 ■ Widget build(BuildContext context) {
215 ■   return Padding(
216 ■     padding:
217 ■       EdgeInsets.only(bottom: MediaQuery.of(context).viewInsets.bottom),
218 ■     child: SafeArea(
219 ■       child: SingleChildScrollView(
220 ■         padding: const EdgeInsets.all(20),
221 ■         child: Form(
222 ■           key: _formKey,
223 ■           child: Column(
224 ■             mainAxisAlignment: MainAxisAlignment.min,
225 ■             crossAxisAlignment: CrossAxisAlignment.stretch,
226 ■             children: <Widget>[
227 ■               Text(
228 ■                 widget.bus == null ? 'Add bus' : 'Edit bus',
229 ■                 style: NmbTypography.screenTitle,
230 ■               ),
231 ■               const SizedBox(height: 16),
232 ■               TextFormField(

```

```

233 ■         controller: _number,
234 ■         decoration: const InputDecoration(
235 ■             hintText: 'Bus number (e.g. Bus 3)',),
236 ■         validator: (String? v) =>
237 ■             Validators.requiredField(v, label: 'Bus number'),
238 ■     ),
239 ■     const SizedBox(height: 12),
240 ■     TextFormField(
241 ■         controller: _plate,
242 ■         decoration: const InputDecoration(
243 ■             hintText: 'Plate number (e.g. UK07 PA 1234)',),
244 ■         validator: (String? v) =>
245 ■             Validators.requiredField(v, label: 'Plate number'),
246 ■     ),
247 ■     const SizedBox(height: 12),
248 ■     TextFormField(
249 ■         controller: _capacity,
250 ■         keyboardType: TextInputType.number,
251 ■         decoration:
252 ■             const InputDecoration(hintText: 'Seat capacity'),
253 ■     ),
254 ■     SwitchListTile(
255 ■         contentPadding: EdgeInsets.zero,
256 ■         title: const Text('Bus is active'),
257 ■         value: _isActive,
258 ■         onChanged: (bool v) => setState(() => _isActive = v),
259 ■     ),
260 ■     const SizedBox(height: 10),
261 ■     FilledButton(
262 ■         onPressed: _saving ? null : _save,
263 ■         child: Text(widget.bus == null ? 'Create bus' : 'Save'),
264 ■     ),
265 ■     const SizedBox(height: 8),
266 ■ ],
267 ■ ),
268 ■ ),
269 ■ ),
270 ■ ),
271 ■ );
272 ■ }
273 ■ }

```

lib/features/admin/screens/admin_routes_stops_screen.dart

323 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/theme/nmb_colors.dart';
5 ■ import '../core/theme/nmb_typography.dart';
6 ■ import '../core/utils/validators.dart';
7 ■ import '../core/widgets/nmb_card.dart';
8 ■ import '../core/widgets/nmb_dialogs.dart';
9 ■ import '../core/widgets/responsive_scaffold_body.dart';
10 ■ import '../core/widgets/state_views.dart';
11 ■ import '../models/bus.dart';
12 ■ import '../models/bus_stop.dart';
13 ■ import '../models/geo_point_data.dart';
14 ■ import '../providers/app_providers.dart';
15 ■ import '../providers/data_providers.dart';
16 ■
17 ■ /// Routes & Stops manager: pick a bus, then build its ordered stop list.
18 ■ /// School → Stop 1 → Stop 2 → ... exactly as the transport structure defines.
19 ■ class AdminRoutesStopsScreen extends ConsumerStatefulWidget {
20 ■   const AdminRoutesStopsScreen({super.key});
21 ■
22 ■   @override
23 ■   ConsumerState<AdminRoutesStopsScreen> createState() =>
24 ■     _AdminRoutesStopsScreenState();
25 ■ }
26 ■
27 ■ class _AdminRoutesStopsScreenState
28 ■   extends ConsumerState<AdminRoutesStopsScreen> {
29 ■   String? _busId;
30 ■
31 ■   @override
32 ■   Widget build(BuildContext context) {
33 ■     final List<Bus> buses =
34 ■       ref.watch(allBusesProvider).valueOrNull ?? const <Bus>[];
35 ■     final List<BusStop> stops = _busId == null
36 ■       ? const <BusStop>[]
37 ■       : ref.watch(stopsOfBusProvider(_busId!)).valueOrNull ??
38 ■         const <BusStop>[];
39 ■
40 ■     return Scaffold(
41 ■       appBar: AppBar(title: const Text('Routes & Stops')),
42 ■       floatingActionButton: _busId == null
43 ■         ? null
44 ■         : FloatingActionButton.extended(
45 ■           onPressed: () => _StopEditorSheet.show(
46 ■             context,
47 ■             busId: _busId!,
48 ■             nextOrder: stops.length + 1,
49 ■           ),
50 ■           icon: const Icon(Icons.add_location_alt_rounded),
51 ■           label: const Text('Add stop'),
52 ■         ),
53 ■       body: ResponsiveBody(
54 ■         child: Column(
55 ■           crossAxisAlignment: CrossAxisAlignment.stretch,
56 ■           children: <Widget>[
57 ■             DropdownButtonFormField<String>(
58 ■               value: _busId,

```

```

59 ■      decoration: const InputDecoration(
60 ■          hintText: 'Select a bus to manage its route',
61 ■          prefixIcon: Icon(Icons.directions_bus_rounded),
62 ■      ),
63 ■      items: <DropDownMenuItem<String>>[
64 ■          for (final Bus b in buses)
65 ■              DropDownMenuItem<String>(
66 ■                  value: b.id,
67 ■                  child: Text('${b.busNumber} – ${b.plateNumber}'),
68 ■              ),
69 ■      ],
70 ■      onChanged: (String? v) => setState(() => _busId = v),
71 ■  ),
72 ■  const SizedBox(height: 16),
73 ■  if (_busId == null)
74 ■      const Padding(
75 ■          padding: EdgeInsets.only(top: 40),
76 ■          child: EmptyState(
77 ■              icon: Icons.route_outlined,
78 ■              title: 'Pick a bus',
79 ■              message:
80 ■                  'Choose a bus above to view and edit its route and '
81 ■                  'stops in order.',
82 ■          ),
83 ■  )
84 ■  else if (stops.isEmpty)
85 ■      const Padding(
86 ■          padding: EdgeInsets.only(top: 40),
87 ■          child: EmptyState(
88 ■              icon: Icons.add_location_alt_outlined,
89 ■              title: 'No stops yet',
90 ■              message:
91 ■                  'Add the first stop for this route. Stops appear to '
92 ■                  'students in the order you create them.',
93 ■          ),
94 ■  )
95 ■  else ...<Widget>[
96 ■      const Text('Route stops (in order)',
97 ■          style: NmbTypography.sectionTitle,),
98 ■      const SizedBox(height: 10),
99 ■      for (int i = 0; i < stops.length; i++) ...<Widget>[
100 ■          NmbCard(
101 ■              onTap: () => _StopEditorSheet.show(
102 ■                  context,
103 ■                  busId: _busId!,
104 ■                  stop: stops[i],
105 ■                  nextOrder: stops[i].order,
106 ■              ),
107 ■              padding: const EdgeInsets.all(14),
108 ■              child: Row(
109 ■                  children: <Widget>[
110 ■                      Container(
111 ■                          width: 30,
112 ■                          height: 30,
113 ■                          decoration: const BoxDecoration(
114 ■                              color: NmbColors.primarySoft,
115 ■                              shape: BoxShape.circle,
116 ■                          ),

```

```

117 ■         child: Center(
118 ■           child: Text(
119 ■             '${i + 1}',
120 ■             style: NmbTypography.caption.copyWith(
121 ■               color: NmbColors.primaryDark,
122 ■               fontWeight: FontWeight.w800,
123 ■             ),
124 ■           ),
125 ■         ),
126 ■       ),
127 ■     const SizedBox(width: 12),
128 ■     Expanded(
129 ■       child: Column(
130 ■         crossAxisAlignment: CrossAxisAlignment.start,
131 ■         children: <Widget>[
132 ■           Text(stops[i].name,
133 ■             style: NmbTypography.cardTitle, ),
134 ■           Text(
135 ■             '${stops[i].location.lat.toStringAsFixed(5)}, '
136 ■             '${stops[i].location.lng.toStringAsFixed(5)}'
137 ■             '${stops[i].scheduledTime != null ? ' ' • ${stops[i].scheduledTime}'
138 ■           : ''}',
139 ■             style: NmbTypography.caption,
140 ■           ),
141 ■         ],
142 ■       ),
143 ■       const Icon(Icons.edit_outlined,
144 ■         size: 18, color: NmbColors.textTertiary, ),
145 ■     ],
146 ■   ),
147 ■ ),
148 ■   const SizedBox(height: 8),
149 ■ ],
150 ■ ],
151 ■ const SizedBox(height: 90),
152 ■ ],
153 ■ ),
154 ■ ),
155 ■ );
156 ■ }
157 ■ }
158 ■
159 ■ class _StopEditorSheet extends ConsumerStatefulWidget {
160 ■   const _StopEditorSheet({
161 ■     required this.busId,
162 ■     required this.nextOrder,
163 ■     this.stop,
164 ■   });
165 ■
166 ■   final String busId;
167 ■   final int nextOrder;
168 ■   final BusStop? stop;
169 ■
170 ■   static Future<void> show(
171 ■     BuildContext context, {
172 ■     required String busId,
173 ■     required int nextOrder,

```

```

174 ■     BusStop? stop,
175 ■   }) =>
176 ■     showModalBottomSheet<void>(
177 ■       context: context,
178 ■       isScrollControlled: true,
179 ■       shape: const RoundedRectangleBorder(
180 ■         borderRadius: BorderRadius.vertical(top: Radius.circular(24)),
181 ■       ),
182 ■       builder: (_) => _StopEditorSheet(
183 ■         busId: busId,
184 ■         nextOrder: nextOrder,
185 ■         stop: stop,
186 ■       ),
187 ■     );
188 ■
189 ■   @override
190 ■   ConsumerState<_StopEditorSheet> createState() => _StopEditorSheetState();
191 ■ }
192 ■
193 ■ class _StopEditorSheetState extends ConsumerState<_StopEditorSheet> {
194 ■   final GlobalKey<FormState> _formKey = GlobalKey<FormState>();
195 ■   late final TextEditingController _name =
196 ■     TextEditingController(text: widget.stop?.name);
197 ■   late final TextEditingController _lat = TextEditingController(
198 ■     text: widget.stop == null ? '' : '${widget.stop!.location.lat}',);
199 ■   late final TextEditingController _lng = TextEditingController(
200 ■     text: widget.stop == null ? '' : '${widget.stop!.location.lng}',);
201 ■   late final TextEditingController _time =
202 ■     TextEditingController(text: widget.stop?.scheduledTime);
203 ■   bool _saving = false;
204 ■
205 ■   @override
206 ■   void dispose() {
207 ■     _name.dispose();
208 ■     _lat.dispose();
209 ■     _lng.dispose();
210 ■     _time.dispose();
211 ■     super.dispose();
212 ■   }
213 ■
214 ■   Future<void> _save() async {
215 ■     if (!(_formKey.currentState?.validate() ?? false)) return;
216 ■     setState(() => _saving = true);
217 ■     try {
218 ■       final BusStop stop = BusStop(
219 ■         id: widget.stop?.id ?? '',
220 ■         name: _name.text.trim(),
221 ■         routeId: widget.stop?.routeId ?? '',
222 ■         busId: widget.busId,
223 ■         location: GeoPointData(
224 ■           lat: double.parse(_lat.text.trim()),
225 ■           lng: double.parse(_lng.text.trim()),
226 ■         ),
227 ■         order: widget.nextOrder,
228 ■         scheduledTime:
229 ■           _time.text.trim().isEmpty ? null : _time.text.trim(),
230 ■       );
231 ■       if (widget.stop == null) {

```

```

232 ■      await ref.read(firestoreServiceProvider).createStop(stop);
233 ■    } else {
234 ■      await ref
235 ■        .read(firestoreServiceProvider)
236 ■        .updateStop(widget.stop!.id, stop.toMap());
237 ■    }
238 ■    if (!mounted) return;
239 ■    Navigator.of(context).pop();
240 ■    showNmbSnack(context, 'Stop saved.', isSuccess: true);
241 ■  } catch (_) {
242 ■    if (!mounted) return;
243 ■    showNmbSnack(context, 'Couldn\'t save the stop.', isError: true);
244 ■  } finally {
245 ■    if (mounted) setState(() => _saving = false);
246 ■  }
247 ■ }
248 ■
249 ■ @override
250 ■ Widget build(BuildContext context) {
251 ■   return Padding(
252 ■     padding:
253 ■       EdgeInsets.only(bottom: MediaQuery.of(context).viewInsets.bottom),
254 ■     child: SafeArea(
255 ■       child: SingleChildScrollView(
256 ■         padding: const EdgeInsets.all(20),
257 ■         child: Form(
258 ■           key: _formKey,
259 ■           child: Column(
260 ■             mainAxisAlignment: MainAxisAlignment.min,
261 ■             crossAxisAlignment: CrossAxisAlignment.stretch,
262 ■             children: <Widget>[
263 ■               Text(
264 ■                 widget.stop == null ? 'Add stop' : 'Edit stop',
265 ■                 style: NmbTypography.screenTitle,
266 ■               ),
267 ■               const SizedBox(height: 16),
268 ■               TextFormField(
269 ■                 controller: _name,
270 ■                 decoration: const InputDecoration(
271 ■                   hintText: 'Stop name (e.g. Rajpur Road Gate 2)',
272 ■                   validator: (String? v) =>
273 ■                     Validators.requiredField(v, label: 'Stop name'),
274 ■                 ),
275 ■               const SizedBox(height: 12),
276 ■               Row(
277 ■                 children: <Widget>[
278 ■                   Expanded(
279 ■                     child: TextFormField(
280 ■                       controller: _lat,
281 ■                       keyboardType:
282 ■                         const TextInputType.numberWithOptions(
283 ■                           decimal: true, signed: true,
284 ■                         ),
285 ■                       decoration:
286 ■                         const InputDecoration(hintText: 'Latitude'),
287 ■                       validator: Validators.latitude,
288 ■                     ),
289 ■                   const SizedBox(width: 12),

```

```

290 ■         Expanded(
291 ■             child: TextFormField(
292 ■                 controller: _lng,
293 ■                 keyboardType:
294 ■                     const TextInputType.numberWithOptions(
295 ■                         decimal: true, signed: true,),
296 ■                 decoration:
297 ■                     const InputDecoration(hintText: 'Longitude'),
298 ■                 validator: Validators.longitude,
299 ■             ),
300 ■         ),
301 ■     ],
302 ■ ),
303 ■     const SizedBox(height: 12),
304 ■     TextFormField(
305 ■         controller: _time,
306 ■         decoration: const InputDecoration(
307 ■             hintText: 'Scheduled time (e.g. 07:15) – optional',),
308 ■     ),
309 ■     const SizedBox(height: 18),
310 ■     FilledButton(
311 ■         onPressed: _saving ? null : _save,
312 ■         child:
313 ■             Text(widget.stop == null ? 'Create stop' : 'Save'),
314 ■     ),
315 ■     const SizedBox(height: 8),
316 ■ ],
317 ■ ),
318 ■ ),
319 ■ ),
320 ■ ),
321 ■ );
322 ■ }
323 ■ }

```

lib/features/admin/screens/admin_monitoring_screen.dart

195 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■ import 'package:google_maps_flutter/google_maps_flutter.dart';
4 ■
5 ■ import '../core/constants/enums.dart';
6 ■ import '../core/theme/nmb_colors.dart';
7 ■ import '../core/theme/nmb_typography.dart';
8 ■ import '../core/utils/formatters.dart';
9 ■ import '../core/widgets/nmb_card.dart';
10 ■ import '../core/widgets/state_views.dart';
11 ■ import '../core/widgets/status_pill.dart';
12 ■ import '../models/bus.dart';
13 ■ import '../models/live_location.dart';
14 ■ import '../models/trip.dart';
15 ■ import '../providers/data_providers.dart';
16 ■ import '../shared/bus_marker_icon.dart';
17 ■
18 ■ /// Live fleet monitoring: all active buses on one map + trip status list.
19 ■ /// This screen is reachable only by admin (route fence + backend rules).
20 ■ class AdminMonitoringScreen extends ConsumerStatefulWidget {
21 ■   const AdminMonitoringScreen({super.key});
22 ■
23 ■   @override
24 ■   ConsumerState<AdminMonitoringScreen> createState() =>
25 ■     _AdminMonitoringScreenState();
26 ■ }
27 ■
28 ■ class _AdminMonitoringScreenState
29 ■   extends ConsumerState<AdminMonitoringScreen> {
30 ■   GoogleMapController? _controller;
31 ■   BitmapDescriptor? _busIcon;
32 ■
33 ■   @override
34 ■   void initState() {
35 ■     super.initState();
36 ■     BusMarkerIcon.load().then((BitmapDescriptor icon) {
37 ■       if (mounted) setState(() => _busIcon = icon);
38 ■     });
39 ■   }
40 ■
41 ■   @override
42 ■   Widget build(BuildContext context) {
43 ■     final AsyncValue<List<Trip>> tripsAsync = ref.watch(activeTripsProvider);
44 ■     final List<Bus> buses =
45 ■       ref.watch(allBusesProvider).valueOrNull ?? const <Bus>[];
46 ■
47 ■     return Scaffold(
48 ■       appBar: AppBar(title: const Text('Live Monitoring')),
49 ■       body: tripsAsync.when(
50 ■         loading: () => const LoadingView(),
51 ■         error: (Object e, _) => ErrorView(
52 ■           message: 'Couldn\'t load active trips.',
53 ■           onRetry: () => ref.invalidate(activeTripsProvider),
54 ■         ),
55 ■         data: (List<Trip> trips) {
56 ■           if (trips.isEmpty) {
57 ■             return const EmptyState(
58 ■               icon: Icons.radar_rounded,

```

```

59 ■         title: 'No active trips',
60 ■         message: 'When drivers start trips, their buses appear here '
61 ■             'in real time.',
62 ■     );
63 ■ }
64 ■
65 ■ final Set<Marker> markers = <Marker>{};
66 ■ LatLng? firstPos;
67 ■ for (final Trip trip in trips) {
68 ■     final LiveLocation? live = ref
69 ■         .watch(busLiveLocationProvider(trip.busId))
70 ■         .valueOrNull;
71 ■     if (live == null) continue;
72 ■     final LatLng pos =
73 ■         LatLng(live.position.lat, live.position.lng);
74 ■     firstPos ??= pos;
75 ■     Bus? bus;
76 ■     for (final Bus b in buses) {
77 ■         if (b.id == trip.busId) bus = b;
78 ■     }
79 ■     markers.add(
80 ■         Marker(
81 ■             markerId: MarkerId('bus_${trip.busId}'),
82 ■             position: pos,
83 ■             icon: _busIcon ??
84 ■                 BitmapDescriptor.defaultMarkerWithHue(
85 ■                     BitmapDescriptor.hueYellow,),
86 ■             rotation: live.heading,
87 ■             flat: true,
88 ■             anchor: const Offset(0.5, 0.5),
89 ■             infowindow: Infowindow(
90 ■                 title: bus?.busNumber ?? trip.busId,
91 ■                 snippet:
92 ■                     '${live.speedKmh.toStringAsFixed(0)} km/h • updated '
93 ■                     '${Formatters.relativeTime(live.updatedAt)}',
94 ■             ),
95 ■         ),
96 ■     );
97 ■ }
98 ■
99 ■ return Column(
100 ■     children: <Widget>[
101 ■         Expanded(
102 ■             flex: 3,
103 ■             child: GoogleMap(
104 ■                 initialCameraPosition: CameraPosition(
105 ■                     target: firstPos ?? const LatLng(30.3165, 78.0322),
106 ■                     zoom: 12,
107 ■                 ),
108 ■                 markers: markers,
109 ■                 zoomControlsEnabled: false,
110 ■                 onMapCreated: (GoogleMapController c) => _controller = c,
111 ■             ),
112 ■         ),
113 ■         Expanded(
114 ■             flex: 2,
115 ■             child: ListView(
116 ■                 padding: const EdgeInsets.all(16),

```

```

117 ■         children: <Widget>[
118 ■             for (final Trip trip in trips) ...<Widget>[
119 ■                 _TripStatusTile(
120 ■                     trip: trip,
121 ■                     buses: buses,
122 ■                     onFocus: (LatLng pos) => _controller?.animateCamera(
123 ■                         CameraUpdate.newLatLngZoom(pos, 15),
124 ■                     ),
125 ■                 ),
126 ■                 const SizedBox(height: 10),
127 ■             ],
128 ■         ],
129 ■     ),
130 ■ ),
131 ■ ],
132 ■ );
133 ■ },
134 ■ ),
135 ■ );
136 ■ }
137 ■ }
138 ■
139 ■ class _TripStatusTile extends ConsumerWidget {
140 ■     const _TripStatusTile({
141 ■         required this.trip,
142 ■         required this.buses,
143 ■         required this.onFocus,
144 ■     });
145 ■
146 ■     final Trip trip;
147 ■     final List<Bus> buses;
148 ■     final void Function(LatLng) onFocus;
149 ■
150 ■     @override
151 ■     Widget build(BuildContext context, WidgetRef ref) {
152 ■         final LiveLocation? live =
153 ■             ref.watch(busLiveLocationProvider(trip.busId)).valueOrNull;
154 ■         ref.watch(freshnessTickProvider);
155 ■         final LocationFreshness freshness =
156 ■             live?.freshness() ?? LocationFreshness.unavailable;
157 ■
158 ■         Bus? bus;
159 ■         for (final Bus b in buses) {
160 ■             if (b.id == trip.busId) bus = b;
161 ■         }
162 ■
163 ■         return NmbCard(
164 ■             onTap: live == null
165 ■                 ? null
166 ■                 : () => onFocus(LatLng(live.position.lat, live.position.lng)),
167 ■             padding: const EdgeInsets.all(14),
168 ■             child: Row(
169 ■                 children: <Widget>[
170 ■                     const Icon(Icons.directions_bus_rounded,
171 ■                         color: NmbColors.primary,),
172 ■                     const SizedBox(width: 12),
173 ■                     Expanded(
174 ■                         child: Column(
175 ■                             crossAxisAlignment: CrossAxisAlignment.start,
176 ■                             children: <Widget>[
177 ■                                 Text(bus?.busNumber ?? trip.busId,
178 ■                                     style: NmbTypography.cardTitle,),
179 ■                                 Text(
180 ■                                     live == null
181 ■                                         ? '${trip.direction.label} • waiting for GPS...'
182 ■                                         : '${trip.direction.label} • '
183 ■                                             '${live.speedKmh.toStringAsFixed(0)} km/h • '
184 ■                                             '${Formatters.relativeTime(live.updatedAt)}',
185 ■                                     style: NmbTypography.bodySecondary,
186 ■                                 ),
187 ■                             ],
188 ■                         ),
189 ■                     ),
190 ■                     StatusPill.freshness(freshness),
191 ■                 ],
192 ■             ),
193 ■         );
194 ■     }
195 ■ }

```

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/theme/nmb_typography.dart';
5 ■ import '../core/utils/result.dart';
6 ■ import '../core/utils/validators.dart';
7 ■ import '../core/widgets/nmb_card.dart';
8 ■ import '../core/widgets/nmb_dialogs.dart';
9 ■ import '../core/widgets/responsive_scaffold_body.dart';
10 ■ import '../models/bus.dart';
11 ■ import '../providers/app_providers.dart';
12 ■ import '../providers/data_providers.dart';
13 ■
14 ■ /// Compose an announcement to everyone or to one bus's students.
15 ■ /// Delivery is server-side (Cloud Function): audiences are resolved from
16 ■ /// the database, so a client can never widen the recipient list.
17 ■ class AdminNotificationsScreen extends ConsumerStatefulWidget {
18 ■   const AdminNotificationsScreen({super.key});
19 ■
20 ■   @override
21 ■   ConsumerState<AdminNotificationsScreen> createState() =>
22 ■     _AdminNotificationsScreenState();
23 ■ }
24 ■
25 ■ class _AdminNotificationsScreenState
26 ■   extends ConsumerState<AdminNotificationsScreen> {
27 ■   final GlobalKey<FormState> _formKey = GlobalKey<FormState>();
28 ■   final TextEditingController _title = TextEditingController();
29 ■   final TextEditingController _body = TextEditingController();
30 ■   String _scope = 'all';
31 ■   String? _busId;
32 ■   bool _sending = false;
33 ■
34 ■   @override
35 ■   void dispose() {
36 ■     _title.dispose();
37 ■     _body.dispose();
38 ■     super.dispose();
39 ■   }
40 ■
41 ■   Future<void> _send() async {
42 ■     if (!(_formKey.currentState?.validate() ?? false)) return;
43 ■     if (_scope == 'bus' && _busId == null) {
44 ■       showNmbSnackBar(context, 'Choose a bus for a bus-specific announcement.',
45 ■         isError: true,);
46 ■       return;
47 ■     }
48 ■     final bool confirmed = await showNmbConfirmDialog(
49 ■       context,
50 ■       title: 'Send announcement?',
51 ■       message: _scope == 'all'
52 ■         ? 'This will notify every active user.'
53 ■         : 'This will notify all students and the driver of the selected '
54 ■           'bus.',
55 ■       confirmLabel: 'Send',
56 ■       icon: Icons.campaign_rounded,
57 ■     );
58 ■     if (!confirmed || !mounted) return;

```

```

59 ■
60 ■   setState(() => _sending = true);
61 ■   final Result<void> result =
62 ■       await ref.read(adminFunctionsServiceProvider).sendAnnouncement(
63 ■           title: _title.text.trim(),
64 ■           body: _body.text.trim(),
65 ■           scope: _scope,
66 ■           busId: _scope == 'bus' ? _busId : null,
67 ■       );
68 ■   if (!mounted) return;
69 ■   setState(() => _sending = false);
70 ■   result.when(
71 ■       ok: (_) {
72 ■         _title.clear();
73 ■         _body.clear();
74 ■         showNmbSnack(context, 'Announcement sent.', isSuccess: true);
75 ■       },
76 ■       err: (AppFailure f) => showNmbSnack(context, f.message, isError: true),
77 ■   );
78 ■ }
79 ■
80 ■ @override
81 ■ Widget build(BuildContext context) {
82 ■   final List<Bus> buses =
83 ■       ref.watch(allBusesProvider).valueOrNull ?? const <Bus>[];
84 ■
85 ■   return Scaffold(
86 ■       appBar: AppBar(title: const Text('Notifications')),
87 ■       body: ResponsiveBody(
88 ■         child: Form(
89 ■           key: _formKey,
90 ■           child: Column(
91 ■             crossAxisAlignment: CrossAxisAlignment.stretch,
92 ■             children: <Widget>[
93 ■               const Text('Send an announcement',
94 ■                 style: NmbTypography.sectionTitle,),
95 ■               const SizedBox(height: 12),
96 ■               NmbCard(
97 ■                 child: Column(
98 ■                   crossAxisAlignment: CrossAxisAlignment.stretch,
99 ■                   children: <Widget>[
100 ■                     TextFormField(
101 ■                       controller: _title,
102 ■                       decoration:
103 ■                         const InputDecoration(hintText: 'Title'),
104 ■                       maxLength: 80,
105 ■                       validator: (String? v) =>
106 ■                         Validators.requiredField(v, label: 'Title'),
107 ■                     ),
108 ■                     const SizedBox(height: 8),
109 ■                     TextFormField(
110 ■                       controller: _body,
111 ■                       decoration:
112 ■                         const InputDecoration(hintText: 'Message'),
113 ■                       maxLines: 4,
114 ■                       maxLength: 400,
115 ■                       validator: (String? v) =>
116 ■                         Validators.requiredField(v, label: 'Message'),

```

```

117 ■      ),
118 ■      const SizedBox(height: 12),
119 ■      SegmentedButton<String>(  

120 ■        segments: const <ButtonSegment<String>>[  

121 ■          ButtonSegment<String>(  

122 ■            value: 'all',  

123 ■            label: Text('Everyone'),  

124 ■            icon: Icon(Icons.groups_rounded),  

125 ■          ),  

126 ■          ButtonSegment<String>(  

127 ■            value: 'bus',  

128 ■            label: Text('One bus'),  

129 ■            icon: Icon(Icons.directions_bus_rounded),  

130 ■          ),  

131 ■        ],  

132 ■        selected: <String>{_scope},  

133 ■        onSelectionChanged: (Set<String> s) =>  

134 ■          setState(() => _scope = s.first),  

135 ■      ),  

136 ■      if (_scope == 'bus') ...<Widget>[  

137 ■        const SizedBox(height: 12),  

138 ■        DropdownButtonFormField<String>(  

139 ■          value: _busId,  

140 ■          decoration:  

141 ■            const InputDecoration(hintText: 'Select bus'),  

142 ■          items: <DropDownMenuItem<String>>[  

143 ■            for (final Bus b in buses)  

144 ■              DropDownMenuItem<String>(  

145 ■                value: b.id,  

146 ■                child: Text(b.busNumber),  

147 ■              ),  

148 ■          ],  

149 ■          onChanged: (String? v) =>  

150 ■            setState(() => _busId = v),  

151 ■        ),  

152 ■      ],  

153 ■      const SizedBox(height: 16),  

154 ■      FilledButton.icon(  

155 ■        onPressed: _sending ? null : _send,  

156 ■        icon: const Icon(Icons.send_rounded),  

157 ■        label: Text(  

158 ■          _sending ? 'Sending...' : 'Send announcement',),  

159 ■      ),  

160 ■    ],  

161 ■  ),  

162 ■ ),  

163 ■ const SizedBox(height: 16),  

164 ■ const NmbCard(  

165 ■   child: Text(  

166 ■     'Automatic notifications (trip started, bus approaching, '  

167 ■     'bus reached stop/school, tracking unavailable) are sent '  

168 ■     'by the system and don\'t need manual action.',  

169 ■     style: NmbTypography.bodySecondary,  

170 ■   ),  

171 ■ ),  

172 ■ const SizedBox(height: 24),  

173 ■ ],  

174 ■ ),  

175 ■ ),  

176 ■ ),  

177 ■ );  

178 ■ }  

179 ■ }

```

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/theme/nmb_colors.dart';
5 ■ import '../core/theme/nmb_typography.dart';
6 ■ import '../core/utils/formatters.dart';
7 ■ import '../core/widgets/nmb_card.dart';
8 ■ import '../core/widgets/responsive_scaffold_body.dart';
9 ■ import '../core/widgets/state_views.dart';
10 ■ import '../models/audit_log.dart';
11 ■ import '../providers/data_providers.dart';
12 ■
13 ■ /// Read-only audit trail. Entries are written exclusively by Cloud
14 ■ /// Functions – neither admins nor anyone else can edit or delete them
15 ■ /// from the client.
16 ■ class AdminAuditLogsScreen extends ConsumerWidget {
17 ■   const AdminAuditLogsScreen({super.key});
18 ■
19 ■   static const Map<String, IconData> _icons = <String, IconData>{
20 ■     'USER_PROVISIONED': Icons.person_add_alt_1_rounded,
21 ■     'STUDENT_BUS_REASSIGNED': Icons.swap_horiz_rounded,
22 ■     'DRIVER_BUS_REASSIGNED': Icons.swap_horiz_rounded,
23 ■     'ACCOUNT_DISABLED': Icons.block_rounded,
24 ■     'ACCOUNT_ENABLED': Icons.check_circle_outline_rounded,
25 ■     'TRIP_STARTED': Icons.play_circle_outline_rounded,
26 ■     'TRIP_ENDED': Icons.stop_circle_outlined,
27 ■     'TRIP_AUTO_CLOSED': Icons.timer_off_outlined,
28 ■     'ANNOUNCEMENT_SENT': Icons.campaign_rounded,
29 ■   };
30 ■
31 ■   @override
32 ■   Widget build(BuildContext context, WidgetRef ref) {
33 ■     final AsyncValue<List<AuditLog>> logsAsync = ref.watch(auditLogsProvider);
34 ■
35 ■     return Scaffold(
36 ■       appBar: AppBar(title: const Text('Activity Logs')),
37 ■       body: logsAsync.when(
38 ■         loading: () => const LoadingView(),
39 ■         error: (Object e, _) => ErrorView(
40 ■           message: 'Couldn\'t load activity logs.',
41 ■           onRetry: () => ref.invalidate(auditLogsProvider),
42 ■         ),
43 ■         data: (List<AuditLog> logs) {
44 ■           if (logs.isEmpty) {
45 ■             return const EmptyState(
46 ■               icon: Icons.receipt_long_outlined,
47 ■               title: 'No activity yet',
48 ■               message: 'Administrative actions and trip events will be '
49 ■                 'recorded here.',
50 ■             );
51 ■           }
52 ■           return ResponsiveBody(
53 ■             child: Column(
54 ■               crossAxisAlignment: CrossAxisAlignment.stretch,
55 ■               children: <Widget>[
56 ■                 for (final AuditLog log in logs) ...<Widget>[
57 ■                   NmbCard(
58 ■                     padding: const EdgeInsets.all(14),

```

```

59 ■      child: Row(
60 ■        crossAxisAlignment: CrossAxisAlignment.start,
61 ■        children: <Widget>[
62 ■          Icon(
63 ■            _icons[log.action] ?? Icons.info_outline_rounded,
64 ■            size: 22,
65 ■            color: NmbColors.primary,
66 ■          ),
67 ■          const SizedBox(width: 12),
68 ■          Expanded(
69 ■            child: Column(
70 ■              crossAxisAlignment: CrossAxisAlignment.start,
71 ■              children: <Widget>[
72 ■                Text(
73 ■                  log.action.replaceAll('_', ' '),
74 ■                  style: NmbTypography.cardTitle,
75 ■                ),
76 ■                Text(
77 ■                  '${log.targetType} ${log.targetId}'
78 ■                  '${log.details.isEmpty ? '' : ' • ${log.details}'}',
79 ■                  style: NmbTypography.bodySecondary,
80 ■                  maxLines: 2,
81 ■                  overflow: TextOverflow.ellipsis,
82 ■                ),
83 ■                const SizedBox(height: 4),
84 ■                Text(
85 ■                  'by ${log.actorRole} • '
86 ■                  '${Formatters.dateTime(log.at)}',
87 ■                  style: NmbTypography.caption,
88 ■                ),
89 ■              ],
90 ■            ),
91 ■          ),
92 ■        ],
93 ■      ),
94 ■    ),
95 ■    const SizedBox(height: 8),
96 ■  ],
97 ■  const SizedBox(height: 16),
98 ■ ],
99 ■ ),
100 ■ );
101 ■ },
102 ■ ),
103 ■ );
104 ■ }
105 ■ }

```

```

1 ■ import 'package:cloud_firestore/cloud_firestore.dart';
2 ■ import 'package:flutter/material.dart';
3 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
4 ■
5 ■ import '../core/theme/nmb_typography.dart';
6 ■ import '../core/utils/validators.dart';
7 ■ import '../core/widgets/nmb_card.dart';
8 ■ import '../core/widgets/nmb_dialogs.dart';
9 ■ import '../core/widgets/responsive_scaffold_body.dart';
10 ■ import '../providers/data_providers.dart';
11 ■
12 ■ /// School profile settings (contact card shown to students) – admin only.
13 ■ class AdminSettingsScreen extends ConsumerStatefulWidget {
14 ■   const AdminSettingsScreen({super.key});
15 ■
16 ■   @override
17 ■   ConsumerState<AdminSettingsScreen> createState() =>
18 ■     _AdminSettingsScreenState();
19 ■ }
20 ■
21 ■ class _AdminSettingsScreenState extends ConsumerState<AdminSettingsScreen> {
22 ■   final GlobalKey<FormState> _formKey = GlobalKey<FormState>();
23 ■   final TextEditingController _name = TextEditingController();
24 ■   final TextEditingController _phone = TextEditingController();
25 ■   final TextEditingController _email = TextEditingController();
26 ■   final TextEditingController _address = TextEditingController();
27 ■   final TextEditingController _lat = TextEditingController();
28 ■   final TextEditingController _lng = TextEditingController();
29 ■   bool _loaded = false;
30 ■   bool _saving = false;
31 ■
32 ■   @override
33 ■   void dispose() {
34 ■     _name.dispose();
35 ■     _phone.dispose();
36 ■     _email.dispose();
37 ■     _address.dispose();
38 ■     _lat.dispose();
39 ■     _lng.dispose();
40 ■     super.dispose();
41 ■   }
42 ■
43 ■   void _fill(Map<String, dynamic>? config) {
44 ■     if (_loaded || config == null) return;
45 ■     _loaded = true;
46 ■     _name.text = (config['name'] as String?) ?? '';
47 ■     _phone.text = (config['phone'] as String?) ?? '';
48 ■     _email.text = (config['email'] as String?) ?? '';
49 ■     _address.text = (config['address'] as String?) ?? '';
50 ■     final Map<String, dynamic>? loc =
51 ■       (config['location'] as Map?)?.cast<String, dynamic>();
52 ■     if (loc != null) {
53 ■       _lat.text = '${loc['lat']}';
54 ■       _lng.text = '${loc['lng']}';
55 ■     }
56 ■   }
57 ■
58 ■   Future<void> _save() async {

```

```

59 ■ if (!(_formKey.currentState?.validate() ?? false)) return;
60 ■ setState(() => _saving = true);
61 ■ try {
62 ■   await FirebaseFirestore.instance
63 ■     .collection('config')
64 ■     .doc('school')
65 ■     .set(<String, dynamic>{
66 ■       'name': _name.text.trim(),
67 ■       'phone': _phone.text.trim(),
68 ■       'email': _email.text.trim(),
69 ■       'address': _address.text.trim(),
70 ■       if (_lat.text.trim().isNotEmpty && _lng.text.trim().isNotEmpty)
71 ■         'location': <String, double>{
72 ■           'lat': double.parse(_lat.text.trim()),
73 ■           'lng': double.parse(_lng.text.trim()),
74 ■         },
75 ■     }, SetOptions(merge: true),);
76 ■   if (!mounted) return;
77 ■   showNmbSnack(context, 'School profile saved.', isSuccess: true);
78 ■ } catch (_) {
79 ■   if (!mounted) return;
80 ■   showNmbSnack(context, 'Couldn\'t save. Try again.', isError: true);
81 ■ } finally {
82 ■   if (mounted) setState(() => _saving = false);
83 ■ }
84 ■ }
85 ■
86 ■ @override
87 ■ Widget build(BuildContext context) {
88 ■   _fill(ref.watch(schoolConfigProvider).valueOrNull);
89 ■
90 ■   return Scaffold(
91 ■     appBar: AppBar(title: const Text('Admin Settings')),
92 ■     body: ResponsiveBody(
93 ■       child: Form(
94 ■         key: _formKey,
95 ■         child: Column(
96 ■           crossAxisAlignment: CrossAxisAlignment.stretch,
97 ■           children: <Widget>[
98 ■             const Text('School profile', style: NmbTypography.sectionTitle),
99 ■             const SizedBox(height: 6),
100 ■             const Text(
101 ■               'Shown to students and drivers as the school contact card.',
102 ■               style: NmbTypography.bodySecondary,
103 ■             ),
104 ■             const SizedBox(height: 14),
105 ■             NmbCard(
106 ■               child: Column(
107 ■                 children: <Widget>[
108 ■                   TextFormField(
109 ■                     controller: _name,
110 ■                     decoration:
111 ■                       const InputDecoration(hintText: 'School name'),
112 ■                     validator: (String? v) =>
113 ■                       Validators.requiredField(v, label: 'School name'),
114 ■                   ),
115 ■                   const SizedBox(height: 12),
116 ■                   TextFormField(

```

```

117 ■         controller: _phone,
118 ■         keyboardType: TextInputType.phone,
119 ■         decoration:
120 ■             const InputDecoration(hintText: 'Office phone'),
121 ■     ),
122 ■     const SizedBox(height: 12),
123 ■     TextFormField(
124 ■         controller: _email,
125 ■         keyboardType: TextInputType.emailAddress,
126 ■         decoration:
127 ■             const InputDecoration(hintText: 'Office email'),
128 ■     ),
129 ■     const SizedBox(height: 12),
130 ■     TextFormField(
131 ■         controller: _address,
132 ■         decoration:
133 ■             const InputDecoration(hintText: 'Address'),
134 ■     ),
135 ■     const SizedBox(height: 12),
136 ■     Row(
137 ■         children: <Widget>[
138 ■             Expanded(
139 ■                 child: TextFormField(
140 ■                     controller: _lat,
141 ■                     keyboardType:
142 ■                         const TextInputType.numberWithOptions(
143 ■                             decimal: true, signed: true,),
144 ■                     decoration: const InputDecoration(
145 ■                         hintText: 'School latitude',),
146 ■                 ),
147 ■             ),
148 ■             const SizedBox(width: 12),
149 ■             Expanded(
150 ■                 child: TextFormField(
151 ■                     controller: _lng,
152 ■                     keyboardType:
153 ■                         const TextInputType.numberWithOptions(
154 ■                             decimal: true, signed: true,),
155 ■                     decoration: const InputDecoration(
156 ■                         hintText: 'School longitude',),
157 ■                 ),
158 ■             ),
159 ■         ],
160 ■     ),
161 ■ ],
162 ■ ),
163 ■ ),
164 ■ const SizedBox(height: 16),
165 ■ FilledButton(
166 ■     onPressed: _saving ? null : _save,
167 ■     child: Text(_saving ? 'Saving...' : 'Save school profile'),
168 ■ ),
169 ■ const SizedBox(height: 24),
170 ■ ],
171 ■ ),
172 ■ ),
173 ■ ),
174 ■ );

```

```

175 ■ }
176 ■ }

```


[illegible]

firebase/database.rules.json

16 lines

```

1 ■ {
2 ■   "rules": {
3 ■     ".read": false,
4 ■     ".write": false,
5 ■
6 ■     "liveLocations": {
7 ■       "$busId": {
8 ■         ".read": "auth != null && auth.token.active === true && (auth.token.role === 'admin' ||
9 ■           ((auth.token.role === 'student' || auth.token.role === 'driver') && auth.token.busId === $busId
10 ■         d))",
11 ■
12 ■         ".write": "auth != null && auth.token.active === true && auth.token.role === 'driver' &&
13 ■           auth.token.busId === $busId",
14 ■
15 ■         ".validate": "!newData.exists() || (newData.hasChildren(['lat','lng','tripId','updatedAt
16 ■           ']) && newData.child('lat').isNumber() && newData.child('lat').val() >= -90 && newData.child('
17 ■           lat').val() <= 90 && newData.child('lng').isNumber() && newData.child('lng').val() >= -180 &&
18 ■           newData.child('lng').val() <= 180 && newData.child('tripId').isString() && newData.child('upda
19 ■           tedAt').val() === now)"
20 ■       }
21 ■     }
22 ■   }
23 ■ }

```

firebase/firestore.indexes.json

54 lines

```

1 ■ {
2 ■   "indexes": [
3 ■     {
4 ■       "collectionGroup": "users",
5 ■       "queryScope": "COLLECTION",
6 ■       "fields": [
7 ■         { "fieldPath": "role", "order": "ASCENDING" },
8 ■         { "fieldPath": "fullName", "order": "ASCENDING" }
9 ■       ]
10 ■     },
11 ■     {
12 ■       "collectionGroup": "users",
13 ■       "queryScope": "COLLECTION",
14 ■       "fields": [
15 ■         { "fieldPath": "role", "order": "ASCENDING" },
16 ■         { "fieldPath": "busId", "order": "ASCENDING" }
17 ■       ]
18 ■     },
19 ■     {
20 ■       "collectionGroup": "users",
21 ■       "queryScope": "COLLECTION",
22 ■       "fields": [
23 ■         { "fieldPath": "role", "order": "ASCENDING" },
24 ■         { "fieldPath": "stopId", "order": "ASCENDING" }
25 ■       ]
26 ■     },
27 ■     {
28 ■       "collectionGroup": "stops",
29 ■       "queryScope": "COLLECTION",
30 ■       "fields": [
31 ■         { "fieldPath": "busId", "order": "ASCENDING" },
32 ■         { "fieldPath": "isActive", "order": "ASCENDING" },
33 ■         { "fieldPath": "order", "order": "ASCENDING" }
34 ■       ]
35 ■     },
36 ■     {
37 ■       "collectionGroup": "trips",
38 ■       "queryScope": "COLLECTION",
39 ■       "fields": [
40 ■         { "fieldPath": "busId", "order": "ASCENDING" },
41 ■         { "fieldPath": "status", "order": "ASCENDING" }
42 ■       ]
43 ■     },
44 ■     {
45 ■       "collectionGroup": "routes",
46 ■       "queryScope": "COLLECTION",
47 ■       "fields": [
48 ■         { "fieldPath": "busId", "order": "ASCENDING" },
49 ■         { "fieldPath": "isActive", "order": "ASCENDING" }
50 ■       ]
51 ■     }
52 ■   ],
53 ■   "fieldOverrides": []
54 ■ }

```

firebase/firebase.json

20 lines

```
1 ■ {
2 ■   "firestore": {
3 ■     "rules": "firestore.rules",
4 ■     "indexes": "firestore.indexes.json"
5 ■   },
6 ■   "database": {
7 ■     "rules": "database.rules.json"
8 ■   },
9 ■   "functions": {
10 ■     "source": "functions",
11 ■     "runtime": "nodejs20"
12 ■   },
13 ■   "emulators": {
14 ■     "auth": { "port": 9099 },
15 ■     "firestore": { "port": 8080 },
16 ■     "database": { "port": 9000 },
17 ■     "functions": { "port": 5001 },
18 ■     "ui": { "enabled": true }
19 ■   }
20 ■ }
```

12. Cloud Functions (TypeScript)

Privileged backend: provisioning, claims, assignment, announcements, geofence alerts, stale-trip sweeper, audit logging.

index.ts · package.json · tsconfig.json

firebase/functions/src/index.ts

537 lines

```

1  ■ /**
2  ■  * NEVER MISS BUS — privileged backend.
3  ■  *
4  ■  * Everything security-critical lives here, running with the Admin SDK:
5  ■  *   - account provisioning + custom-claim management (roles, bus binding)
6  ■  *   - assignment changes with refresh-token revocation
7  ■  *   - account enable/disable
8  ■  *   - notification fan-out to explicit FCM tokens (no topics)
9  ■  *   - geofence "bus approaching / reached stop / reached school" alerts
10 ■  *   - stale-trip sweeper (driver phone died mid-trip)
11 ■  *   - tamper-proof audit logging
12 ■  *
13 ■  * No secrets are embedded: the Admin SDK uses the runtime service account.
14 ■  */
15 ■ import * as admin from "firebase-admin";
16 ■ import { onCall, HttpsError, CallableRequest } from "firebase-functions/v2/https";
17 ■ import { onDocumentWritten } from "firebase-functions/v2/firestore";
18 ■ import { onValueWritten } from "firebase-functions/v2/database";
19 ■ import { onSchedule } from "firebase-functions/v2/scheduler";
20 ■
21 ■ admin.initializeApp();
22 ■ const db = admin.firestore();
23 ■ const rtddb = admin.database();
24 ■
25 ■ type Role = "student" | "driver" | "admin";
26 ■
27 ■ // ████████████████████████████████████████████████████████████████████████████████
28 ■ // Helpers
29 ■ // ████████████████████████████████████████████████████████████████████████████████
30 ■
31 ■ /** Rejects unless the caller's verified ID token carries the admin claim. */
32 ■ function requireAdmin(req: CallableRequest): string {
33 ■   const auth = req.auth;
34 ■   if (!auth) throw new HttpsError("unauthenticated", "Sign in required.");
35 ■   if (auth.token.role !== "admin" || auth.token.active !== true) {
36 ■     throw new HttpsError("permission-denied", "Admin access required.");
37 ■   }
38 ■   return auth.uid;
39 ■ }
40 ■
41 ■ function requireString(v: unknown, field: string, maxLen = 200): string {
42 ■   if (typeof v !== "string" || v.trim().length === 0 || v.length > maxLen) {
43 ■     throw new HttpsError("invalid-argument", `Invalid ${field}.`);
44 ■   }
45 ■   return v.trim();
46 ■ }
47 ■
48 ■ function optionalString(v: unknown, field: string, maxLen = 200): string | null {
49 ■   if (v === undefined || v === null || v === "") return null;
50 ■   return requireString(v, field, maxLen);
51 ■ }
52 ■
53 ■ async function audit(
54 ■   actorUid: string,
55 ■   actorRole: string,
56 ■   action: string,
57 ■   target: { type: string; id: string },
58 ■   details: Record<string, unknown> = {},

```

```

59 ■ ): Promise<void> {
60 ■   await db.collection("auditLogs").add({
61 ■     actorUid,
62 ■     actorRole,
63 ■     action,
64 ■     target,
65 ■     details,
66 ■     at: admin.firestore.FieldValue.serverTimestamp(),
67 ■   });
68 ■ }
69 ■
70 ■ /** Sends a push to explicit tokens and writes per-user inbox copies. */
71 ■ async function notifyUsers(
72 ■   uids: string[],
73 ■   payload: { type: string; title: string; body: string },
74 ■ ): Promise<void> {
75 ■   if (uids.length === 0) return;
76 ■   const userDocs = await Promise.all(
77 ■     uids.map(uid => db.collection("users").doc(uid).get()),
78 ■   );
79 ■
80 ■   const tokens: string[] = [];
81 ■   const batch = db.batch();
82 ■   const now = admin.firestore.FieldValue.serverTimestamp();
83 ■
84 ■   for (const doc of userDocs) {
85 ■     if (!doc.exists) continue;
86 ■     const data = doc.data()!;
87 ■     if (data.isActive === false) continue;
88 ■
89 ■     // Per-user inbox copy (private; readable only by that user).
90 ■     const inboxRef = db
91 ■       .collection("users").doc(doc.id)
92 ■       .collection("inbox").doc();
93 ■     batch.set(inboxRef, { ...payload, read: false, receivedAt: now });
94 ■
95 ■     const fcmTokens = data.fcmTokens as Record<string, unknown> | undefined;
96 ■     if (fcmTokens) tokens.push(...Object.keys(fcmTokens));
97 ■   }
98 ■   await batch.commit();
99 ■
100 ■   // FCM allows 500 tokens per multicast call.
101 ■   for (let i = 0; i < tokens.length; i += 500) {
102 ■     const chunk = tokens.slice(i, i + 500);
103 ■     try {
104 ■       const res = await admin.messaging().sendEachForMulticast({
105 ■         tokens: chunk,
106 ■         notification: { title: payload.title, body: payload.body },
107 ■         data: { type: payload.type },
108 ■         android: { priority: "high" },
109 ■         apns: { payload: { aps: { sound: "default" } } },
110 ■       });
111 ■       // Prune dead tokens.
112 ■       const dead: string[] = [];
113 ■       res.responses.forEach((r, idx) => {
114 ■         if (
115 ■           !r.success &&
116 ■           (r.error?.code === "messaging/registration-token-not-registered" ||

```

```

117 r.error?.code === "messaging/invalid-registration-token")
118 ) {
119   dead.push(chunk[idx]);
120 }
121 });
122 if (dead.length > 0) {
123   for (const doc of userDocs) {
124     if (!doc.exists) continue;
125     const updates: Record<string, unknown> = {};
126     for (const t of dead) {
127       if ((doc.data()!.fcmTokens ?? {})[t] !== undefined) {
128         updates[`fcmTokens.${t}`] = admin.firestore.FieldValue.delete();
129       }
130     }
131     if (Object.keys(updates).length > 0) await doc.ref.update(updates);
132   }
133 }
134 } catch (e) {
135   console.error("FCM multicast failed", e);
136 }
137 }
138 }
139
140 async function studentsOfBus(busId: string): Promise<string[]> {
141   const snap = await db
142     .collection("users")
143     .where("role", "==", "student")
144     .where("busId", "==", busId)
145     .get();
146   return snap.docs.map((d) => d.id);
147 }
148
149 // =====
150 // 1. provisionUser - admin creates student/driver/admin accounts
151 // =====
152 export const provisionUser = onCall(async (req) => {
153   const adminUid = requireAdmin(req);
154
155   const email = requireString(req.data.email, "email");
156   const fullName = requireString(req.data.fullName, "fullName");
157   const role = requireString(req.data.role, "role") as Role;
158   const password = requireString(req.data.temporaryPassword, "password", 128);
159   const busId = optionalString(req.data.busId, "busId");
160   const stopId = optionalString(req.data.stopId, "stopId");
161   const classSection = optionalString(req.data.classSection, "classSection", 20);
162   const phone = optionalString(req.data.phone, "phone", 20);
163
164   if (!["student", "driver", "admin"].includes(role)) {
165     throw new HttpsError("invalid-argument", "Invalid role.");
166   }
167   if (password.length < 10) {
168     throw new HttpsError("invalid-argument", "Password must be ≥10 characters.");
169   }
170   if (busId) {
171     const bus = await db.collection("buses").doc(busId).get();
172     if (!bus.exists) throw new HttpsError("invalid-argument", "Bus not found.");
173   }
174 }

```

```

175 # const user = await admin.auth().createUser({
176 #   email,
177 #   password,
178 #   displayName: fullName,
179 # });
180 #
181 # const claims: Record<string, unknown> = { role, active: true };
182 # if (busId) claims.busId = busId;
183 # await admin.auth().setCustomUserClaims(user.uid, claims);
184 #
185 # await db.collection("users").doc(user.uid).set({
186 #   role,
187 #   fullName,
188 #   email,
189 #   busId: busId ?? null,
190 #   stopId: role === "student" ? stopId ?? null : null,
191 #   classSection: role === "student" ? classSection ?? null : null,
192 #   phone: role !== "student" ? phone ?? null : null,
193 #   isActive: true,
194 #   fcmTokens: {},
195 #   settings: {},
196 #   createdAt: admin.firestore.FieldValue.serverTimestamp(),
197 #   updatedAt: admin.firestore.FieldValue.serverTimestamp(),
198 # });
199 #
200 # await audit(adminUid, "admin", "USER_PROVISIONED",
201 #   { type: "user", id: user.uid }, { role, busId });
202 #
203 # return { uid: user.uid };
204 # });
205 #
206 # //
207 # // 2. assignUserToBus – student re-assignment (claims + doc + revocation)
208 # //
209 # export const assignUserToBus = onCall(async (req) => {
210 #   const adminUid = requireAdmin(req);
211 #   const uid = requireString(req.data.uid, "uid");
212 #   const busId = optionalString(req.data.busId, "busId");
213 #   const stopId = optionalString(req.data.stopId, "stopId");
214 #
215 #   const userDoc = await db.collection("users").doc(uid).get();
216 #   if (!userDoc.exists) throw new HttpsError("not-found", "User not found.");
217 #   const role = userDoc.data()?.role as Role;
218 #   if (role !== "student") {
219 #     throw new HttpsError("invalid-argument", "Use assignDriverToBus for drivers.");
220 #   }
221 #   if (busId) {
222 #     const bus = await db.collection("buses").doc(busId).get();
223 #     if (!bus.exists) throw new HttpsError("invalid-argument", "Bus not found.");
224 #   }
225 #   if (stopId) {
226 #     const stop = await db.collection("stops").doc(stopId).get();
227 #     if (!stop.exists || stop.data()?.busId !== busId) {
228 #       throw new HttpsError("invalid-argument", "Stop does not belong to that bus.");
229 #     }
230 #   }
231 #
232 #   const existing = (await admin.auth().getUser(uid)).customClaims ?? {};

```

```

233   await admin.auth().setCustomUserClaims(uid, {
234     ...existing,
235     role: "student",
236     busId: busId ?? null,
237   });
238   await db.collection("users").doc(uid).update({
239     busId: busId ?? null,
240     stopId: stopId ?? null,
241     updatedAt: admin.firestore.FieldValue.serverTimestamp(),
242   });
243
244   // Old ID tokens (with old busId claim) die now.
245   await admin.auth().revokeRefreshTokens(uid);
246
247   await audit(adminUid, "admin", "STUDENT_BUS_REASSIGNED",
248     { type: "user", id: uid }, { busId, stopId });
249   return { ok: true };
250 });
251
252 // =====
253 // 3. assignDriverToBus – one driver per bus, enforced transactionally
254 // =====
255 export const assignDriverToBus = onCall(async (req) => {
256   const adminUid = requireAdmin(req);
257   const uid = requireString(req.data.uid, "uid");
258   const busId = optionalString(req.data.busId, "busId");
259
260   const userDoc = await db.collection("users").doc(uid).get();
261   if (!userDoc.exists || userDoc.data()?.role !== "driver") {
262     throw new HttpsError("invalid-argument", "Driver account not found.");
263   }
264
265   await db.runTransaction(async (tx) => {
266     if (busId) {
267       const busRef = db.collection("buses").doc(busId);
268       const bus = await tx.get(busRef);
269       if (!bus.exists) throw new HttpsError("invalid-argument", "Bus not found.");
270       const currentDriver = bus.data()?.driverId as string | null;
271       if (currentDriver && currentDriver !== uid) {
272         // Unassign previous driver's doc; claims fixed below via revocation.
273         tx.update(db.collection("users").doc(currentDriver), { busId: null });
274       }
275       tx.update(busRef, { driverId: uid });
276     }
277     // Clear driver from any bus that previously pointed at them.
278     const previous = await db
279       .collection("buses").where("driverId", "==", uid).get();
280     previous.docs.forEach((d) => {
281       if (d.id !== busId) tx.update(d.ref, { driverId: null });
282     });
283     tx.update(db.collection("users").doc(uid), {
284       busId: busId ?? null,
285       updatedAt: admin.firestore.FieldValue.serverTimestamp(),
286     });
287   });
288
289   const existing = (await admin.auth().getUser(uid)).customClaims ?? {};
290   await admin.auth().setCustomUserClaims(uid, {

```

```

291   ...existing,
292   role: "driver",
293   busId: busId ?? null,
294   });
295   await admin.auth().revokeRefreshTokens(uid);
296
297   await audit(adminUid, "admin", "DRIVER_BUS_REASSIGNED",
298     { type: "user", id: uid }, { busId });
299   return { ok: true };
300   });
301
302   // =====
303   // 4. setAccountActive – enable/disable with immediate effect
304   // =====
305   export const setAccountActive = onCall(async (req) => {
306     const adminUid = requireAdmin(req);
307     const uid = requireString(req.data.uid, "uid");
308     const active = req.data.active === true;
309
310     if (uid === adminUid && !active) {
311       throw new HttpsError("failed-precondition", "You cannot disable your own account.");
312     }
313
314     const existing = (await admin.auth().getUser(uid)).customClaims ?? {};
315     await admin.auth().updateUser(uid, { disabled: !active });
316     await admin.auth().setCustomUserClaims(uid, { ...existing, active });
317     await admin.auth().revokeRefreshTokens(uid);
318     await db.collection("users").doc(uid).update({
319       isActive: active,
320       updatedAt: admin.firestore.FieldValue.serverTimestamp(),
321     });
322
323     await audit(adminUid, "admin", active ? "ACCOUNT_ENABLED" : "ACCOUNT_DISABLED",
324       { type: "user", id: uid });
325     return { ok: true };
326   });
327
328   // =====
329   // 5. sendAnnouncement – audience resolved SERVER-side
330   // =====
331   export const sendAnnouncement = onCall(async (req) => {
332     const adminUid = requireAdmin(req);
333     const title = requireString(req.data.title, "title", 80);
334     const body = requireString(req.data.body, "body", 400);
335     const scope = requireString(req.data.scope, "scope", 10);
336
337     let uids: string[] = [];
338     if (scope === "all") {
339       const snap = await db.collection("users")
340         .where("isActive", "==", true).get();
341       uids = snap.docs.map((d) => d.id);
342     } else if (scope === "bus") {
343       const busId = requireString(req.data.busId, "busId");
344       uids = await studentsOfBus(busId);
345       const bus = await db.collection("buses").doc(busId).get();
346       const driverId = bus.data()?.driverId as string | undefined;
347       if (driverId) uids.push(driverId);
348     } else if (scope === "user") {

```

```

349   uids = [requireString(req.data.uid, "uid")];
350 } else {
351   throw new HttpsError("invalid-argument", "Invalid scope.");
352 }
353
354 await db.collection("notifications").add({
355   type: "announcement", title, body,
356   audience: { scope, busId: req.data.busId ?? null, uid: req.data.uid ?? null },
357   createdBy: adminUid,
358   createdAt: admin.firestore.FieldValue.serverTimestamp(),
359 });
360 await notifyUsers(uids, { type: "announcement", title, body });
361 await audit(adminUid, "admin", "ANNOUNCEMENT_SENT",
362   { type: "notification", id: scope }, { title });
363 return { ok: true };
364 });
365
366 // =====
367 // 6. onTripWrite – trip started/ended notifications + audit
368 // =====
369 export const onTripWrite = onDocumentWritten("trips/{tripId}", async (event) => {
370   const before = event.data?.before?.data();
371   const after = event.data?.after?.data();
372   if (!after) return;
373
374   const busId = after.busId as string;
375   const direction = after.direction === "drop" ? "drop-off" : "pickup";
376
377   // Trip started.
378   if (!before && after.status === "active") {
379     const uids = await studentsOfBus(busId);
380     await notifyUsers(uids, {
381       type: "tripStarted",
382       title: "Bus trip started",
383       body: `Your bus has started its ${direction} trip. Track it live now!`,
384     });
385     await audit(after.driverId, "driver", "TRIP_STARTED",
386       { type: "trip", id: event.params.tripId }, { busId });
387     return;
388   }
389
390   // Trip ended.
391   if (before?.status === "active" && after.status !== "active") {
392     // Defensive cleanup: remove live node even if the client failed to.
393     await rtodb.ref(`liveLocations/${busId}`).remove().catch(() => undefined);
394     await audit(after.driverId ?? "system",
395       after.status === "autoClosed" ? "system" : "driver",
396       after.status === "autoClosed" ? "TRIP_AUTO_CLOSED" : "TRIP_ENDED",
397       { type: "trip", id: event.params.tripId }, { busId });
398   }
399 });
400
401 // =====
402 // 7. onLocationWrite – geofence alerts (approaching / reached / school)
403 // =====
404 const APPROACH_METERS = 800;
405 const REACHED_METERS = 120;
406

```

```

407 ■ function haversine(aLat: number, aLng: number, bLat: number, bLng: number): number {
408 ■   const R = 6371000;
409 ■   const dLat = ((bLat - aLat) * Math.PI) / 180;
410 ■   const dLng = ((bLng - aLng) * Math.PI) / 180;
411 ■   const s =
412 ■     Math.sin(dLat / 2) ** 2 +
413 ■     Math.cos((aLat * Math.PI) / 180) *
414 ■     Math.cos((bLat * Math.PI) / 180) *
415 ■     Math.sin(dLng / 2) ** 2;
416 ■   return 2 * R * Math.atan2(Math.sqrt(s), Math.sqrt(1 - s));
417 ■ }
418 ■
419 ■ export const onLocationWrite = onValueWritten(
420 ■   { ref: "/liveLocations/{busId}" },
421 ■   async (event) => {
422 ■     const after = event.data.after.val();
423 ■     if (!after) return;
424 ■     const busId = event.params.busId;
425 ■     const { lat, lng, tripId } = after as { lat: number; lng: number; tripId: string };
426 ■     if (typeof lat !== "number" || typeof lng !== "number" || !tripId) return;
427 ■
428 ■     const tripRef = db.collection("trips").doc(tripId);
429 ■     const trip = await tripRef.get();
430 ■     if (!trip.exists || trip.data()?.status !== "active") return;
431 ■     const alreadyNotified: Record<string, string> =
432 ■       trip.data()?.geofenceState ?? {};
433 ■
434 ■     const stopsSnap = await db
435 ■       .collection("stops")
436 ■       .where("busId", "==", busId)
437 ■       .where("isActive", "==", true)
438 ■       .get();
439 ■
440 ■     const updates: Record<string, string> = {};
441 ■     for (const stopDoc of stopsSnap.docs) {
442 ■       const stop = stopDoc.data();
443 ■       const dist = haversine(lat, lng, stop.location.lat, stop.location.lng);
444 ■       const state = alreadyNotified[stopDoc.id];
445 ■
446 ■       if (dist <= REACHED_METERS && state !== "reached") {
447 ■         updates[stopDoc.id] = "reached";
448 ■         const students = await db
449 ■           .collection("users")
450 ■           .where("role", "==", "student")
451 ■           .where("stopId", "==", stopDoc.id)
452 ■           .get();
453 ■         await notifyUsers(students.docs.map((d) => d.id), {
454 ■           type: "busReachedStop",
455 ■           title: "Bus at your stop ■",
456 ■           body: `The bus has reached ${stop.name}.`,
457 ■         });
458 ■       } else if (dist <= APPROACH_METERS && !state) {
459 ■         updates[stopDoc.id] = "approaching";
460 ■         const students = await db
461 ■           .collection("users")
462 ■           .where("role", "==", "student")
463 ■           .where("stopId", "==", stopDoc.id)
464 ■           .get();

```

```

465   await notifyUsers(students.docs.map((d) => d.id), {
466     type: "busApproaching",
467     title: "Bus approaching ■",
468     body: `Your bus is close to ${stop.name}. Please be ready!`,
469   });
470 }
471 }
472
473 // School arrival (pickup trips).
474 const routeSnap = await db
475   .collection("routes")
476   .where("busId", "==", busId)
477   .where("isActive", "==", true)
478   .limit(1)
479   .get();
480 if (!routeSnap.empty && !alreadyNotified["__school"]) {
481   const school = routeSnap.docs[0].data().schoolLocation;
482   if (
483     school &&
484     haversine(lat, lng, school.lat, school.lng) <= REACHED_METERS &&
485     trip.data().direction === "pickup"
486   ) {
487     updates["__school"] = "reached";
488     const uids = await studentsOfBus(busId);
489     await notifyUsers(uids, {
490       type: "busReachedSchool",
491       title: "Bus reached school ■",
492       body: "The bus has arrived at SRBS International School.",
493     });
494   }
495 }
496
497 if (Object.keys(updates).length > 0) {
498   const merged: Record<string, string> = { ...alreadyNotified, ...updates };
499   await tripRef.update({ geofenceState: merged });
500 }
501 },
502 );
503
504 // ████████████████████████████████████████████████████████████████████████████████
505 // 8. sweepStaleTrips – driver phone died / app killed mid-trip
506 // ████████████████████████████████████████████████████████████████████████████████
507 export const sweepStaleTrips = onSchedule("every 5 minutes", async () => {
508   const STALE_MS = 15 * 60 * 1000;
509   const active = await db
510     .collection("trips").where("status", "=", "active").get();
511
512   for (const tripDoc of active.docs) {
513     const busId = tripDoc.data().busId as string;
514     const snap = await rtDb.ref(`liveLocations/${busId}`).get();
515     const last = snap.exists() ? (snap.val().updatedAt as number) : 0;
516     const startedAt =
517       (tripDoc.data().startedAt as admin.firestore.Timestamp | undefined)
518         ?.toMillis() ?? 0;
519     const reference = Math.max(last, startedAt);
520
521     if (Date.now() - reference > STALE_MS) {
522       await tripDoc.ref.update({
523         status: "autoClosed",
524         endedAt: admin.firestore.FieldValue.serverTimestamp(),
525       });
526       await rtDb.ref(`liveLocations/${busId}`).remove().catch(() => undefined);
527       const uids = await studentsOfBus(busId);
528       await notifyUsers(uids, {
529         type: "trackingUnavailable",
530         title: "Tracking unavailable",
531         body:
532           "We lost contact with the bus and tracking has been paused. " +
533           "Please contact the school if you need help.",
534       });
535     }
536   }
537 });

```

```
1 ■ {
2 ■   "name": "never-miss-bus-functions",
3 ■   "description": "Never Miss Bus – privileged backend (Cloud Functions for Firebase)",
4 ■   "private": true,
5 ■   "engines": {
6 ■     "node": "20"
7 ■   },
8 ■   "main": "lib/index.js",
9 ■   "scripts": {
10 ■     "build": "tsc",
11 ■     "serve": "npm run build && firebase emulators:start --only functions",
12 ■     "deploy": "firebase deploy --only functions",
13 ■     "lint": "eslint src"
14 ■   },
15 ■   "dependencies": {
16 ■     "firebase-admin": "^12.6.0",
17 ■     "firebase-functions": "^6.1.0"
18 ■   },
19 ■   "devDependencies": {
20 ■     "typescript": "^5.6.0"
21 ■   }
22 ■ }
```

firebase/functions/tsconfig.json

16 lines

```
1 ■ {
2 ■   "compilerOptions": {
3 ■     "module": "commonjs",
4 ■     "target": "es2022",
5 ■     "lib": ["es2022"],
6 ■     "outDir": "lib",
7 ■     "rootDir": "src",
8 ■     "strict": true,
9 ■     "noImplicitReturns": true,
10 ■     "noUnusedLocals": true,
11 ■     "esModuleInterop": true,
12 ■     "sourceMap": true,
13 ■     "skipLibCheck": true
14 ■   },
15 ■   "include": ["src"]
16 ■ }
```

AndroidManifest.xml · build.gradle · proguard-rules.pro · AppDelegate.swift

70 lines

Page 161

```

58 ■
59 ■     <meta-data
60 ■         android:name="flutterEmbedding"
61 ■         android:value="2" />
62 ■     </application>
63 ■
64 ■     <queries>
65 ■         <intent>
66 ■             <action android:name="android.intent.action.PROCESS_TEXT"/>
67 ■             <data android:mimeType="text/plain"/>
68 ■         </intent>
69 ■     </queries>
70 ■ </manifest>

```

android/app/build.gradle

64 lines

```

1 ■ plugins {
2 ■     id "com.android.application"
3 ■     id "kotlin-android"
4 ■     // The Flutter Gradle Plugin must be applied after the Android and Kotlin Gradle plugins.
5 ■     id "dev.flutter.flutter-gradle-plugin"
6 ■     // ■■■■ FIREBASE – uncomment after adding android/app/google-services.json
7 ■     //     (generated by `flutterfire configure`):
8 ■     // id "com.google.gms.google-services"
9 ■ }
10 ■
11 ■ // Reads the Google Maps key from android/local.properties (never committed).
12 ■ //   MAPS_API_KEY=Aiza...
13 ■ def localProperties = new Properties()
14 ■ def localPropertiesFile = rootProject.file("local.properties")
15 ■ if (localPropertiesFile.exists()) {
16 ■     localPropertiesFile.withReader("UTF-8") { reader ->
17 ■         localProperties.load(reader)
18 ■     }
19 ■ }
20 ■ def mapsApiKey = localProperties.getProperty("MAPS_API_KEY") ?: "MAPS_KEY_NOT_CONFIGURED"
21 ■
22 ■ android {
23 ■     namespace = "com.srbs.never_miss_bus"
24 ■     compileSdk = flutter.compileSdkVersion
25 ■     ndkVersion = flutter.ndkVersion
26 ■
27 ■     compileOptions {
28 ■         sourceCompatibility = JavaVersion.VERSION_1_8
29 ■         targetCompatibility = JavaVersion.VERSION_1_8
30 ■     }
31 ■
32 ■     kotlinOptions {
33 ■         jvmTarget = JavaVersion.VERSION_1_8
34 ■     }
35 ■
36 ■     defaultConfig {
37 ■         applicationId = "com.srbs.never_miss_bus"
38 ■         // firebase_auth / cloud_firestore need at least 23; maps needs 21+.
39 ■         minSdk = 23
40 ■         targetSdk = flutter.targetSdkVersion
41 ■         versionCode = flutter.versionCode
42 ■         versionName = flutter.versionName
43 ■         multiDexEnabled = true
44 ■
45 ■         // Injects the Maps key into AndroidManifest ${MAPS_API_KEY}.
46 ■         manifestPlaceholders["MAPS_API_KEY"] = mapsApiKey
47 ■     }
48 ■
49 ■     buildTypes {
50 ■         release {
51 ■             // ■■■■ PRODUCTION SIGNING – create android/key.properties and a
52 ■             // proper signing config before publishing. Debug signing is used
53 ■             // here only so `flutter run --release` works during development.
54 ■             signingConfig = signingConfigs.debug
55 ■             minifyEnabled = true
56 ■             shrinkResources = true
57 ■             proguardFiles getDefaultProguardFile("proguard-android-optimize.txt"), "proguard-rul
58 ■         }
59 ■     }
60 ■ }
61 ■
62 ■ flutter {
63 ■     source = "../.."
64 ■ }

```

android/app/proguard-rules.pro

5 lines

```
1 ■ # Never Miss Bus – release shrink rules
2 ■ -keep class io.flutter.** { *; }
3 ■ -keep class com.google.firebase.** { *; }
4 ■ -keep class com.google.android.gms.maps.** { *; }
5 ■ -dontwarn com.google.errorprone.annotations.**
```

ios/Runner/AppDelegate.swift

29 lines

```

1 import Flutter
2 import UIKit
3 import GoogleMaps
4
5 @main
6 @objc class AppDelegate: FlutterAppDelegate {
7   override func application(
8     _ application: UIApplication,
9     didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey: Any]?
10  ) -> Bool {
11    // =====
12    // GOOGLE MAPS API KEY – REQUIRED CONFIGURATION
13    // Create an iOS-restricted key in Google Cloud Console (Maps SDK
14    // for iOS). Recommended: keep it OUT of source control by reading
15    // it from a git-ignored plist:
16    // ios/Runner/Secrets.plist → key "MAPS_API_KEY"
17    // =====
18    if let path = Bundle.main.path(forResource: "Secrets", ofType: "plist"),
19        let secrets = NSDictionary(contentsOfFile: path),
20        let mapsKey = secrets["MAPS_API_KEY"] as? String {
21      GMServices.provideAPIKey(mapsKey)
22    } else {
23      NSLog("WARNING: ios/Runner/Secrets.plist missing – Google Maps will not render. See docs/S
24  ETUP.md")
25    }
26
27    GeneratedPluginRegistrant.register(with: self)
28    return super.application(application, didFinishLaunchingWithOptions: launchOptions)
29  }
30 }

```

14. Tests

Unit tests: validators, freshness honesty, ETA guardrails, role parsing.

`unit_test.dart · widget_test.dart`

`test/unit_test.dart`

161 lines

```

1 ■ import 'package:flutter_test/flutter_test.dart';
2 ■ import 'package:never_miss_bus/core/constants/enums.dart';
3 ■ import 'package:never_miss_bus/core/utils/formatters.dart';
4 ■ import 'package:never_miss_bus/core/validators.dart';
5 ■ import 'package:never_miss_bus/models/bus_stop.dart';
6 ■ import 'package:never_miss_bus/models/geo_point_data.dart';
7 ■ import 'package:never_miss_bus/models/live_location.dart';
8 ■ import 'package:never_miss_bus/services/eta_service.dart';
9 ■
10 ■ void main() {
11 ■   group('Validators', () {
12 ■     test('accepts valid email', () {
13 ■       expect(Validators.email('student@srbs.edu.in'), isNull);
14 ■     });
15 ■     test('rejects invalid email', () {
16 ■       expect(Validators.email('not-an-email'), isNotNull);
17 ■       expect(Validators.email(''), isNotNull);
18 ■     });
19 ■     test('password length enforced', () {
20 ■       expect(Validators.password('short'), isNotNull);
21 ■       expect(Validators.password('longenough1'), isNull);
22 ■     });
23 ■     test('strong password requires upper + digit + length', () {
24 ■       expect(Validators.strongPassword('weakpassword'), isNotNull);
25 ■       expect(Validators.strongPassword('Weakpassword'), isNotNull);
26 ■       expect(Validators.strongPassword('Str0ngPassword'), isNull);
27 ■     });
28 ■     test('lat/lng bounds', () {
29 ■       expect(Validators.latitude('91'), isNotNull);
30 ■       expect(Validators.latitude('30.32'), isNull);
31 ■       expect(Validators.longitude('-181'), isNotNull);
32 ■       expect(Validators.longitude('78.03'), isNull);
33 ■     });
34 ■   });
35 ■
36 ■   group('GeoPointData', () {
37 ■     test('haversine distance is sane (Dehradun ~1km check)', () {
38 ■       const GeoPointData a = GeoPointData(lat: 30.3165, lng: 78.0322);
39 ■       const GeoPointData b = GeoPointData(lat: 30.3255, lng: 78.0322);
40 ■       final double d = a.distanceTo(b);
41 ■       expect(d, greaterThan(900));
42 ■       expect(d, lessThan(1100));
43 ■     });
44 ■   });
45 ■
46 ■   group('LiveLocation freshness (honest status)', () {
47 ■     LiveLocation fixAgedSeconds(int seconds) => LiveLocation(
48 ■       position: const GeoPointData(lat: 30, lng: 78),
49 ■       heading: 0,
50 ■       speedKmh: 20,
51 ■       accuracy: 5,
52 ■       tripId: 't1',
53 ■       updatedAt: DateTime.now().subtract(Duration(seconds: seconds)),
54 ■     );
55 ■
56 ■     test('fresh fix is LIVE', () {
57 ■       expect(fixAgedSeconds(5).freshness(), LocationFreshness.live);
58 ■     });

```

```

59 ■ test('45s old fix is STALE, never live', () {
60 ■     expect(fixAgedSeconds(45).freshness(), LocationFreshness.stale);
61 ■ });
62 ■ test('4min old fix is UNAVAILABLE', () {
63 ■     expect(fixAgedSeconds(240).freshness(), LocationFreshness.unavailable);
64 ■ });
65 ■ });
66 ■
67 ■ group('EtaService (never over-promises)', () {
68 ■     final List<BusStop> stops = <BusStop>[
69 ■         const BusStop(
70 ■             id: 's1',
71 ■             name: 'Stop 1',
72 ■             routeId: 'r',
73 ■             busId: 'b',
74 ■             location: GeoPointData(lat: 30.30, lng: 78.03),
75 ■             order: 1,
76 ■         ),
77 ■         const BusStop(
78 ■             id: 's2',
79 ■             name: 'Stop 2',
80 ■             routeId: 'r',
81 ■             busId: 'b',
82 ■             location: GeoPointData(lat: 30.32, lng: 78.03),
83 ■             order: 2,
84 ■         ),
85 ■     ];
86 ■
87 ■     test('no live data → unavailable', () {
88 ■         final EtaResult r = EtaService().etaToStop(
89 ■             live: null,
90 ■             targetStop: stops[1],
91 ■             orderedStops: stops,
92 ■         );
93 ■         expect(r, isA<EtaUnavailable>());
94 ■     });
95 ■
96 ■     test('stale data → unavailable (no fake ETA)', () {
97 ■         final LiveLocation stale = LiveLocation(
98 ■             position: const GeoPointData(lat: 30.29, lng: 78.03),
99 ■             heading: 0,
100 ■             speedKmh: 25,
101 ■             accuracy: 5,
102 ■             tripId: 't',
103 ■             updatedAt: DateTime.now().subtract(const Duration(minutes: 10)),
104 ■         );
105 ■         final EtaResult r = EtaService().etaToStop(
106 ■             live: stale,
107 ■             targetStop: stops[1],
108 ■             orderedStops: stops,
109 ■         );
110 ■         expect(r, isA<EtaUnavailable>());
111 ■     });
112 ■
113 ■     test('fresh data → approximate ETA with floored speed', () {
114 ■         final LiveLocation live = LiveLocation(
115 ■             position: const GeoPointData(lat: 30.29, lng: 78.03),
116 ■             heading: 0,

```

```

117 ■      speedKmh: 0, // stationary: min-speed floor must prevent ∞
118 ■      accuracy: 5,
119 ■      tripId: 't',
120 ■      updatedAt: DateTime.now(),
121 ■    );
122 ■    final EtaResult r = EtaService().etaToStop(
123 ■      live: live,
124 ■      targetStop: stops[1],
125 ■      orderedStops: stops,
126 ■    );
127 ■    expect(r, isA<EtaAvailable>());
128 ■    final EtaAvailable eta = r as EtaAvailable;
129 ■    expect(eta.duration.inMinutes, lessThan(60));
130 ■    expect(eta.duration.inSeconds, greaterThan(0));
131 ■  });
132 ■ };
133 ■
134 ■ group('Formatters', () {
135 ■   test('ETA wording is always approximate', () {
136 ■     expect(
137 ■       Formatters.eta(const Duration(minutes: 8)),
138 ■       contains('approximately'),
139 ■     );
140 ■     expect(Formatters.etaShort(const Duration(minutes: 8)), '~8 min');
141 ■   });
142 ■   test('relative time', () {
143 ■     expect(Formatters.relativeTime(DateTime.now()), 'just now');
144 ■     expect(
145 ■       Formatters.relativeTime(
146 ■         DateTime.now().subtract(const Duration(minutes: 5)),
147 ■       ),
148 ■       '5 minutes ago',
149 ■     );
150 ■   });
151 ■ });
152 ■
153 ■ group('Role parsing (no privilege from bad data)', () {
154 ■   test('unknown role parses to null, never defaults to admin', () {
155 ■     expect(UserRole.tryParse('superadmin'), isNull);
156 ■     expect(UserRole.tryParse(''), isNull);
157 ■     expect(UserRole.tryParse(null), isNull);
158 ■     expect(UserRole.tryParse('admin'), UserRole.admin);
159 ■   });
160 ■ });
161 ■ }

```

test/widget_test.dart

2 lines

```

1 ■ // Placeholder replaced by real tests in test/unit_test.dart.
2 ■ void main() {}

```